

TeacherSupporter

Trọn bộ công nghệ

Hướng dẫn tự học

Từ Spring Boot đến Kubernetes và CI/CD —
cầm nang trọn vẹn, dạy từ một dự án thật

Vuong Dang

viết cùng Claude

Bản tiếng Việt — tháng 9 năm 2026

Cách đọc cuốn sách này

Cuốn sách này dạy mọi công nghệ có trong dự án TeacherSupporter — Spring Boot và Spring Cloud, Postgres và MongoDB, Kafka, Docker, Kubernetes, và pipeline Jenkins gắn kết tất cả lại với nhau — bằng chính các file của dự án làm ví dụ. Không có gì được bịa ra cho trang sách: mọi cấu hình bạn sẽ đọc đều đang được triển khai, hoặc sắp được triển khai, trên một máy chủ gia đình thật hai nhân tên là ts-server.

Những quy tắc cuốn sách tuân theo

Khái niệm trước, cấu hình sau. Mỗi chương trước hết dạy ý tưởng bằng ngôn ngữ trung lập, không gắn với nhà cung cấp nào, rồi đọc file thật của dự án từng dòng một, rồi đưa một hai ví dụ từ *bên ngoài* dự án để bạn phân biệt được cái gì là quy luật chung, cái gì chỉ là thói quen cục bộ.

Quyết định là nội dung chính. Ở bất cứ đâu dự án phải chọn giữa các phương án (Jib thay vì docker build, Kustomize thay vì Helm, một server Postgres thứ hai thay vì một cơ sở dữ liệu thứ hai), lý do nằm trong hộp DECISION. Đó chính là những câu hỏi “tại sao bạn lại...” mà các buổi phỏng vấn được dựng nên từ đó.

Lỗi được trình bày từ triệu chứng. Các hộp PITFALL bắt đầu bằng thứ bạn thực sự nhìn thấy (CrashLoopBackOff, một lỗi TLS, một 404 lặng lẽ) rồi lần ngược về nguyên nhân, vì đó là hướng mà việc gỡ lỗi thật diễn ra.

Chú giải các hộp

QUYẾT ĐỊNH — VÍ DỤ

Vì sao dự án chọn thiết kế này thay vì thiết kế kia, và điều gì phải thay đổi để lựa chọn còn lại thắng thế.

CẠM BÃY — VÍ DỤ

Một triệu chứng sớm muộn bạn cũng sẽ gặp, kèm nguyên nhân và cách sửa.

THỰC HÀNH — VÍ DỤ

Một lệnh để chạy trên stack thật, cùng kết quả bạn nên mong đợi. Cuốn sách này được viết để đọc bên cạnh một terminal.

TRÊN PHẦN CỨNG NÀY — VÍ DỤ

Một lựa chọn chỉ đúng trên chiếc máy này (2 nhân, 19.4 GB RAM, SSD SATA, Wi-Fi). Đừng bê những thứ này vào trung tâm dữ liệu.

TỰ KIỂM TRA (đáp án: Phụ lục A)

Mỗi chương kết thúc bằng vài câu hỏi trong hộp này. Đáp án ở Phụ lục A. Hãy viết câu trả lời ra giấy trước khi xem đáp án — nhận ra không phải là nhớ được.

Ký hiệu

Code inline và tên file trông như thế này: `application.yml`. Các listing có chú thích mang những dấu khoanh tròn — ①, ② — được giải thích ngay bên dưới listing. Lệnh shell được hiển thị với dấu nhắc `$` cho server và `PS>` cho Windows PowerShell; đừng gõ dấu nhắc.

Điều gì được giả định, điều gì không

Giả định: bạn đọc được Java, và đã dùng Maven và git. Không giả định: bất kỳ kiến thức nào trước đó về Kubernetes, Jenkins, Kafka, bên trong Docker, hay frontend. Ở những chỗ tài liệu của dự án đã trình bày việc cài đặt máy một lần (`docs/RUNBOOK-SERVER-SETUP.md`), cuốn sách chỉ tóm tắt rồi đi tiếp.

Bản đề dự án

TeacherSupporter là một nền tảng quản lý khóa học được xây dựng dưới dạng các microservice Spring Cloud: giáo viên quản lý khóa học, học sinh, bài tập và bài nộp; một dictionary service phục vụ tra cứu từ vựng; đăng ký và đăng nhập (mật khẩu, Google OAuth2, và xác thực hai lớp TOTP) nằm trong một auth service riêng; thông báo e-mail chảy qua Kafka. Dự án cổ tình dùng stack Spring Cloud “cổ điển” — Eureka, Config Server, Spring Cloud Gateway — vì stack đó vẫn đang chạy ở rất nhiều doanh nghiệp, và hiểu nó khiến các lựa chọn thay thế cloud-native (Chương 27) trở nên có ý nghĩa thay vì huyền bí.

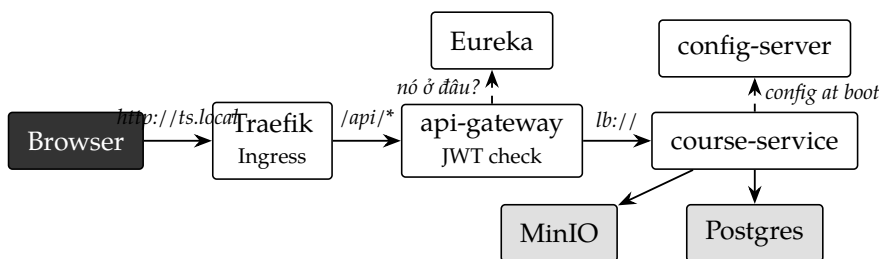
Dàn nhân vật

Service	Cổng	Kho lưu trữ	Nhiệm vụ
config-server	8888	— (classpath)	phục vụ cấu hình cho mọi service
discovery-server	8761	—	registry Eureka
api-gateway	8080	—	điểm vào duy nhất, kiểm tra JWT
auth-service	8081	Postgres ts_auth	tài khoản, JWT, OAuth2, 2FA
course-service	8082	Postgres ts_course, MinIO	khóa học, học sinh, file
dictionary-service	8083	MongoDB	tra cứu từ vựng
notification-service	8084	— (Kafka vào)	sự kiện → e-mail
frontend	3000	—	giao diện web (React)

Bảng 1: Tám đơn vị triển khai và những gì mỗi cái sở hữu.

Một request, từ đầu đến cuối

Mỗi chương soi sáng một mắt xích của chuỗi này. Phần I là các hộp bên phải; Phần II là các kho dữ liệu; Phần III là đường Kafka (không vẽ — nó chờ sự kiện, không chờ request); các Phần IV–VI là cách tất cả những thứ đó được đóng gói, chạy, và giao đi.



Hình 1: Đường đi của request mà cuốn sách này sẽ quay lại nhiều lần.

Mọi thứ nằm ở đâu trong repository

TeacherSupporter/	
pom.xml	parent: modules, shared plugin config (Jib!)
common/	shared DTOs and events
config-server/	+ src/main/resources/configurations/*.yaml
discovery-server/	api-gateway/ auth-service/
course-service/	dictionary-service/ notification-service/
frontend/	
docker-compose.yml	local dev stack
Jenkinsfile	the CI/CD pipeline (Chapter 23)
deploy/	
jenkins/	Jenkins controller image + run script
k8s/	
create-secrets.sh	secrets are made, not committed
jenkins-rbac.yaml	what Jenkins may touch in the cluster
base/	every Kubernetes manifest (Part V)
docs/	design docs, runbooks, and this book

Cỗ máy bên dưới

Mọi thứ được triển khai lên `ts-server`: một chiếc laptop tận dụng lại chạy Ubuntu 24.04 với 2 nhân vật lý, 19.4 GB RAM và một SSD SATA, truy cập được từ Windows qua Tailscale. Nó chạy k3s (Kubernetes một node), một Jenkins chạy trong Docker, và một image registry cục bộ. Ràng buộc định hình gần như mọi quyết định vận hành trong cuốn sách này là *CPU*, chứ không phải bộ nhớ: hai nhân nghĩa là build và demo không được chồng lên nhau.

Mục lục

Cách đọc cuốn sách này	ii
Bản đồ dự án	iv
I Ứng dụng	1
1 Spring Boot: Cỗ máy bên dưới mọi service	2
1.1 Vấn đề Spring Boot giải quyết	2
1.1.1 Starter	2
1.1.2 Auto-configuration	3
1.2 Cấu hình ngoài: thứ tự phân giải thuộc tính	3
1.2.1 Ràng buộc nới lỏng (relaxed binding)	4
1.2.2 Placeholder với giá trị mặc định	4
1.3 File của dự án, có chú thích	4
1.4 Profile	5
1.5 Actuator	5
1.6 Bên ngoài dự án	5
2 Cấu hình tập trung: Config Server	7
2.1 Vấn đề	7
2.2 Spring Cloud Config hoạt động thế nào	7
2.3 Server của dự án, có chú thích	7
2.4 Thứ tự khởi động và điều nhảy bootstrap	9
2.5 Làm mới cấu hình mà không khởi động lại	9
2.6 Bên ngoài dự án	9
3 Khám phá dịch vụ: Eureka	11
3.1 Vấn đề	11
3.2 Eureka hoạt động thế nào	11
3.3 Server, và thiết lập sống còn duy nhất trên k8s	11
3.4 Bên ngoài dự án	12

4	API Gateway	14
4.1	Vấn đề	14
4.2	Mô hình của Spring Cloud Gateway	14
4.3	Các route của dự án, có chú giải	14
4.4	Hai bộ định tuyến: gateway và Ingress	16
4.5	Những góc nâng cao	16
4.6	Ngoài dự án	16
5	Bảo mật: JWT, OAuth2 và biên tin cậy	18
5.1	Spring Security trong một trang	18
5.2	JWT: danh tính phi trạng thái	19
5.3	Mẫu biên tin cậy	19
5.4	Đăng nhập OAuth2 với Google	20
5.5	Xác thực hai lớp TOTP	20
5.6	Ngoài dự án	21
6	Các service nghiệp vụ và module common	22
6.1	course-service: con ngựa thồ	22
6.2	auth-service: danh tính	22
6.3	dictionary-service: anh nhỏ	22
6.4	notification-service: consumer thuần túy	22
6.5	Đồng bộ và bất đồng bộ, trong codebase này	23
6.6	Module common	23
II	Dữ liệu	25
7	Dữ liệu quan hệ: Postgres, JPA, Flyway	26
7.1	Mỗi service một cơ sở dữ liệu	26
7.2	JPA và Hibernate	26
7.3	Vì sao ddl-auto: none	27
7.4	Flyway: migration dưới dạng code	27
7.5	Connection pool	28
7.6	Ngoài dự án	28
8	Dữ liệu dạng document: MongoDB	30
8.1	Mô hình	30
8.2	Spring Data MongoDB	30
8.3	Cấu hình — toàn bộ	31
8.4	Không schema là con dao hai lưỡi	31
8.5	Vận hành trong cụm này	32

8.6	Ngoài phạm vi dự án	32
9	Lưu trữ đối tượng: MinIO và mô hình S3	33
9.1	Bài toán	33
9.2	Cấu hình, có chú giải	33
9.3	Presigned URL: mẫu đáng để biết	34
9.4	Ngoài phạm vi dự án	34
III	Truyền thông điệp	36
10	Kafka: Mô hình lõi	37
10.1	Một log, không phải một hàng đợi	37
10.2	Consumer group	38
10.3	Broker, replication, KRaft	38
10.4	Mê cung advertised listener	38
10.5	Ngữ nghĩa chuyển giao	39
10.6	Ngoài phạm vi dự án	39
11	Spring Kafka: Producer, Listener, Poison Pill	41
11.1	Phát sự kiện	41
11.2	Tiêu thụ	41
11.3	Poison pill	42
11.4	Những góc nâng cao	43
IV	Container	44
12	Docker: Image, Container, Compose	45
12.1	Container thực chất là gì	45
12.2	Dockerfile của dự án, có chú giải	45
12.3	Mạng và Compose	46
13	Image không cần daemon: Jib và registry cục bộ	48
13.1	Vì sao không cứ docker build trong CI?	48
13.2	Cấu hình, có chú giải	48
13.3	Registry, và vì sao lại là localhost:5000	49
13.4	Tag, digest, và cái bẫy SNAPSHOT	50

V Kubernetes	51
14 Kubernetes: Giải phẫu cụm và vòng lặp điều hòa	52
14.1 Ý tưởng duy nhất mà mọi thứ khác treo lên	52
14.2 Các bộ phận chuyển động	52
14.3 Object, label, namespace	53
14.4 Đưa kubectl tới cụm	53
15 Workload: Pod, Deployment, StatefulSet	55
15.1 Pod	55
15.2 Deployment và rollout	55
15.3 StatefulSet: khi danh tính quan trọng	56
15.4 Các loại workload khác	57
16 Cấu hình và Secret trong cụm	58
16.1 Hai cơ chế tiêm cấu hình	58
16.2 Secret: là gì và không là gì	58
16.3 create-secrets.sh, có chú giải	58
16.4 ConfigMap	59
16.5 Ngoài phạm vi dự án	60
17 Probe sức khỏe và quản lý tài nguyên	61
17.1 Ba loại probe	61
17.2 Requests, limits và JVM	63
18 Mạng: Service, DNS, Ingress	65
18.1 Vấn đề mà pod tạo ra	65
18.2 Ingress và Traefik	65
18.3 Đường đi trơn vẹn của một request	66
18.4 Ngoài phạm vi dự án	67
19 Lưu trữ: Volume, Claim và trạng thái trên một node	68
19.1 Chồng trừu tượng	68
19.2 local-path trên k3s	68
19.3 Hai kiểu claim trong manifest	69
19.4 Ngoài phạm vi dự án	70
20 Kustomize: Manifest theo bộ	71
20.1 Vấn đề	71
20.2 Kustomization, có chú giải	71
20.3 Base và overlay	72

VI	CI/CD	75
21	CI/CD: Các khái niệm	76
21.1	Hai thực hành, một pipeline	76
21.2	Từ vựng	76
22	Jenkins: CI server	79
22.1	Kiến trúc	79
22.2	Image của controller, có chú giải	79
22.3	Credential	80
22.4	Danh tính triển khai: một ServiceAccount, không phải admin . . .	80
23	Pipeline, từng dòng một	83
23.1	Phần mở đầu và environment	83
23.2	Ba stage	84
23.3	Xử lý thất bại	86
23.4	Cùng pipeline đó, ở nơi khác	86
VII	Vận hành và chặng đường phía trước	88
24	Vận hành stack: Cẩm nang gỡ lỗi thực địa	89
24.1	Bốn câu hỏi	89
24.2	Bảng phân loại trạng thái	89
24.3	Đọc thể giới của một service từ bên trong	90
24.4	Thói quen giữ cho cuối tuần được rảnh	91
25	Kafka trong cụm: Strimzi và luồng gửi mail	93
25.1	Operator và custom resource	93
25.2	Cài operator: một việc của admin	93
25.3	Cụm, được khai báo	94
25.4	Topic dưới dạng object	95
25.5	Service cuối cùng, và hộp thư của nó	96
25.6	Ngoài dự án	98
26	Frontend và một host duy nhất	99
26.1	Frontend là gì, dưới mắt người vận hành	99
26.2	Hai giai đoạn, một Dockerfile	99
26.3	nginx: fallback giúp SPA hoạt động	100
26.4	Giành lấy /: một origin	101
26.5	Manifest	102

26.6	Pipeline mọc thêm một stage	103
26.7	Kiểm chứng cột mốc	104
26.8	Truy cập từ bất cứ đâu: tên trên tailnet	105
26.9	Ngoài dự án	106
27	Chặng đường phía trước	107
27.1	Khả năng quan sát	107
27.2	GitOps: Argo CD	107
27.3	Cho stack cổ điển về hưu	108
27.4	Jenkins vào cụm	108
27.5	OpenShift, nói ngắn	108
27.6	Bạn đang đứng ở đâu	109
A	Đáp án tự kiểm tra	110
B	Thuật ngữ	123
C	Bảng tra nhanh	126
D	Các file cấu hình	129

Phần I

Ứng dụng

Spring Boot: Cỗ máy bên dưới mọi service

Bây trong tám đơn vị triển khai của dự án này là ứng dụng Spring Boot. Trước khi bất kỳ file YAML nào có thể có nghĩa, bạn cần ba ý tưởng: một *starter* kéo theo những gì, *auto-configuration* quyết định những gì thay cho bạn, và — quan trọng nhất cho mọi thứ trong Phần V và VI — chính xác *Spring phân giải một thuộc tính cấu hình như thế nào* khi cùng một key được định nghĩa ở nhiều nơi cùng lúc.

1.1 Vấn đề Spring Boot giải quyết

Spring cổ điển bắt bạn nối dây mọi thứ: một servlet container để triển khai vào, cấu hình XML hoặc Java khai báo từng bean, và một danh sách dài các dependency tương thích phiên bản với nhau. Spring Boot đảo ngược điều đó. Một ứng dụng chỉ là một phương thức `main()` bình thường:

```
@SpringBootApplication
public class CourseServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(CourseServiceApplication.class, args);
    }
}
```

`@SpringBootApplication` là ba annotation núp chung trong một chiếc áo choàng: `@Configuration` (lớp này có thể định nghĩa bean), `@ComponentScan` (tìm `@Service`, `@RestController`, ... trong package này và bên dưới), và `@EnableAutoConfiguration` — cái thú vị nhất.

1.1.1 Starter

Một starter như `spring-boot-starter-web` là một jar rỗng mà công việc duy nhất là mang một danh sách dependency được tuyển chọn: Spring MVC, Jackson, và một Tomcat *nhúng*. “Nhúng” quan trọng về mặt vận hành: server nằm bên trong jar, nên `java -jar course-service.jar` là một web server hoàn chỉnh. Riêng sự thật đó là điều khiến các container image ở Chương 13 đơn giản đến vậy — không có Tomcat nào để cài, chỉ cần cung cấp một JVM.

1.1.2 Auto-configuration

Lúc khởi động, Spring Boot duyệt qua một danh sách các cấu hình ứng viên và áp dụng từng cái mà điều kiện của nó thỏa mãn. Các điều kiện đọc lên kiểu như:

- *lớp này có trên classpath không?* (@ConditionalOnClass — bạn đã thêm driver Postgres, nên một DataSource được cấu hình)
- *người dùng đã tự định nghĩa bean này chưa?* (@ConditionalOnMissingBean — bean tường minh của bạn luôn thắng)
- *một thuộc tính có được đặt không?* (@ConditionalOnProperty)

Auto-configuration là lý do course-service không chứa dòng code nào tạo connection pool, Kafka producer, hay nối dây S3 client — thêm dependency cộng vài thuộc tính *chính là* cấu hình. Khi hành vi dường như xuất hiện từ hư không, hãy chạy ứng dụng với --debug một lần: “condition evaluation report” liệt kê mọi auto-configuration được áp dụng và bị bỏ qua, kèm lý do.

1.2 Cấu hình ngoài: thứ tự phân giải thuộc tính

Đây là cơ chế chịu lực nhiều nhất trong cả cuốn sách. Mọi mẹo mà các manifest Kubernetes ở Phần V giở ra đều hoạt động nhờ nó.

Một thuộc tính Spring như `spring.datasource.url` có thể được định nghĩa trong nhiều *property source* (nguồn thuộc tính). Spring gộp tất cả vào một Environment, và khi hai nguồn định nghĩa cùng một key, nguồn cụ thể hơn thắng. Rút gọn về những nguồn dự án này thực sự dùng, từ yếu nhất đến mạnh nhất:

1. giá trị mặc định trong `application.yml` đóng gói trong jar
2. cấu hình lấy từ `config-server` (Chương 2)
3. biến môi trường của hệ điều hành
4. tham số dòng lệnh

QUYẾT ĐỊNH — VÌ SAO CỤM GHI ĐỀ BẰNG BIẾN MÔI TRƯỜNG

Cùng một image phải chạy không đổi trên laptop (Compose) lẫn trong k3s. Nướng sự khác biệt giữa các môi trường vào jar nghĩa là mỗi môi trường một image — đúng thứ container sinh ra để tránh. Biến môi trường đúng trên cả YAML đóng gói lẫn `config-server`, nên Deployment của Kubernetes có thể trở lại một service sang `postgres-course:5432` mà không đụng đến một file nào thuộc về lập trình viên. Một artifact, nhiều môi trường, khác biệt được tiêm vào ở biên.

1.2.1 Ràng buộc nới lỏng (relaxed binding)

Tên biến môi trường không thể chứa dấu chấm. Vì vậy Spring ánh xạ `SPRING_DATASOURCE_URL` → `spring.datasource.url`: viết hoa, dấu chấm thành gạch dưới. Đọc bất kỳ Deployment nào trong `deploy/k8s/base/` và giờ bạn có thể giải mã mọi mục `env`: thành “cái này ghi đè key YAML kia.” Ví dụ `SPRING_DATA_MONGODB_URI` trong `dictionary-service.yaml` ghi đè `spring.data.mongodb.uri` từ `config-server`.

1.2.2 Placeholder với giá trị mặc định

Các file cấu hình của dự án dùng thêm một thành ngữ nữa:

```
app:
  jwt:
    secret: ${JWT_SECRET:mySecretKeyThatIs...}
```

`${NAME:default}` nghĩa là “dùng biến môi trường nếu có, không thì dùng giá trị mặc định.” Đây là lý do một lập trình viên có thể chạy `auth-service` mà không cần cài đặt gì, trong khi `production` tiêm `secret` thật — và là lý do ở Chương 16 `Secret` của Kubernetes chỉ cần cung cấp `JWT_SECRET` là nắm quyền kiểm soát giá trị này.

1.3 File của dự án, có chú thích

Mỗi service chỉ mang một `application.yml` tí hon; phần cấu hình thú vị nằm trên `config-server`. Đây là file của `course-service`, đầy đủ:

```
spring:
  application:
    name: course-service ①
  config:
    import: optional:configserver:http://localhost:8888 ②
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/ ③
```

- ① Danh tính của service. `Config-server` dùng nó để chọn file cần phục vụ (`configurations/course-service.yml`); `Eureka` dùng nó làm tên đăng ký; gateway định tuyến tới nó bằng `lb://course-service`. Một chuỗi ký tự buộc cả ba hệ thống lại với nhau.
- ② Import cấu hình từ xa lúc bootstrap. `optional`: nghĩa là “cứ khởi động dù `config-server` có sập” — dễ chịu khi dev, và đáng bàn ở `production` (một service

khởi động với nửa cấu hình có thể hỏng theo những cách lạ lùng hơn một service từ chối khởi động). Giá trị mặc định `localhost:8888` được ghi đè bởi `SPRING_CONFIG_IMPORT` trong cả Compose lẫn Kubernetes.

- ③ Eureka ở đâu — lại một giá trị mặc định `localhost`, lại được ghi đè theo từng môi trường bằng `EUREKA_CLIENT_SERVICEURL_DEFAULTZONE`.

1.4 Profile

Một *profile* Spring là một biến thể cấu hình có tên. `spring.profiles.active=native` trên `config-server` (Chương 2) là chỗ dùng profile duy nhất của dự án cho đến giờ, nhưng cơ chế này tổng quát hơn: `application-prod.yml` được nạp đề lên `application.yml` khi profile `prod` đang kích hoạt, cho phép một jar mang nhiều tính cách. Kubernetes làm giảm nhu cầu đó — thay vào đó môi trường tiềm sự khác biệt vào — đó là lý do dự án này dựa vào biến môi trường, không phải file profile, cho thay đổi theo môi trường.

1.5 Actuator

`spring-boot-starter-actuator` thêm các endpoint vận hành dưới `/actuator: /health` (readiness tổng hợp của DB, đĩa, các kiểm tra tùy chỉnh), `/info`, `/metrics`, `/prometheus`. Hai sự thật quan trọng lúc này:

- Starter phải có trên classpath. Cấu hình exposure mà không có `spring-boot-starter-actuator` thì vô tác dụng — dự án đã mắc đúng lỗi đó ở bốn service và phát hiện ra qua một probe Kubernetes thất bại (Chương 17).
- Endpoint phải được *expose* (`management.endpoints.web.exposure.include`) — dự án expose `health, info, metrics, prometheus`.
- Expose không phải là *phân quyền*: nếu Spring Security bảo vệ `/actuator/**`, endpoint tồn tại nhưng trả lời 401. Chính tương tác này quyết định service nào dùng được probe Kubernetes nào ở Chương 17 — `dictionary-service` cho phép actuator và được probe qua `/actuator/health`; `auth-service` thì không, và phải được probe qua `/v3/api-docs` thay thế.

1.6 Bên ngoài dự án

Hai ví dụ tương phản của cùng một bộ máy thuộc tính:

Một ứng dụng *twelve-factor* kiểu *Heroku* cấu hình mọi thứ bằng biến môi trường — `DATABASE_URL`, `PORT` — và mang một `application.yml` chỉ toàn placeholder `${...}`. Thứ tự phân giải của Spring khiến phong cách đó trở thành bản địa.

Một ngân hàng chạy một artifact qua năm môi trường (dev/test/uat/pre-prod/prod) thường ghép một config-server dựa trên git repository với profile cho từng môi trường: `course-service-uat.yml` nằm trong một repo được kiểm toán, và việc thăng cấp là một lần merge git. Config-server classpath native của dự án (Chương 2) là phiên bản một-lập-trình-viên của cùng hình dạng đó.

CẠM BẦY — GIÁ TRỊ GHI ĐỀ CỦA TÔI BỊ BỎ QUA

Triệu chứng: bạn đặt một biến môi trường nhưng service vẫn dùng giá trị cũ. Các nguyên nhân thường gặp, theo thứ tự tần suất: tên biến không ánh xạ tới key bạn nghĩ (kiểm tra cách viết theo relaxed binding — `SERVICEURL` trong key Eureka là một từ, một cái tên bất quy tắc phải chép chính xác); giá trị cũng được đặt bởi một nguồn *mạnh hơn* (một tham số dòng lệnh); hoặc service đọc key lúc build thay vì từ Environment. Chẩn đoán bằng endpoint actuator `/actuator/env` (khi được expose), nó liệt kê mọi property source và nguồn nào đã thắng.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Một key được định nghĩa trong `application.yml` của jar, trong file của config-server cho service đó, và dưới dạng biến môi trường trong Deployment. Giá trị nào thắng, và vì sao thứ tự đó là đúng cho container?
2. `@SpringBootApplication` gộp ba annotation nào, và annotation nào trong số đó giải thích vì sao xóa một dependency không dùng đến có thể làm thay đổi hành vi lúc chạy?
3. `${GOOGLE_CLIENT_ID:placeholder}` — biểu thức này phân giải thành gì khi biến chưa được đặt, và điều đó cho auth-service tính chất vận hành nào lúc khởi động?
4. Vì sao expose một endpoint actuator không đảm bảo rằng một probe Kubernetes dùng được nó?

Cấu hình tập trung: Config Server

2.1 Vấn đề

Bây service, mỗi cái đều cần URL database, địa chỉ Kafka, thiết lập JWT, host mail. Chép những thứ đó vào bầy file `application.yml` và mỗi thay đổi trở thành bầy lần sửa, bầy lần build lại, cùng lời đảm bảo rằng sớm muộn hai bản sao sẽ lệch nhau. Câu trả lời của Spring Cloud cổ điển: một HTTP service nhỏ mà công việc duy nhất là trao cho từng ứng dụng cấu hình của nó lúc khởi động.

2.2 Spring Cloud Config hoạt động thế nào

Server expose cấu hình qua một hợp đồng HTTP đơn giản:

```
GET /{application}/{profile}
GET /course-service/default    -> the config for course-service
```

Phía client là dòng `spring.config.import: configserver:...` bạn đã thấy ở Chương 1. Trong lúc khởi động — *trước* khi hầu hết bean được tạo — client lấy tài liệu của mình về và gộp vào Environment như một property source đứng trên YAML đóng gói nhưng thua biến môi trường. Thứ tự phân giải ở Mục 1.2 làm toàn bộ công việc; config-server chỉ là thêm một nguồn nữa trong chồng.

2.3 Server của dự án, có chú thích

```
server:
  port: 8888                                ①
spring:
  application:
    name: config-server
  profiles:
    active: native                           ②
  cloud:
    config:
      server:
        native:
          search-locations: classpath:/configurations ③
```

```
management:
  endpoints:
    web:
      exposure:
        include: health
```

④

- ① 8888 là cổng config-server theo quy ước, cũng như 8761 là của Eureka. Quy ước tiết kiệm tài liệu.
- ② Profile native nghĩa là “phục vụ file từ một vị trí trên filesystem/classpath” thay vì backend git mặc định.
- ③ Các file được phục vụ nằm *ngay trong jar của chính config-server*: configurations/course-service-auth-service.yml, v.v. — được chọn bằng cách khớp tên file với spring.application.name của service gửi yêu cầu.
- ④ Chỉ health được expose — và đây là endpoint mà healthcheck của Compose và các probe của Kubernetes dựa vào.

QUYẾT ĐỊNH — CLASSPATH NATIVE VS. BACKEND GIT

Backend chuẩn doanh nghiệp là một git repository: thay đổi cấu hình là các commit — được kiểm toán, được review, hoàn tác được mà không cần build lại gì cả. Cái giá là một repository thứ hai và việc quản lý thông tin đăng nhập. Với một dự án một-lập-trình-viên mà thay đổi cấu hình dù sao cũng đi cùng thay đổi code, classpath native giữ mọi thứ trong một repo và một pipeline. Thứ bị đánh đổi: thay một giá trị giờ đòi hỏi build lại image config-server. Hãy biết nhà tuyển dụng của bạn đứng ở phía nào của sự đánh đổi đó.

CẠM BÃY — SERVICE KHỞI ĐỘNG VỚI GIÁ TRỊ MẶC ĐỊNH RỒI HỎNG MỘT CÁCH KỲ LẠ

Triệu chứng: auth-service khởi động được nhưng mọi request đều thất bại với lỗi kết nối tới localhost:5432. Nguyên nhân: config-server không liên lạc được lúc khởi động, và optional:configserver: để service khởi động từ giá trị mặc định đóng gói sẵn — vốn trở về localhost. Vì thất bại xuất hiện vài phút sau và cách xa nguyên nhân, nó trông như một vấn đề database. Kiểm tra: log khởi động in ra cấu hình từ xa có được lấy về hay không. File Compose ngăn vấn đề thứ tự này bằng depends_on: condition: service_healthy; Kubernetes không có thứ tự khởi động (Chương 15), nên các service phải chịu được một config-server đến muộn bằng cách crash rồi khởi động lại cho đến khi nó xuất hiện.

2.4 Thứ tự khởi động và điều nhảy bootstrap

Config-server tạo ra một ràng buộc trình tự ngầm trên toàn hệ thống: mọi service khác đều muốn nó lên trước. Bạn sẽ gặp hệ quả của điều này ba lần trong cuốn sách:

- Compose mã hóa thứ tự một cách tường minh bằng `depends_on` và một healthcheck dựa trên `curl` (Chương 12).
- Kubernetes từ chối mã hóa thứ tự khởi động; hệ thống hội tụ bằng `retry` (Chương 15).
- Pipeline Jenkins triển khai mọi manifest trong một lệnh `kubectl apply -k` và để các rollout tự ổn định đồng thời (Chương 23).

2.5 Làm mới cấu hình mà không khởi động lại

Mặc định, một client đọc cấu hình của nó một lần lúc khởi động. Spring Cloud cung cấp `/actuator/refresh` (lấy lại theo yêu cầu, từng instance, áp dụng cho các bean @RefreshScope) và Spring Cloud Bus (phát tán lệnh refresh qua một message broker). Dự án này không dùng cả hai: trên Kubernetes một thay đổi cấu hình được giao đi dưới dạng image mới hoặc thay đổi env, và rolling update của Deployment (Chương 15) dù sao cũng khởi động lại các pod. Đó là câu trả lời hiện đại cho câu hỏi mà config-server ban đầu được thiết kế để giải quyết.

2.6 Bên ngoài dự án

Một thiết lập doanh nghiệp điển hình dựa trên git, để đối chiếu:

```
spring:
  cloud:
    config:
      server:
        git:
          uri: https://git.company.com/platform/config-repo
          default-label: main
          search-paths: '{application}'
```

Cấu hình nằm theo từng service trong các thư mục của một repo riêng; label là một branch, nên `default-label: release-42` ghim một môi trường vào một bản phát hành. Mẫu phổ biến thứ hai bỏ hẳn config-server: ConfigMap và Secret của Kubernetes được gắn vào dưới dạng biến môi trường — chính là hướng dự án

này đang đi tới (Chương 16 và 26); config-server tồn tại ở đây chủ yếu như một ví dụ sống của stack cổ điển.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Các nguồn này thắng theo thứ tự nào cho `spring.datasource.url`: YAML đóng gói, tài liệu config-server, biến môi trường của Deployment?
2. `profiles.active`: native thay đổi gì về nơi server tìm tài liệu, và backend git mua được gì mà native không thể?
3. Vì sao `optional`: trong `spring.config.import` là con dao hai lưỡi? Mô tả kiểu thất bại mà nó cho phép.
4. Cấu hình của chính config-server không thể đến từ một config-server. Nó phải nằm ở đâu, và đó là nguyên lý bootstrap tổng quát nào?

Khám phá dịch vụ: Eureka

3.1 Vấn đề

Gateway phải chuyển tiếp `/api/courses` tới `course-service`. Tới địa chỉ nào? Trong Compose nó có thể ghi cứng `course-service:8082` — nhưng khi đó mỗi replica mới, mỗi service dời chỗ, mỗi môi trường đều là một thay đổi cấu hình. Khám phá dịch vụ (service discovery) thay địa chỉ bằng *tên được phân giải lúc chạy*: các service tự đăng ký vào một registry; client hỏi registry.

3.2 Eureka hoạt động thế nào

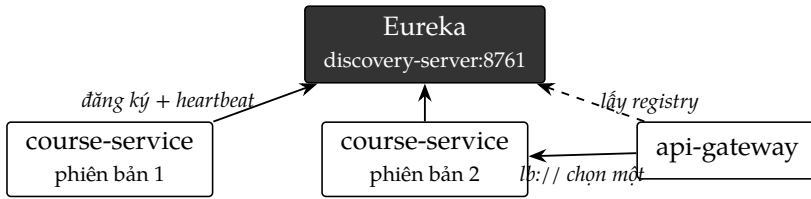
Eureka (từ Netflix, tổ tiên của stack Spring Cloud) là một REST service giữ một registry các instance:

- **Đăng ký.** Lúc khởi động mỗi client POST danh tính của mình: tên ứng dụng (`spring.application.name`), host, cổng, URL health.
- **Heartbeat.** Mỗi 30 s (mặc định) client gia hạn lease của mình. Bỏ lỡ vài lần gia hạn và instance bị loại bỏ (evict).
- **Cache phía client.** Bên tiêu thụ lấy registry về và cache lại (mặc định làm mới mỗi 30 s), rồi cân bằng tải *cục bộ* giữa các instance mà nó biết. Registry sập trong chốc lát không làm dừng lưu lượng — bên gọi tiếp tục dùng cache của mình.
- **Tự bảo toàn.** Nếu quá nhiều heartbeat biến mất cùng lúc, Eureka giả định đó là sự cố mạng chứ không phải chết hàng loạt và ngừng loại bỏ. Ít bất ngờ hơn khi mạng chập chờn; dữ liệu cũ hơn khi sự cố là thật.

Phía tiêu thụ trong dự án này là URI `lb://course-service` của gateway (Chương 4): “tra tên này trong registry, chọn một instance.”

3.3 Server, và thiết lập sống còn duy nhất trên k8s

Module `discovery-server` là Eureka với gần như không có cấu hình — một annotation `@EnableEurekaServer` và một cổng. Cấu hình thú vị nằm trên mỗi *client*, và



Hình 3.1: Đăng ký, heartbeat, và cân bằng tải phía client.

một dòng trong đó xứng đáng có mục riêng vì nó quyết định cả hệ thống có chạy được trên Kubernetes hay không:

```
- name: EUREKA_INSTANCE_PREFER_IP_ADDRESS
  value: "true"
```

Mặc định một client đăng ký *hostname* của nó. Trên laptop của bạn thế là ổn. Bên trong một pod Kubernetes, *hostname* là *tên pod* — `course-service-7d9f6c-x2v1q` — thứ các pod khác không phân giải được. Mọi đăng ký sẽ là một địa chỉ không ai gọi tới được. *IP* của pod, ngược lại, định tuyến được trên toàn cụm. Vì thế mọi Deployment trong `deploy/k8s/base/` đều đặt cờ này: đăng ký IP, không phải tên.

CẠM BÃY — MỌI THỨ ĐỀU ĐĂNG KÝ, KHÔNG GÌ GỌI ĐƯỢC GÌ

Triệu chứng: dashboard Eureka hiện mọi service đều UP, nhưng mọi request đi qua gateway đều thất bại với `UnknownHostException`. Nguyên nhân: *hostname* được đăng ký là tên pod (xem ở trên). Đây là lỗi Eureka-trên-Kubernetes phổ biến nhất, và nó vô hình cho đến lần gọi chéo service đầu tiên.

QUYẾT ĐỊNH — VÌ SAO VẪN DỪNG EUREKA, KHI KUBERNETES ĐÃ CÓ SERVICE

Kubernetes đã cho mỗi Service một tên DNS (`course-service:8082`) với cân bằng tải tích hợp sẵn — khám phá dịch vụ là bản địa của nền tảng. Eureka ở đây là sự dư thừa có chủ đích vì CV: stack cổ điển vẫn phổ biến trong ngành, và việc di cư *khỏi* nó (Chương 27: thay `lb://` bằng DNS Service thuần, xóa hai đơn vị triển khai) tự thân là loại công việc hiện đại hóa mà doanh nghiệp trả tiền để có. Xây cả hai stack là để học cuộc di cư đó.

3.4 Bên ngoài dự án

Consul (HashiCorp) là lựa chọn thay thế chính ngoài JVM: dựa trên agent, không phụ thuộc ngôn ngữ, với health check do agent thực hiện thay vì heartbeat tự báo

cáo, cộng một kho key-value có thể thay luôn cả config-server. *Khám phá dịch vụ bản địa của Kubernetes* là cực còn lại: không có tiến trình registry nào cả — các controller của nền tảng theo dõi readiness của pod và cập nhật endpoint của Service; “đăng ký” đơn giản là một pod trở thành Ready (Chương 17 cho thấy bộ máy đó). Hãy để ý mẫu hình: nền tảng cung cấp càng nhiều, ứng dụng càng phải mang ít.

TRÊN PHẦN CỨNG NÀY

Các giá trị mặc định bảo thủ của Eureka (heartbeat 30 s, cache client 30 s, loại bỏ sau 90 s) nghĩa là một instance đã crash vẫn có thể nhận lưu lượng tới một phút. Với một replica cho mỗi service thì điều đó hầu như không quan trọng — bên gọi đăng nào cũng thất bại — nhưng nó giải thích vì sao các đợt triển khai cuốn chiếu ở Chương 23 chờ readiness của Kubernetes, chứ không chờ trạng thái Eureka.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Lần theo những gì xảy ra từ lúc course-service khởi động đến lúc gateway định tuyến tới nó thành công: những lần đăng ký, lấy về, và cache nào có liên quan, và độ trễ trong trường hợp xấu nhất là bao nhiêu?
2. Vì sao prefer-ip-address quan trọng trên Kubernetes nhưng không quan trọng trên Compose? (Mỗi nền tảng làm cho cái gì phân giải được?)
3. Chế độ tự bảo toàn đánh đổi tính chất nào lấy tính chất nào? Cho một kịch bản nó giúp ích và một kịch bản nó gây hại.
4. Nếu bỏ Eureka và trở gateway thẳng tới `http://course-service:8082`, bộ máy Kubernetes nào thay thế từng tính năng của Eureka?

API Gateway

4.1 Vấn đề

Không có gateway, frontend phải biết địa chỉ của sáu service, mỗi service phải tự cài đặt xác thực, và những mối quan tâm xuyên suốt (CORS, rate limit, header truy vết) bị làm sáu lần hoặc không lần nào. Gateway cho client bên ngoài *một* địa chỉ duy nhất và gom mọi mối quan tâm ở biên về một chỗ; phía sau nó, các service có thể tin tưởng bên gọi mình.

4.2 Mô hình của Spring Cloud Gateway

Ba khái niệm tạo nên mọi route:

Route một URI đích cộng với các điều kiện và phép viết lại bên dưới.

Predicates điều kiện quyết định request có khớp hay không: path, method, header, host. Route khớp đầu tiên thắng.

Filters các phép biến đổi áp dụng trên đường đi: bỏ hoặc thêm đoạn path, thêm header, rate limit, retry.

Gateway được xây trên stack *reactive* (Project Reactor + Netty, web-application-type: reactive trong application.yml của nó) thay vì Tomcat servlet. Một proxy dành cả đời để chờ các server khác; mô hình event-loop giữ được hàng nghìn request đang bay mà không cần mỗi request một thread. Về vận hành, đây là lý do Deployment của gateway được cấp memory request nhỏ hơn các service MVC (384 Mi so với 512 Mi) và khởi động nhanh hơn — điều Chương 17 tận dụng bằng một startup probe ngắn hơn.

4.3 Các route của dự án, có chú giải

Từ configurations/api-gateway.yml (rút gọn — khối course-service lặp lại cho students, assignments, enrollments, submissions):

```
spring:
  cloud:
    gateway:
      routes:
```

```

- id: auth-service
  uri: lb://auth-service      ①
  predicates:
    - Path=/api/auth/**      ②
  filters:
    - StripPrefix=2          ③
- id: auth-oauth2
  uri: lb://auth-service
  predicates:
    - Path=/oauth2/**        ④
- id: course-service-courses
  uri: lb://course-service
  predicates:
    - Path=/api/courses/**
  filters:
    - StripPrefix=2
app:
  jwt:
    secret: ${JWT_SECRET:mySecret...} ⑤

```

- ① lb:// = “phân giải tên này qua discovery client rồi cân bằng tải.” Tên đó là `spring.application.name` của đích. Đổi lb:// thành http:// là bạn có một proxy tĩnh, không dính gì tới Eureka.
- ② Predicate theo path. ** khớp mọi độ sâu.
- ③ StripPrefix=2 bỏ hai đoạn: `/api/auth/login` tới auth-service dưới dạng `/login`. Đây là lý do `SecurityConfig` của auth-service (Chương 5) cho phép `/login` chứ không phải `/api/auth/login` — service không hề biết tiền tố công khai của mình.
- ④ Không có StripPrefix ở đây: redirect URI OAuth2 của Google được đăng ký với đường dẫn đầy đủ, nên nó phải tới auth-service nguyên vẹn.
- ⑤ Gateway giữ khóa *xác minh* JWT. Cùng secret với auth-service, bên ký — một secret dùng chung, hai vai trò, và trên Kubernetes là một object Secret mount vào cả hai (Chương 16).

CẠM BẤY — SERVICE TRẢ 404, CURL TRÊN POD TRẢ 200

Triệu chứng: gọi `/api/courses/list` qua gateway nhận 404, nhưng cũng endpoint đó lại trả lời khi gọi thẳng vào service. Nguyên nhân: lệch tiền tố — hoặc StripPrefix đã bỏ đoạn mà controller mong đợi, hoặc mapping của controller chứa tiền tố mà gateway đã bỏ rồi. Gỡ lỗi bằng cách log đường dẫn *sau khi viết lại*: bật debug logging cho `spring.cloud.gateway` và so với `@RequestMapping` của controller.

4.4 Hai bộ định tuyến: gateway và Ingress

Trên Kubernetes dự án này có *hai* bộ định tuyến L7 nối tiếp nhau: Traefik (Ingress controller, Chương 18) ở biên cụm, rồi tới gateway này. Sự phân công là có chủ ý:

- Traefik sở hữu *host*: TLS về sau, một cửa vào công khai, tách nhánh theo tiền tố path. Nó gửi `/api`, `/oauth2`, `/login/oauth2` tới gateway và sẽ gửi `/` tới frontend.
- Gateway sở hữu *biên ứng dụng*: phân giải qua Eureka, xác minh JWT, viết lại theo từng route.

QUYẾT ĐỊNH — KHÔNG CÓ CỬA HỒNG

Cột mốc 3 từng có một Ingress gửi `/courses` thẳng tới `course-service` — giản giáo hữu ích, nhưng nó đi vòng qua bước xác minh JWT: một lối vào không xác thực tới dữ liệu được bảo vệ. Nó bị xóa ngay ngày gateway được triển khai. Quy tắc rút ra: *mọi* route bên ngoài phải đi qua thành phần thực thi xác thực, nếu không xác thực chỉ là đồ trang trí.

4.5 Những góc nâng cao

Đáng biết, nhưng chưa dùng ở đây: filter `RequestRateLimiter` (token bucket dựa trên Redis), `Retry` có backoff cho các GET lũy đẳng, filter `CircuitBreaker` (Resilience4j) trả về fallback khi một service sập, và các filter mặc định áp cho mọi route. Mỗi thứ chỉ vài dòng YAML — gateway là nơi những chính sách như vậy thuộc về, vì đó là điểm duy nhất mọi request bên ngoài đều đi qua.

4.6 Ngoài dự án

Cùng ý tưởng ba khái niệm trên route đó trong một gateway thuần Kubernetes (không Eureka, dùng DNS của Service thay thế) — kiểu annotation của `nginx-ingress`:

```
nginx.ingress.kubernetes.io/rewrite-target: /$2
# path: /api/courses(/|$)(.*) -> service: course-service
```

Và phiên bản cloud quản lý: bảng route của AWS API Gateway hay GCP API Gateway làm predicate/filter dưới dạng cấu hình của nhà cung cấp, với việc xác minh JWT giao cho nhà cung cấp (Cognito, IAP). Khái niệm chuyển 1:1; chỉ cú pháp và hóa đơn là khác.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Lần theo POST /api/auth/login từ trình duyệt tới phương thức controller trên Kubernetes: kể tên mọi chặng và mọi lần viết lại đường dẫn.
2. Vì sao các route OAuth2 không được bỏ tiền tố trong khi các route API thì phải?
3. Gateway xác minh JWT; auth-service ký chúng. Nếu hai bên có lúc giữ secret khác nhau thì cái gì hỏng, với triệu chứng gì?
4. Vì sao stack reactive là lựa chọn đúng cho proxy nhưng không đương nhiên đúng cho course-service?

Bảo mật: JWT, OAuth2 và biên tin cậy

Bảo mật là nơi dự án này dạy được nhiều nhất, vì nó dùng ba cơ chế mà ngành dùng liên tục — JWT phi trạng thái, đăng nhập OAuth2, xác thực hai lớp TOTP — và nối chúng qua một gateway theo mẫu “biên tin cậy” (trusted edge) chuẩn mực.

5.1 Spring Security trong một trang

Spring Security là một chuỗi filter servlet đứng trước các controller của bạn. Mỗi request đi qua chuỗi đó; các filter xác thực (bạn là ai?) và các quy tắc phân quyền quyết định (bạn được phép không?). Chuỗi của auth-service trong dự án, từ SecurityConfig của nó:

```
http
    .csrf(AbstractHttpConfigurer::disable) ①
    .sessionManagement(s ->
        s.sessionCreationPolicy(SessionCreationPolicy.STATELESS)) ②
    .authorizeHttpRequests(auth -> auth
        .requestMatchers("/register", "/login", "/activate",
            "/refresh", "/verify-2fa", "/change-password",
            "/oauth2/**", "/login/oauth2/**",
            "/v3/api-docs/**", "/swagger-ui/**").permitAll() ③
        .anyRequest().authenticated())
    .oauth2Login(o -> o.successHandler(oAuth2LoginSuccessHandler))
    .addFilterBefore(gatewayHeaderAuthFilter,
        UsernamePasswordAuthenticationFilter.class); ④
```

- ① Bảo vệ CSRF che chắn cho phiên cookie của trình duyệt; một API JWT phi trạng thái không có phiên cookie nào để lợi dụng, nên nó bị tắt. Tắt vì lý do đó — không phải vì một tutorial bảo thế.
- ② STATELESS: không tạo HTTP session. Mỗi request phải tự mang bằng chứng danh tính. Đây là tính chất cho phép bất kỳ replica nào trả lời bất kỳ request nào — điều kiện tiên quyết để mở rộng.
- ③ Các đường dẫn công khai. Để ý đó là các đường dẫn *đã bỏ tiền tố* (/login, không phải /api/auth/login) — Chương 4 giải thích vì sao. Cũng để ý thứ *vắng mặt*: /actuator/**. Sự vắng mặt đó quyết định chiến lược probe Kubernetes của service này ở Chương 17.

- ④ Filter tùy biến tin tưởng các header danh tính từ gateway — chủ đề của Mục 5.3.

5.2 JWT: danh tính phi trạng thái

Một JSON Web Token gồm ba phần `base64url`, `header.payload.signature`. Payload mang các *claim*: `subject` (id người dùng), `vai trò`, `hạn dùng`. Chữ ký là HMAC-SHA256 trên header và payload với một `secret` dùng chung — chuỗi tối thiểu 256 bit mà cả `auth-service` lẫn `gateway` đều giữ.

Các tính chất quan trọng:

- **Tự chứa.** Xác minh không cần gọi cơ sở dữ liệu: kiểm tra chữ ký, kiểm tra hạn, đọc *claim*. Chính điều này cho phép *gateway* xác thực mà không sở hữu dữ liệu người dùng.
- **Đọc được.** Base64 là encoding, không phải mã hóa (encryption) — ai cũng giải mã được payload. Đừng bao giờ đặt bí mật trong *claim*.
- **Không thu hồi được cho tới khi hết hạn.** Token đã ký thì không thể “hủy ký”. Vì thế mới có mẫu hai token: một *access token* sống ngắn (ở đây 15 phút) đi cùng một *refresh token* sống dài (7 ngày) mà có được đối chiếu với cơ sở dữ liệu tại `/refresh` — việc thu hồi diễn ra ở đó.

5.3 Mẫu biên tin cậy

`JwtAuthenticationFilter` của `gateway` xác minh token, rồi chuyển tiếp request với danh tính dưới dạng header thuần:

```
ServerHttpRequest mutatedRequest = exchange.getRequest().mutate()
    .header(JwtConstants.HEADER_USER_ID, userId)
    .header(JwtConstants.HEADER_USER_ROLE, role)
    .build();
```

Ở phía sau, `GatewayHeaderAuthFilter` của mỗi service đọc các header đó và dựng Spring Security context từ chúng — không xác minh lại. Service giữ được sự đơn giản; mật mã chỉ chạy một lần.

Nghĩa vụ đi kèm mẫu này: các header đó chỉ đáng tin nếu *mọi* đường tới một service đều đi qua `gateway`. Một client với được thẳng tới `course-service` có thể giả mạo `X-User-Id`. Đây là lý do sâu xa khiến Ingress `/courses` trực tiếp bị xóa (Chương 4), và là lý do lưu lượng nội cụm vẫn là điểm yếu còn lại — pod nào cũng vẫn có thể gọi `course-service:8082` với header bịa ra. `NetworkPolicy` hay

mTLS (service mesh) là cách gia cố trong công nghiệp; Chương 27 bàn khi nào chúng xứng đáng với độ phức tạp của mình.

5.4 Đăng nhập OAuth2 với Google

Luồng *authorization code* của OAuth2, như nó chạy ở đây:

1. Trình duyệt gọi `/oauth2/authorization/google` (qua gateway); Spring chuyển hướng tới Google kèm `client-id` của ứng dụng và một `redirect_uri`.
2. Người dùng đồng ý tại Google; Google chuyển hướng trình duyệt về `redirect_uri` — `/login/oauth2/code/google` — mang theo một *code* dùng một lần.
3. Auth-service đổi `code` + `client-secret` lấy token theo đường server-tới-server, đọc e-mail/hồ sơ người dùng, rồi `OAuth2LoginSuccessHandler` của dự án tiếp quản: tìm hoặc tạo tài khoản cục bộ và phát hành JWT *của chính chúng ta*. Google xác thực; ứng dụng vẫn phân quyền bằng token của riêng mình.

CẠM BẦY — REDIRECT_URI_MISMATCH

Triệu chứng: Google hiện “Error 400: redirect_uri_mismatch” thay vì màn hình đồng ý. Nguyên nhân: URI ứng dụng gửi đi không trùng từng byte với URI đã đăng ký trong Google Cloud console. Nó thay đổi theo môi trường (`localhost:8080` khi dev, `ts.local` trong cụm — đó là lý do Deployment ghi đề `...GOOGLE_REDIRECT_URI`), và mọi biến thể đều phải được đăng ký. URI phải tới auth-service *chưa bị bỏ tiền tố*, đó là lý do các route OAuth2 trên gateway không có `StripPrefix`.

5.5 Xác thực hai lớp TOTP

Mật khẩu dùng một lần theo thời gian (Time-based One-Time Password, RFC 6238): khi đăng ký, server sinh một secret, hiện nó dưới dạng mã QR, và ứng dụng authenticator suy ra mã 6 chữ số từ `HMAC(secret, time/30s)`. Đăng nhập thành hai bước — kiểm tra mật khẩu phát ra một trạng thái chờ sống ngắn, rồi `/verify-2fa` kiểm tra mã và phát hành token thật. Server và điện thoại không bao giờ liên lạc với nhau; chúng chỉ chia sẻ secret và một chiếc đồng hồ.

5.6 Ngoài dự án

Lựa chọn thay thế chính trong công nghiệp đưa việc phát hành token ra khỏi code của bạn hoàn toàn: một identity provider (Keycloak tự host, hoặc Auth0/Cognito được quản lý) phát hành token ký bằng cặp khóa *bất đối xứng* (RS256). Service xác minh bằng khóa *công khai* lấy từ một URL JWKS — không có secret dùng chung phải phân phối, xoay khóa có sẵn:

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          jwk-set-uri: https://idp.company.com/realms/app/protocol/openid-connect/certs
```

Hiểu phiên bản HS256 tự tay làm — thứ dự án này xây — chính là điều làm cho cấu hình kia có ý nghĩa: cùng claim, cùng cách xác minh, chỉ khác nghi thức khóa.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Vì sao có thể tắt bảo vệ CSRF cho một API JWT phi trạng thái nhưng không thể cho một webapp dùng phiên cookie?
2. Kẻ tấn công đánh cắp được một access token. Nó dùng được bao lâu, trong lúc đó server không thể làm gì, và endpoint nào cuối cùng chặn được phiên?
3. Trong mẫu biên tin cậy, phát biểu chính xác bất biến phải giữ để X-User-Id đáng tin, và nêu hai sai lầm triển khai sẽ phá vỡ nó.
4. Trong luồng Google, secret nào không bao giờ rời server, giá trị nào đi qua trình duyệt, và vì sao giá trị trình duyệt thấy được lại an toàn?
5. HS256 so với RS256: ai xác minh được, ai ký được, và vì sao các identity provider chọn RS256?

Các service nghiệp vụ và module common

Các chương trước bàn về những service hạ tầng. Chương này vẽ bản đồ ba service làm việc thật, cộng với module common dùng chung — và qua chúng, hai cách microservice nói chuyện với nhau.

6.1 course-service: con ngựa thồ

Khóa học, học viên, bài tập, ghi danh, bài nộp — CRUD kinh điển trên Postgres (Chương 7), cộng thêm file đính kèm lưu trong MinIO (Chương 9). Nó *phát* sự kiện `ts.assignment.created` và *lắng nghe* `ts.user.activated` — mẫu đáng để ý là nó phản ứng với sự kiện nghiệp vụ của service khác mà không bao giờ gọi service đó.

6.2 auth-service: danh tính

Mọi thứ Chương 5 đã mô tả: đăng ký kèm kích hoạt qua e-mail, đăng nhập bằng mật khẩu và Google, 2FA TOTP, phát hành JWT, refresh token. Nó phát ba sự kiện: `ts.user.registered`, `ts.user.activated`, `ts.user.admin-provisioned`.

6.3 dictionary-service: anh nhỏ

Tra cứu từ vựng trên MongoDB (Chương 8). Đơn giản có chủ ý — nó tồn tại một phần để chứng minh kiến trúc chứa được một document store, một phần làm service ít rủi ro để thử mọi thứ trước.

6.4 notification-service: consumer thuần túy

Không có REST API đáng kể, không cơ sở dữ liệu. Nó đăng ký ba topic và biến sự kiện thành e-mail (Maildev khi dev, một SMTP relay thật về sau):

Topic	Producer	Mail gửi đi
ts.user.registered	auth-service	link kích hoạt
ts.user.admin-provisioned	auth-service	thông báo tài khoản
ts.assignment.created	course-service	thông báo bài tập mới

Vì đầu vào *duy nhất* của nó là Kafka, đây là service duy nhất không thể chạy hữu ích trước khi có broker — đó là lý do manifest Kubernetes của nó đi cùng cột mốc Kafka (Chương 25) chứ không cùng các service khác.

6.5 Đồng bộ và bất đồng bộ, trong codebase này

Dự án dùng cả hai kiểu tương tác, mỗi kiểu đúng chỗ của nó:

Đồng bộ (HTTP qua gateway) khi bên gọi cần câu trả lời để đi tiếp: frontend lấy danh sách khóa học. Ràng buộc về thời gian — cả hai bên phải đang chạy.

Bất đồng bộ (sự kiện qua Kafka) khi bên gọi chỉ cần ghi nhận một sự thật: một đăng ký vừa xảy ra; ai quan tâm thì hành động. auth-service không biết notification-service tồn tại. Tách rời về thời gian — nếu consumer sập, mail đi muộn chứ không mất.

Quy tắc ngón tay cái codebase tuân theo: *lệnh* (command) cần câu trả lời thì dùng HTTP; *sự thật* mà người khác có thể phản ứng thì là sự kiện, đặt tên ở thì quá khứ (UserRegistered, AssignmentCreated).

6.6 Module common

```
com/ts/common/
dto/          CourseDto, StudentDto, UserDto, ...
              UserRegisteredEvent, UserActivatedEvent,
              AssignmentCreatedEvent, AdminProvisionedUserEvent
exception/    ApiException, ErrorResponse
security/     JwtConstants      <- header names, claim names
```

Ba kiểu chia sẻ, với ba hồ sơ rủi ro khác nhau:

- **Các record sự kiện** là *hợp đồng* giữa producer và consumer. Chia sẻ class bảo đảm hai bên nhất trí — với cái giá là phải triển khai đồng bộ khi hình dạng thay đổi. (Lựa chọn thay thế trong công nghiệp — schema registry — xuất hiện ở Chương 11.)

- **JwtConstants** chia sẻ *tên* header và claim giữa gateway và các service. Một tên header gõ sai sẽ hỏng âm thầm — request đơn giản là tới nơi dưới dạng vô danh — nên gom về một chỗ là khoản bảo hiểm rẻ.
- **DTO và exception** là chia sẻ cho tiện: dễ chịu trong monorepo, và là thứ đầu tiên nên nhân-bản-thay-vì-chia-sẻ nếu các service có ngày tách ra thành các repository riêng, vì các class tiện ích dùng chung chính là cách những service “độc lập” rất cuộc lại phải phát hành cùng nhau.

QUYẾT ĐỊNH — CÓ NÊN CÓ MODULE DÙNG CHUNG KHÔNG?

Những người theo chủ nghĩa thuần túy microservice cấm chia sẻ code giữa các service. Lựa chọn thuần túy — mỗi service tự định nghĩa bản sao của mọi sự kiện — lại cho phép trôi lệch đúng ở chỗ sự nhất trí là bắt buộc. Dự án này vạch ranh giới ở: chia sẻ *hợp đồng* (sự kiện, tên header), nghĩ kỹ trước khi chia sẻ bất cứ thứ gì có hành vi. `jib.skip=true` trong POM của common đánh dấu nó là module duy nhất là thư viện, không phải thứ để triển khai.

CẠM BÃY — BÃY TRIỂN KHAI ĐỒNG BỘ (LOCKSTEP)

Triệu chứng: sau khi thêm một trường vào `UserRegisteredEvent` và chỉ triển khai `auth-service`, `notification-service` bắt đầu ném lỗi giải tuần tự trên mọi thông điệp — và vì nó dùng chung consumer group, nó cứ thử lại mãi cùng một thông điệp hỏng. Nguyên nhân: hợp đồng thay đổi chỉ ở một phía, và topic giờ chứa sự kiện ở hai hình dạng. Các quy tắc ngăn chuyện đó: chỉ *thêm* trường tùy chọn, không bao giờ đổi tên hay xóa; triển khai consumer trước producer; và cấu hình xử lý lỗi cho consumer (Chương 11) để một thông điệp độc không làm nghẽn cả topic.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Với mỗi trường hợp, nói HTTP hay sự kiện là phù hợp và vì sao: (a) frontend cần danh sách học viên; (b) một bài nộp vừa được chấm và học viên cần được báo; (c) `course-service` phải xác minh một JWT.
2. Vì sao `notification-service` là service duy nhất có manifest Kubernetes phải chờ cột mốc Kafka, trong khi `auth-service` cũng dùng Kafka?
3. Nếu một thứ thuộc về `common` và một thứ không bao giờ nên ở đó, kèm lập luận.
4. Quy tắc thứ tự triển khai nào làm cho thay đổi schema sự kiện an toàn, và bản thân thay đổi đó phải có tính chất gì?

Phần II

Dữ liệu

Dữ liệu quan hệ: Postgres, JPA, Flyway

Hai trong số các service, mỗi service sở hữu một cơ sở dữ liệu PostgreSQL: auth-service sở hữu `ts_auth`, course-service sở hữu `ts_course`. Chương này bàn ba tầng nằm giữa một phương thức @Service và các byte trên đĩa — JPA/Hibernate, connection pool, và Flyway — cùng quy tắc kiến trúc giải thích vì sao lại có tới *hai* cơ sở dữ liệu.

7.1 Mỗi service một cơ sở dữ liệu

QUYẾT ĐỊNH — HAI SERVER POSTGRES, KHÔNG PHẢI HAI SCHEMA TRONG MỘT

Mỗi service sở hữu dữ liệu của mình một cách độc quyền; đường duy nhất tới dữ liệu của service khác là qua API hoặc sự kiện của nó. Điều này cấm kiểu thất bại kinh điển của cơ sở dữ liệu dùng chung — service A lặng lẽ join vào bảng của service B, trói schema hai bên với nhau mãi mãi. Dự án đi xa hơn mức cần thiết: không chỉ cơ sở dữ liệu riêng mà cả tiến trình server riêng, nên ngay cả kết nối chéo vô tình cũng bất khả, và mỗi bên có thể được tinh chỉnh, sao lưu, hay làm hỏng độc lập. Chi phí trên máy này: khoảng 256 MiB cho mỗi server thêm. Trong một tài khoản cloud, cùng mức cô lập đó thường là hai cơ sở dữ liệu trên một instance RDS được quản lý — cô lập bằng thông tin đăng nhập thay vì bằng tiến trình — vì instance được quản lý tính tiền theo server.

7.2 JPA và Hibernate

JPA là đặc tả (các annotation như @Entity, @OneToMany, EntityManager); Hibernate là bản cài đặt mà Spring Boot tự cấu hình. Spring Data JPA nằm bên trên, suy ra truy vấn từ tên phương thức repository (findByCourseIdAndStudentId) và sinh phần mã lập.

Cấu hình từ configurations/course-service.yml:

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/ts_course ①
```

```

username: postgres
password: root
jpa:
  hibernate:
    ddl-auto: none
    show-sql: true
flyway:
  enabled: true

```

- ① Mặc định khi dev. Compose ghi đè host thành postgres-course; Kubernetes ghi đè y hệt qua SPRING_DATASOURCE_URL. Để ý Service k8s được cố ý đặt tên postgres-course để chuỗi ghi đè giống nhau ở cả hai môi trường.
- ② Thông tin đăng nhập dạng văn bản thuần chỉ chấp nhận được vì file này là *mặc định khi dev*; trong cụm, SPRING_DATASOURCE_USERNAME/PASSWORD tới từ một Secret Kubernetes (Chương 16) và có ưu tiên cao hơn.
- ③ Dòng quan trọng nhất trong khối — Mục 7.3.
- ④ Mọi câu SQL đều được log — một công cụ dạy học và một máy dò truy vấn N+1 khi dev; tắt đi ở nơi dung lượng log tốn tiền.
- ⑤ Quản lý schema thuộc về Flyway, không phải Hibernate.

7.3 Vì sao ddl-auto: none

Hibernate có thể sinh schema từ entity (ddl-auto: update), và tutorial nào cũng dùng nó. Hệ thống production thì không, vì update là cái bánh cóc một chiều không có lịch sử: nó thêm cột nhưng không bao giờ xóa hay đổi tên một cách an toàn; cơ sở dữ liệu của hai lập trình viên lệch nhau âm thầm; không có bản ghi nào về cái gì đổi khi nào, và không có cách nào xem xét một thay đổi schema trước khi nó chạm tới production.

7.4 Flyway: migration dưới dạng code

Flyway coi schema là một chuỗi có thứ tự các script SQL bất biến, mỗi script áp dụng đúng một lần, được theo dõi trong một bảng (flyway_schema_history) ngay trong cơ sở dữ liệu đích:

```

auth-service/src/main/resources/db/migration/
V1__create_users.sql
V2__create_refresh_tokens.sql
V3__add_admin_user_invitations.sql
V4__seed_default_admin.sql

```

Lúc khởi động, Flyway so sánh lịch sử với các script trên classpath và áp dụng những script còn thiếu, theo thứ tự phiên bản, trước khi Hibernate kịp khởi tạo. Các quy tắc làm nó hoạt động: không bao giờ sửa một migration đã áp dụng (mỗi dòng lưu một checksum — sửa sẽ bị phát hiện và từ chối); chỉ thêm phiên bản mới; và viết migration tương thích với phiên bản ứng dụng *trước đó* khi bạn triển khai cuốn chiếu (Chương 15): thêm-rồi-chuyển-rồi-xóa, không bao giờ đổi tên trong một bước.

CẠM BẦY — CHẠY TỐT Ở MÁY MÌNH, CRASHLOOPBACKOFF TRONG CỤM

Triệu chứng: service khởi động được trên máy bạn nhưng pod của nó khởi động lại không ngừng; log cho thấy Flyway lệch checksum hoặc một migration thất bại. Nguyên nhân: cơ sở dữ liệu cục bộ đã được migrate bằng một script cũ hơn (hoặc đã bị sửa) — bảng lịch sử trên laptop của bạn và của cụm không khớp nhau. Đây cũng là lý do startup probe của Kubernetes (Chương 17) nhắm vào một endpoint chỉ trả lời sau khi Spring context đã lên: “migration đã áp dụng” là một phần của nghĩa *sẵn sàng*. Sửa bằng cách thêm một migration hiệu chỉnh, không bao giờ bằng cách sửa lịch sử.

7.5 Connection pool

Mở một kết nối Postgres tốn một bắt tay TCP, xác thực, và một lần fork tiến trình server — quá đắt cho mỗi request. HikariCP (pool mặc định của Spring Boot) giữ một nhóm nhỏ kết nối đang mở để request mượn rồi trả. Tối đa mặc định: 10.

TRÊN PHẦN CỨNG NÀY

Truyền miệng về kích thước pool nói “nhiều kết nối hơn, thông lượng cao hơn”; thực tế trên server 2 nhân thì ngược lại — Postgres làm việc *theo từng kết nối*, và công thức thực dụng ($\approx$ số nhân $\times$ 2) cho thấy mặc định 10 đã là hào phóng với hai service chia nhau hai nhân. Nếu Postgres có lúc báo too many clients, hãy hạ pool xuống trước khi nâng max_connections.

7.6 Ngoài dự án

Phiên bản cloud được quản lý tương đương với thiết lập của chương này — AWS RDS Postgres với cùng luồng Flyway — chỉ thay đổi URL và mô hình tin cậy:

```
spring:
```



```
datasource:  
  url: jdbc:postgresql://app-db.xxxxx.eu-central-1.rds.amazonaws.com:5432/ts_course  
  username: ${DB_USER}      # from AWS Secrets Manager  
  password: ${DB_PASSWORD}
```

Sao lưu, sao chép (replication) và failover thành việc của nhà cung cấp — đó là lý do cảnh báo của Chương 19 về trạng thái tự host lại quan trọng. Biến thể công nghiệp thứ hai đáng biết: chạy Flyway như một bước CI riêng (flyway migrate vào đích) thay vì lúc ứng dụng khởi động, để thay đổi schema triển khai độc lập với code và một migration chậm không ăn hết ngân sách của startup probe.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Nêu hai kiểu thất bại cụ thể của ddl-auto: update trong môi trường làm việc nhóm mà none + Flyway ngăn được.
2. Vì sao không bao giờ được sửa một migration đã áp dụng, và Flyway dùng gì để phát hiện?
3. Bạn cần đổi tên một cột mà không có downtime dưới triển khai cuốn chiếu. Phác thảo chuỗi nhiều bản phát hành.
4. Bảo vệ và phản bác quyết định hai server: mỗi phía một lập luận, và bối cảnh nào thì phía nào thắng.

Dữ liệu dạng document: MongoDB

8.1 Mô hình

MongoDB lưu các *document* — cấu trúc giống JSON (BSON) — trong các *collection*, không có schema cố định. Ở chỗ Postgres chuẩn hóa một mục từ điển thành các hàng rải trên các bảng *words*, *definitions*, *examples* nối với nhau bằng khóa, Mongo lưu cả mục đó thành một document lồng nhau:

```
{ "word": "ubiquitous",
  "phonetic": "yoo-BIK-wi-tuhs",
  "meanings": [
    { "partOfSpeech": "adjective",
      "definitions": [
        { "definition": "present everywhere",
          "example": "smartphones are ubiquitous" } ] } ] }
```

QUYẾT ĐỊNH — VÌ SAO TỪ ĐIỂN LÀ SERVICE DÙNG MONGO

Một mục từ điển được đọc như một khối, ghi như một khối, và lồng nhau một cách tự nhiên — trường hợp đẹp nhất của mô hình document. Không có transaction xuyên mục, không có join nào để mất. Ngược lại, khóa học và ghi danh thực sự có tính quan hệ (một bài nộp tham chiếu một học viên và một bài tập; tính nhất quán giữa chúng là quan trọng), nên chúng ở lại Postgres. “Polyglot persistence” chính là thế này: chọn theo từng service, theo hình dạng dữ liệu — không phải một cơ sở dữ liệu được sùng ái cho mọi thứ, cũng không phải chạy theo cái mới cho vui.

8.2 Spring Data MongoDB

Mô hình lập trình giống JPA đủ gần để dictionary-service làm một phiên đá Rosetta tốt: @Document thay cho @Entity, MongoRepository thay cho JpaRepository, các phương thức truy vấn suy diễn giống hệt về tinh thần. Những gì biến mất: Flyway và DDL schema (collection tự xuất hiện ở lần ghi đầu tiên), dialect, và việc tinh chỉnh pool (driver tự quản lý pool kết nối của nó). Những gì xuất hiện: một URI kết nối duy nhất.

8.3 Cấu hình — toàn bộ

Toàn bộ cấu hình dữ liệu của dictionary-service là một dòng, cộng với bản ghi đề cho cụm:

```
# configurations/dictionary-service.yml (dev default)
spring:
  data:
    mongodb:
      uri: mongodb://localhost:27017/ts_dictionary

# deploy/k8s/base/dictionary-service.yaml (the override)
- name: SPRING_DATA_MONGODB_URI
  value: "mongodb://mongodb:27017/ts_dictionary"
```

URI mang host, cổng, tên database, và (khi bật) thông tin đăng nhập cùng các tùy chọn — một chuỗi duy nhất để ghi đề cho mỗi môi trường, đúng việc mà cơ chế relaxed binding ở Chương 1 sinh ra để làm.

8.4 Không schema là con dao hai lưỡi

Không có migration không có nghĩa là không có schema — nghĩa là schema sống *ngầm trong code*. Hai document do hai phiên bản ứng dụng ghi ra có thể có hình dạng khác nhau trong cùng một collection, và không gì ngoài code đọc của bạn nhận ra điều đó. Các kỷ luật thay thế Flyway: giữ các lớp document đọc ngược tương thích (trường mới là tùy chọn), viết script sửa dữ liệu khi hình dạng thật sự thay đổi, và kiểm tra hợp lệ ở biên. Mongo cũng có thể áp JSON Schema cho từng collection — tùy chọn bật thêm, và không dùng ở đây.

CẠM BẦY — COLLECTION CHƯA BAO GIỜ ĐƯỢC TẠO

Triệu chứng: service khởi động, truy vấn chạy, mọi thứ trả về rỗng — không có lỗi ở đâu cả. Nguyên nhân: URI trở vào một database mới tinh (gõ nhầm, hoặc một PVC mới trong cụm); Mongo lặng lẽ tạo một `ts_dictionary` rỗng ở lần chạm đầu tiên. Postgres sẽ thất bại ầm ĩ (*database does not exist*); cơ chế tạo-khi-ghi của Mongo biến một địa chỉ sai thành kết quả rỗng. Khi một service dùng Mongo “mất” dữ liệu, hãy kiểm tra *nó đang nói chuyện với database nào* trước mọi thứ khác: `kubect1 exec vào node của pod và chạy mongosh --eval 'show dbs'`.

8.5 Vận hành trong cụm này

Manifest (`deploy/k8s/base/mongodb.yaml`) theo cùng mẫu `StatefulSet` như `Postgres` (Chương 15 giải thích loại object này; Chương 19 nói về lưu trữ), với hai lưu ý riêng của `Mongo`, cả hai đều lấy từ comment trong file: `readiness probe exec mongosh --eval "db.adminCommand('ping')"` — “đang nhận lệnh,” chứ không chỉ “cổng đã mở” — và nó chạy không có xác thực, chấp nhận được chỉ vì đây là một `Service ClusterIP` không có `Ingress`: không thể chạm tới từ ngoài cụm. Khoảng khắc giả định đó thay đổi (một `NodePort` lộ ra ngoài, nhiều `tenant`), xác thực phải là việc đầu tiên.

8.6 Ngoài phạm vi dự án

Phiên bản được quản lý là `MongoDB Atlas`; `URI` có thêm thông tin đăng nhập và `TLS`, ngoài ra không gì thay đổi:

```
spring:
  data:
    mongodb:
      uri: mongodb+srv://app:${MONGO_PWD}@cluster0.abc.mongodb.net/ts_dictionary?
      ↪ retryWrites=true
```

Để đối chiếu, `DynamoDB` là một kho document với triết lý ngược lại: không schema như `Mongo`, nhưng các mẫu truy cập phải được thiết kế sẵn vào bảng (`partition key`, không có truy vấn tùy hứng) — một lời nhắc rằng “`NoSQL`” là một họ các đánh đổi, không phải một thứ duy nhất.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Những tính chất nào của dữ liệu từ điển khiến nó hợp với mô hình document, và những tính chất nào của dữ liệu ghi danh khiến nó không hợp?
2. “Không schema” chuyển việc áp đặt schema từ database sang đâu? Nêu hai kỷ luật thay thế cho migration.
3. Vì sao tên database sai thất bại âm ỉ trong `Postgres` nhưng lặng lẽ trong `Mongo`, và bạn thích kiểu thất bại nào hơn ở production?
4. Pod `Mongo` không có mật khẩu trong khi `Postgres` có, trong cùng một cụm. Dừng lại lập luận và nêu điều kiện làm lập luận đó mất hiệu lực.

Lưu trữ đối tượng: MinIO và mô hình S3

9.1 Bài toán

Bài nộp của học viên là file — PDF, ảnh, file nén. File trong cơ sở dữ liệu quan hệ làm phình bản sao lưu và stream rất tệ; file trên đĩa cục bộ của pod biến mất cùng pod. Câu trả lời của ngành là *lưu trữ đối tượng* (object storage): một dịch vụ HTTP lưu các blob bất biến trong các *bucket* có tên, dưới các *key* dạng chuỗi, kèm metadata, không có thư mục (dấu / trong key là quy ước đặt tên, không phải cấu trúc). Amazon S3 định nghĩa API; MinIO là một server tự host nói cùng API đó — nên course-service dùng AWS SDK chuẩn và không phân biệt được sự khác nhau.

9.2 Cấu hình, có chú giải

Từ configurations/course-service.yml:

```
app:
  s3:
    endpoint: ${APP_S3_ENDPOINT:http://localhost:9000} ①
    public-endpoint: ${APP_S3_PUBLIC_ENDPOINT:http://localhost:9000} ②
    access-key: ${APP_S3_ACCESS_KEY:minioadmin} ③
    secret-key: ${APP_S3_SECRET_KEY:minioadmin}
    bucket: ${APP_S3_BUCKET:ts-submissions}
    presigned-url-expiry-minutes: ${APP_S3_PREIGNED_EXPIRY_MIN:15} ④
```

- ① Nơi *service* nói chuyện với MinIO. Trong cụm: `http://minio:9000`.
- ② Nơi *trình duyệt* nói chuyện với MinIO — chủ đề của cái bẫy bên dưới.
- ③ Thông tin đăng nhập với giá trị mặc định cho dev; cụm bơm giá trị thật từ `Secret minio-app.app.s3.*` là cấu hình tùy biến (`@ConfigurationProperties`), không do Spring quản lý — cơ chế ghi đè vẫn áp dụng như nhau.
- ④ Một link `presigned` còn hiệu lực trong bao lâu.

9.3 Presigned URL: mẫu đáng để biết

Cách tải file ngay thơ là dẫn byte *xuyên qua* service: trình duyệt → service → MinIO và ngược lại, chiếm một thread ứng dụng cho mỗi lượt truyền. Cách làm chuẩn của S3: service dùng thông tin đăng nhập của mình để *ký một URL* — bucket, key, hạn hiệu lực, chữ ký HMAC — rồi đưa URL đó cho trình duyệt, và trình duyệt tải *trực tiếp* từ kho lưu trữ. Service tồn vài micro giây; kho lưu trữ làm việc stream; link tự hết hạn. Thủ thuật này cũng dùng được cho upload.

CẠM BẦY — BÀI TOÁN HAI ENDPOINT

Triệu chứng: upload chạy tốt, nhưng bấm vào link tải thì hỏng với ERR_NAME_NOT_RESOLVED trên minio:9000. Nguyên nhân: một presigned URL nhúng sẵn endpoint *mà nó được ký với*. Service ký với minio:9000 — một cái tên chỉ phân giải được bên trong mạng của cụm — còn trình duyệt sống bên ngoài. Vì thế mới có hai endpoint cấu hình: ký link tải với public-endpoint, nói chuyện với kho lưu trữ qua endpoint. Mọi hệ thống S3 đóng container đều gặp lỗi này một lần; một số gặp lại lần nữa với chính chữ ký, vì chữ ký bao trùm cả host — bạn không thể ký với host này rồi sau đó viết lại URL sang host khác.

TRÊN PHẦN CỨNG NÀY

MinIO ở đây là một container đơn với một PVC đơn — ổn cho bài nộp trong một homelab, chẳng giống gì MinIO production (nhiều node, erasure coding) hay độ bền mười một số chín của S3. Điểm thú vị là course-service không phân biệt được: cùng những lời gọi SDK đó sẽ chạm tới S3 thật chỉ với endpoint và thông tin đăng nhập thay đổi. Đó chính là ý nghĩa của việc nói một API chuẩn.

9.4 Ngoài phạm vi dự án

Cùng service này trên AWS thật: đặt endpoint và public-endpoint thành URL S3 thật (hoặc để mặc định của SDK), thay key tính bằng một IAM role mà pod đảm nhận (IRSA trên EKS — hoàn toàn không có thông tin đăng nhập sống lâu), và bật versioning cho bucket cùng các quy tắc vòng đời (tự động chuyển bài nộp cũ sang tầng lưu trữ rẻ hơn). Các khái niệm — bucket, key, ký trước, danh tính thực hiện ký — chuyển sang nguyên vẹn; thứ thay đổi là ai quản lý độ bền và danh tính được chứng minh ra sao.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Vì sao presigned URL mở rộng tốt hơn việc proxy tải file qua service, và chúng đưa vào kiểu lỗi mới nào?
2. Giải thích vì sao endpoint và public-endpoint phải khác nhau bên trong cụm, và vì sao viết lại URL sau khi ký không phải là cách sửa.
3. Một presigned URL bị lộ trong log chat. Điều gì giới hạn thiệt hại, theo thiết kế?
4. Điều gì thay đổi — và điều gì không — khi code này chuyển từ MinIO sang S3 thật?

Phần III

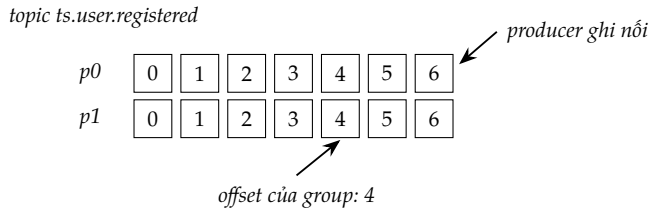
Truyền thông điệp

Kafka: Mô hình lỗi

Kafka là công nghệ đáng học sâu nhất trong stack này: nó xuất hiện trong một phần lớn các mô tả công việc backend, và mô hình của nó thực sự khác với một hàng đợi thông điệp (message queue). Chương này là Kafka độc lập với nền tảng; Chương 11 là phía Spring.

10.1 Một log, không phải một hàng đợi

Trừu tượng lỗi của Kafka là một *log chỉ ghi nối thêm* (append-only log). Một *topic* (ví dụ `ts.user.registered`) được chia thành các *phân vùng* (partition); mỗi partition là một dãy *record* có thứ tự và bất biến, mỗi record ở một *offset* được đánh số. Record không bị xóa khi đọc — chúng hết hạn theo chính sách retention (dự án này: 168 giờ). Đọc không phá hủy; consumer chỉ nhớ *mình đã đọc tới đâu*.



Hình 10.1: Hai partition; producer ghi nối thêm ở đầu, consumer group nhớ một offset cho mỗi partition.

Các hệ quả, mà một hàng đợi không có cái nào:

- **Phát lại được.** Một consumer mới có thể bắt đầu từ offset 0 và thấy lịch sử. Sang năm triển khai một service phân tích mới; nó có thể xử lý các lượt đăng ký của tuần trước.
- **Nhiều người đọc.** Notification-service và course-service cùng đọc sự kiện auth một cách độc lập, theo nhịp riêng, không giành của nhau — mỗi *consumer group* giữ offset riêng của mình.
- **Có thứ tự trong từng partition.** Các record có cùng *key* bám về cùng một partition và giữ nguyên thứ tự. Đây là lý do các publisher trong dự án này dùng `userId` làm key: mọi sự kiện về một người dùng đến theo đúng trình

tự. Giữa các partition thì không có thứ tự — chỉ thiết kế dựa trên thứ tự theo key.

10.2 Consumer group

Một consumer group là một đội có tên chia nhau việc: mỗi partition được gán cho đúng một consumer *trong group*. Hai consumer trong group notification-group trên một topic 2 partition nhận mỗi bên một partition; consumer thứ ba ngồi không — *partition là đơn vị của tính song song*. Khi một consumer tham gia hoặc chết, group *rebalance*: partition được gán lại và các thành viên tạm dừng trong chốc lát. Các group khác nhau vô hình với nhau, và đó là cách cùng một sự kiện tỏa ra cả notification-group lẫn course-group.

10.3 Broker, replication, KRaft

Một cụm production chạy nhiều *broker*; mỗi partition có một leader trên một broker và các *replica* trên những broker khác (replication-factor, RF). Producer ghi vào leader; acks=all chờ cho tới khi các replica đồng bộ (ISR) đã có bản ghi. Việc điều phối metadata trước đây cần ZooKeeper, một hệ phân tán thứ hai; Kafka hiện đại thay nó bằng *KRaft* — các broker tự chạy một quorum Raft nội bộ. Dự án này chạy KRaft với một node đóng cả hai vai broker và controller, RF = 1: ổn cho homelab, khả năng chịu lỗi bằng không — một đĩa hỏng là mất log.

10.4 Mê cung advertised listener

Lỗi Kafka được Google nhiều nhất. Từ docker-compose.yml:

```
KAFKA_LISTENERS: PLAINTEXT://:19092,CONTROLLER://:9093,EXTERNAL://:9092 ①
KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:19092,EXTERNAL://localhost:9092 ②
KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093 ③
```

- ① *Listener*: các socket mà broker mở. Ba cái: một cho client bên trong mạng Docker, một cho controller KRaft, một công bố ra máy host.
- ② *Advertised listener*: các địa chỉ mà broker *bảo client hãy dùng*. Đây là cái bẫy: yêu cầu đầu tiên của một Kafka client là “leader của các partition ở đâu?”, và broker trả lời bằng những chuỗi này. Container phải được bảo là kafka:19092 (phân giải được trên mạng Docker); công cụ trên laptop của bạn phải được bảo là localhost:9092. Một broker, hai đối tượng, hai địa chỉ advertised.

- ③ Quorum KRaft gồm một thành viên: node 1 là toàn bộ cuộc bầu cử.

CẠM BẦY — KẾT NỐI ĐƯỢC, RỒI TIMEOUT

Triệu chứng: client kết nối tới địa chỉ bootstrap thành công, rồi mọi send/poll đều thất bại với Connection to node -1 could not be established hoặc treo. Nguyên nhân: bootstrap chạy được, nhưng *câu trả lời metadata advertise* một địa chỉ mà client không với tới (ví dụ laptop của bạn bị bảo là kafka:19092). Địa chỉ bootstrap chỉ mở đầu cuộc trò chuyện; advertised listener mang toàn bộ lưu lượng sau đó. Chẩn đoán bằng cách hỏi broker advertise cái gì — ví dụ kcat -L -b localhost:9092 — và kiểm tra đúng những chuỗi đó có phân giải được từ nơi client chạy hay không.

10.5 Ngữ nghĩa chuyển giao

Mặc định của Kafka là *ít nhất một lần* (at-least-once): producer gửi lại sau khi mất ack, hoặc consumer sập sau khi xử lý nhưng trước khi commit offset, cả hai đều gây trùng lặp — chứ không mất. Hãy thiết kế consumer *lũy đẳng* (idempotent: xử lý cùng một sự kiện hai lần là vô hại): notification-service gửi trùng một email là chấp nhận được; một service thanh toán thì sẽ phải theo dõi ID các sự kiện đã xử lý. Đúng một lần (exactly-once) có tồn tại (producer lũy đẳng + transaction) nhưng trả giá bằng độ phức tạp; hãy biết rằng ít nhất một lần + consumer lũy đẳng là món cơm bữa của cả ngành.

10.6 Ngoài phạm vi dự án

Kafka được quản lý (AWS MSK, Confluent Cloud) bỏ đi việc vận hành broker và bán cùng giao thức đó — client của bạn đổi một chuỗi bootstrap và có thêm SASL/TLS. Những cái tên khác trong mảng này giải quyết những bài toán khác: RabbitMQ là một *hàng đợi* định tuyến thông minh (phân phối công việc, ack theo từng thông điệp, không phát lại); Redpanda hiện thực lại giao thức Kafka mà không cần JVM. Khi Chương 25 chuyển broker này vào cụm dưới Strimzi, không gì trong chương này thay đổi — Strimzi là *vận hành* cho cùng một Kafka.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Các sự kiện đăng ký của một người dùng phải được xử lý theo thứ tự, nhưng khả năng mở rộng cũng quan trọng. Bạn dùng gì làm key, bao nhiêu partition, và giới hạn song song là gì?

2. Hai group, notification-group và course-group, cùng đọc `ts.user.activated`. Giải thích cách cả hai nhận được mọi sự kiện.
3. Phân biệt `LISTENERS` với `ADVERTISED_LISTENERS`, và giải thích vì sao lỗi chúng gây ra xuất hiện *sau* một kết nối thành công.
4. Vì sao chuyển giao ít nhất một lần đẩy tính lũy đăng về phía consumer, và chiến lược lũy đăng hợp lý của notification-service sẽ là gì?

Spring Kafka: Producer, Listener, Poison Pill

11.1 Phát sự kiện

Phát một sự kiện trong dự án này gồm hai bộ phận: một `KafkaTemplate` và cấu hình chọn serializer. Từ `auth-service`:

```
kafkaTemplate.send("ts.user.registered", event.userId(), event);
//                topic                key                value
```

```
spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
```

Key (`userId`) là một chuỗi; value là record sự kiện được tuần tự hóa thành JSON. Dùng `user id` làm key là quyết định về thứ tự ở Chương 10 được cụ thể hóa. `send()` là bất đồng bộ — nó trả về một `future`; thông điệp nằm trong bộ đệm phía client cho tới khi broker `ack`. Hai hệ quả vận hành: một service có thể *khởi động* mà không có broker (các lần gửi thất bại về sau, theo từng lời gọi — đó là lý do `auth-service` có thể triển khai lên cụm trước Kafka), và một service sập ngay sau khi `send()` trả về có thể mất thông điệp đang nằm trong bộ đệm — mẫu `outbox` bên dưới tồn tại cho khi điều đó quan trọng.

11.2 Tiêu thụ

```
@KafkaListener(topics = "ts.user.registered",
                groupId = "notification-group")
public void on(UserRegisteredEvent event) {
    mailService.sendActivation(event);
}
```

Spring chạy một vòng lặp poll cho mỗi listener container, giải tuần tự từng record, gọi phương thức, và commit offset sau khi xử lý thành công — ít nhất một lần ngay từ đầu. Nửa consumer của file YAML chứa những dòng liên quan tới production nhất của chương này:

```
spring:
  kafka:
    consumer:
      group-id: course-group
      auto-offset-reset: earliest ①
      key-deserializer: ...ErrorHandlingDeserializer ②
      value-deserializer: ...ErrorHandlingDeserializer
      properties:
        spring.deserializer.value.delegate.class: ...JsonDeserializer
        spring.json.trusted.packages: com.ts.common.dto ③
```

- ① Nơi một group *hoàn toàn mới* bắt đầu khi chưa có offset đã commit: earliest = từ đầu khoảng retention; mặc định latest = chỉ thông điệp mới. earliest nghĩa là một consumer triển khai muộn vẫn xử lý các sự kiện đã phát trước khi nó tồn tại — lựa chọn đúng khi sự kiện là những sự thật cần hành động. Không còn ý nghĩa một khi group đã có offset.
- ② Lá chắn chống poison pill — mục tiếp theo.
- ③ Giải tuần tự JSON khởi tạo các lớp được nêu tên trong header thông điệp; chỉ tin cậy com.ts.common.dto ngăn một producer độc hại hoặc có lỗi khiến consumer khởi tạo lớp tùy ý. Một ranh giới bảo mật, không phải mã lập.

11.3 Poison pill

Một *poison pill* (viên thuốc độc) là thông điệp mãi mãi không giải tuần tự được — lỗi ở producer, một trường bị đổi tên, payload hỏng. Không có phòng thủ, listener ném lỗi *trước khi* thông điệp có thể bị bỏ qua, offset không bao giờ tiến lên, và container thử lại cùng một record không ngừng: partition bị kẹt và log đầy lên với tốc độ tối đa.

ErrorHandlingDeserializer bọc deserializer thật: khi thất bại nó phát ra một lỗi có kiểu thay vì ném ngoại lệ bên trong vòng lặp poll, để error handler của Spring bỏ qua record đó (tùy chọn sau vài lần thử lại, tùy chọn phát nó sang một dead-letter topic). Cách ghép — error handler bên ngoài, JSON delegate bên trong — là mẫu production chuẩn, và là lý do cấu hình nêu tên *hai* lớp deserializer cho mỗi bên.

CẠM BẦY — MỘT THÔNG điệp HỒNG, IM LẶNG HOÀN TOÀN

Triệu chứng: email thông báo dừng hẳn; consumer ghi cùng một stack trace giải tuần tự hàng nghìn lần. Nguyên nhân: một poison pill trên một partition mà không có deserializer xử lý lỗi — vòng lặp không xử lý được cũng không bỏ qua được. Lỗi này ẩn mình trong những hệ thống chưa bao giờ thử với một thông điệp sai định dạng. Hãy kiểm chứng lá chắn của bạn một lần, có chủ đích: phát rác vào một topic dev và xem nó bị bỏ qua.

11.4 Những góc nâng cao

Dead-letter topic. `DefaultErrorHandler` + `DeadLetterPublishingRecordRecoverer`: sau N lần thử thất bại, record rơi vào `<topic>.DLT` để xem xét và phát lại — xử lý lỗi có dấu vết kiểm toán.

Mẫu outbox. “Lưu người dùng và phát sự kiện” là hai hệ thống và không thể nguyên tử một cách trực tiếp. Outbox: ghi sự kiện vào một bảng outbox trong cùng *transaction cơ sở dữ liệu* với hàng người dùng; một relay (hoặc Debezium đọc WAL) phát từ bảng đó. Nếu một ngày auth-service không được phép mất sự kiện đăng ký, đây là câu trả lời.

Schema registry. Các lớp sự kiện dùng chung ở Chương 6 giải quyết việc producer/consumer thống nhất với nhau bên trong một repo. Ở quy mô tổ chức, hợp đồng chuyển vào một schema registry (schema Avro/Protobuf, được kiểm tra tương thích lúc phát) — cùng ý tưởng, nhưng do hạ tầng áp đặt thay vì một jar dùng chung.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Vì sao auth-service khởi động được khi Kafka đang sập, và đến thời điểm nào thì một sự cố Kafka trở nên hữu hình với nó?
2. Một consumer group mới được triển khai hôm nay. Giải thích `auto-offset-reset: earliest` khiến nó làm gì, và khi nào thiết lập này không còn tác dụng với group đó.
3. Đi qua kịch bản poison pill khi không có `ErrorHandlingDeserializer`, rồi khi có. Trong một hệ thống production đầy đủ, record hồng cuối cùng nằm ở đâu?
4. Vì sao `spring.json.trusted.packages` là một biện pháp kiểm soát bảo mật?
5. Khi nào mẫu outbox là cần thiết, và nó làm cho cặp thao tác ghi nào trở nên nguyên tử một cách hiệu quả?

Phần IV

Container

Docker: Image, Container, Compose

12.1 Container thực chất là gì

Container không phải là một máy ảo thu nhỏ. Nó là một tiến trình Linux bình thường, được cô lập bởi hai tính năng của kernel: *namespace* (góc nhìn riêng về tiến trình, mạng, các mount của hệ thống file, hostname) và *cgroups* (giới hạn CPU và bộ nhớ). Không có hệ điều hành khách, không có hypervisor — đó là lý do container khởi động trong vài mili giây, và cũng là lý do JVM bên trong phải được báo về giới hạn bộ nhớ của nó (MaxRAMPercentage của Chương 17) — vì nhìn từ bên trong, nó thấy RAM của *máy chủ*.

Image là hệ thống file đã đóng gói cộng với metadata (entrypoint, biến môi trường, cổng khai báo), được build thành một chồng các *layer* bất biến. Layer được định danh theo nội dung và dùng chung: năm mươi image FROM eclipse-temurin:25-jre-alpine chỉ lưu layer JRE một lần. *Registry* (Docker Hub, GHCR, registry:2 của dự án) lưu trữ image; docker push/pull chỉ chuyển những layer mà phía bên kia còn thiếu.

12.2 Dockerfile của dự án, có chú giải

Service nào cũng mang đúng năm dòng này:

```
FROM eclipse-temurin:25-jre-alpine ①
WORKDIR /app
COPY target/*.jar app.jar ②
EXPOSE 8082 ③
ENTRYPOINT ["java", "-jar", "app.jar"] ④
```

- ① Image gốc: một JRE (không phải JDK — lúc chạy không cần trình biên dịch) trên Alpine Linux. Image gốc nhỏ hơn = bề mặt tấn công nhỏ hơn và pull nhanh hơn.
- ② Sao chép fat jar do Maven build ra. Điểm tinh tế: Dockerfile này đòi hỏi mvn package phải chạy *trước* — việc build diễn ra bên ngoài. Hệ quả: một lệnh COPY cho jar cỡ ~60 MB tạo ra một layer to, nên *mọi* lần build đều push lại toàn bộ dù chỉ sửa một dòng. Jib (Chương 13) sinh ra để sửa đúng chuyện này.

- ③ Chỉ là tài liệu, không phải hành động — nó không mở gì cả. Việc công bố cổng diễn ra lúc chạy.
- ④ Entrypoint dạng exec: java là PID 1, nhận tín hiệu trực tiếp, nên docker stop kích hoạt quá trình tắt Spring sạch sẽ.

12.3 Mạng và Compose

Các container trên một *bridge network* của Docker tìm thấy nhau bằng *tên service* — Docker chạy một DNS server trả lời cho postgres-course, kafka, minio. Còn localhost, bên trong một container, chính là *container đó* — cái bấy số một với người mới, và là lý do mọi URL mặc định cho dev trong các file cấu hình đều cần ghi đề theo từng môi trường. Cổng chỉ với tới được từ máy chủ khi đã được *công bố* (-p 5434:5432).

Compose biến một loạt lệnh docker run thành một file khai báo duy nhất — docker-compose.yml của dự án là cả stack dev: bốn kho dữ liệu, Kafka, Zipkin, Maildev và bảy service, với ba thành ngữ đáng gọi tên:

```
discovery-server:
  depends_on:
    config-server:
      condition: service_healthy ①
config-server:
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:8888/actuator/health"] ②
postgres-course:
  ports:
    - "5434:5432" ③
  volumes:
    - postgres-course-data:/var/lib/postgresql/data
```

①+② mã hóa thứ tự khởi động — và không chỉ “đã chạy” mà là “đã trả lời endpoint sức khỏe,” điều làm cho thứ tự này có ý nghĩa thật. ③ ánh xạ hai cổng máy chủ khác nhau (5433, 5434) vào cùng cổng nội bộ 5432 của hai container Postgres — tên thì riêng theo từng mạng, cổng máy chủ thì toàn cục. Volume có tên giữ dữ liệu qua những lần tạo lại container; Chương 19 là phiên bản Kubernetes của cùng nhu cầu này.

QUYẾT ĐỊNH — VÌ SAO COMPOSE VẪN SỐNG CẠNH KUBERNETES

Cụm (cluster) là đích triển khai, nhưng Compose vẫn là vòng lặp bên trong: một lệnh, chạy ngay trên laptop, cổng debug đã map sẵn (mọi service đều mở một cổng JDWP trong Compose — gắn debugger của IDE vào 5082 và bước từng dòng qua course-service). Kubernetes mua được tự phục hồi và triển

khai cuốn chiếu bằng cái giá là nghi thức rườm rà; một lập trình viên đang lặt đi lặt lại trên logic nghiệp vụ không muốn trả cả hai. Hai môi trường, một hợp đồng image chung giữa chúng.

CẠM BẦY — CHẠY TỐT TRONG COMPOSE, CHẾT TRONG KUBERNETES

Triệu chứng: stack xanh ở máy local; cùng image đó lại crash-loop trong cụm. Những nghi phạm quen thuộc, đều thuộc về môi trường: một URL vẫn trỏ tới tên service của Compose trong khi tên Service bên k8s khác đi; một `depends_on` mà cụm không tôn trọng (Kubernetes không có thứ tự khởi động — Chương 15); hoặc giới hạn bộ nhớ có trong cụm nhưng không có ở local, biến một JVM béo thành OOMKill. Image là y hệt — hãy so *môi trường*, không phải code: `docker inspect` với `kubectl describe pod`.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Container và VM đều cô lập. Nêu hai cơ chế container dùng, và hai hệ quả của việc không có kernel khách.
2. Vì sao `localhost:5432` hỏng bên trong container dù chạy được trên laptop của bạn, và Compose với Kubernetes đưa ra hai cách sửa khác nhau nào?
3. Cả hai container Postgres đều lắng nghe cổng 5432 nội bộ mà không xung đột — giải thích. Xung đột lại có thể xảy ra ở đâu?
4. Điều gì khiến `condition: service_healthy` mạnh hơn `depends_on` thường, và file nào của dự án cung cấp tín hiệu sức khỏe mà nó chờ?

Image không cần daemon: Jib và registry cục bộ

13.1 Vì sao không cứ `docker build` trong CI?

Các Dockerfile vẫn chạy tốt. Ba lý do khiến pipeline build image theo cách khác:

1. **CI không cần Docker daemon.** `docker build` đòi hỏi một daemon đang chạy; cấp cho Jenkins một daemon nghĩa là docker-in-docker hoặc mount socket Docker của máy chủ — tức là quyền root trên máy chủ, trao cho mọi build. Jib build và push image bằng Java thuần: không daemon, không đặc quyền.
2. **Phân layer khớp với cách code thay đổi.** Dockerfile sao chép một fat jar: một layer khổng lồ, push lại toàn bộ mỗi lần build. Jib tách ứng dụng thành các layer theo mức độ biến động — dependency / resource / class — nên sửa một dòng code chỉ push lại layer class nhỏ. Trên hai nhân CPU và Wi-Fi, đó là khác biệt giữa vài giây và vài phút cho mỗi service.
3. **Sẵn sàng cho OpenShift một cách tình cờ.** Image do Jib tạo chạy bằng người dùng không phải root với quyền file thuộc group 0 — đúng điều mô hình bảo mật UID tùy ý của OpenShift đòi hỏi. Tính khả chuyển miễn phí.

13.2 Cấu hình, có chú giải

Jib được cấu hình một lần trong `pluginManagement` của `pom.xml` cha; mỗi module chạy được tham gia bằng một tham chiếu `<plugin>` tron, còn common *rút lui* bằng `jib.skip=true`:

```
<plugin>
  <groupId>com.google.cloud.tools</groupId>
  <artifactId>jib-maven-plugin</artifactId>
  <version>3.4.4</version>
  <configuration>
    <from><image>eclipse-temurin:25-jre-alpine</image></from> ①
    <to><image>${docker.registry}/${project.artifactId}:${project.version}</image></to> ②
    <allowInsecureRegistries>true</allowInsecureRegistries> ③
    <container>
      <mainClass>${start-class}</mainClass> ④
    </container>
  </configuration>
</plugin>
```

```
<creationTime>USE_CURRENT_TIMESTAMP</creationTime> ⑤
</container>
</configuration>
</plugin>
```

- ① Cùng image gốc với các Dockerfile — hành vi lúc chạy không đổi, chỉ khác đường đóng gói.
- ② Phân giải thành localhost:5000/course-service:1.0.0-SNAPSHOT. Tên được viết từ *góc nhìn của cụm* — Mục 13.3.
- ③ Registry cục bộ dùng HTTP thường; Jib từ chối push không an toàn trừ khi được bảo. Mọi công cụ trong chuỗi này (Jib, containerd) đều chọn an-toàn-mặc-định như nhau và đều cần cùng một lời cho phép tường minh.
- ④ Đặt tường minh vì bộ quét bytecode đi kèm Jib chưa đọc được file class của Java 25 — một ví dụ thật về việc Java quá mới làm khổ bộ công cụ.
- ⑤ Jib mặc định dùng mốc thời gian epoch-0 để build tái lập được; ở đây ta đánh đổi điều đó để docker images hiển thị thời gian build thật.

13.3 Registry, và vì sao lại là localhost:5000

```
docker run -d --restart=unless-stopped --name registry \
  -p 5000:5000 -v registry-data:/var/lib/registry registry:2
```

Sự thật không hiển nhiên buộc phải có kiến trúc này: **k3s không nhìn thấy image của Docker**. k3s chạy containerd riêng với kho image riêng; docker build trên máy chủ đặt image vào nơi k3s không bao giờ ngó tới. Registry là giao diện trung thực giữa bên build và cụm — thứ mà pipeline đăng nào cũng cần.

Tên image không chỉ là cái tên — nó là *pull từ đâu*: localhost:5000/course-service nghĩa là “pull từ localhost:5000.” Vì bên pull chính là node k3s, và Jenkins (bên push) dùng chung network namespace với máy chủ (Chương 22), cùng một chuỗi này hợp lệ cho cả hai — và localhost không bao giờ trôi như IP LAN hay hostname có thể. Hai điểm cấu hình làm nó chạy được:

```
# /etc/rancher/k3s/registries.yaml -- containerd assumes HTTPS otherwise
mirrors:
  "localhost:5000":
    endpoint: ["http://localhost:5000"]
```

CẠM BÃY — HTTP: SERVER GAVE HTTP RESPONSE TO HTTPS CLIENT

Lỗi đặc trưng của chương này. Cả Jib (push) lẫn containerd (pull) đều mặc định TLS; registry:2 chỉ nói HTTP thường. Nếu một pod kẹt ở ImagePullBackOff với dòng chữ này trong kubectl describe pod, thì mục trong registries.yaml đang thiếu — và lưu ý containerd *chỉ đọc nó lúc khởi động*: systemctl restart k3s sau khi sửa.

QUYẾT ĐỊNH — REGISTRY CỤC BỘ VS. GHCR

GitHub Container Registry sẽ cho phép pull image từ bất cứ đâu và thêm quét lỗ hổng — với cái giá là đẩy hàng trăm megabyte lên Wi-Fi gia đình mỗi lần build. Registry trong LAN giữ vòng lặp build ở tốc độ ổ đĩa. Quyết định này đảo chiều ngay khi có máy thứ hai cần pull image; khi đó trong manifest không gì thay đổi ngoài tiền tố image.

13.4 Tag, digest, và cái bẫy SNAPSHOT

Tag (:1.0.0-SNAPSHOT, :a1b2c3d) là con trỏ có thể dời; *digest* (@sha256:...) là mã băm nội dung bất biến. Pipeline push mỗi build với hai nhãn: tag SNAPSHOT (mà manifest tham chiếu) và tag git-SHA (mà các lần deploy ghim vào — Chương 23). Kubernetes cache image theo tag; một SNAPSHOT push lại sẽ bị node lặng lẽ bỏ qua — đó là lý do mọi Deployment đều đặt imagePullPolicy: Always: kiểm tra lại registry mỗi lần khởi động; nếu digest khớp, phép kiểm tra chỉ tốn một lần tải manifest.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Nếu ba vấn đề CI mà Jib giải quyết so với build dựa trên daemon, và vấn đề nào là chuyện bảo mật.
2. Vì sao tên image phải viết từ góc nhìn của *cụm*, và điều gì hỏng nếu Jenkins và k3s hiểu localhost:5000 khác nhau?
3. Một đồng nghiệp đề xuất docker save | k3s ctr images import thay cho registry. Cách đó tốn gì mỗi lần deploy, và stage nào của pipeline tự nhiên ngừng hoạt động?
4. Giải thích cách một :1.0.0-SNAPSHOT push lại cộng với pull policy mặc định tạo ra tình huống “tôi đã deploy mà chẳng có gì đổi,” và cả hai cách sửa.

Phần V

Kubernetes

Kubernetes: Giải phẫu cụm và vòng lặp điều hòa

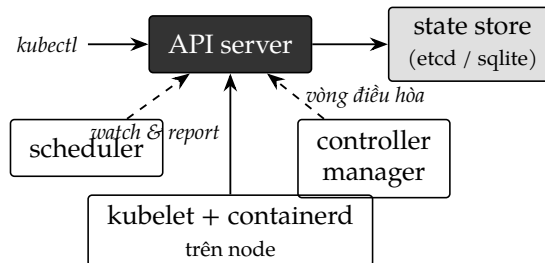
14.1 Ý tưởng duy nhất mà mọi thứ khác treo lên

Bạn không bảo Kubernetes *làm* gì; bạn bảo nó điều gì phải *đúng*. Bạn ghi *trạng thái mong muốn* (“một Deployment tên course-service với 1 replica của image X”) vào cơ sở dữ liệu của nó qua API server; các *controller* chạy vòng lặp điều hòa (reconciliation) mãi mãi, so sánh mong muốn với thực tế và hành động trên phần chênh lệch. Xóa một pod: controller của nó nhận ra thực tế $\neq$ mong muốn và tạo cái khác. Không ai phát lại lệnh của bạn — trạng thái tự hội tụ.

Điều này giải thích những hành vi nếu không sẽ có vẻ lạ:

- `kubectl apply` là một phép gộp vào trạng thái mong muốn, không phải chạy script — apply cùng một file hai lần, lần thứ hai chẳng có gì xảy ra (tính lũy đẳng — thuộc tính mà pipeline dựa vào ở Chương 23).
- Không có “thứ tự khởi động.” Mọi thứ khởi động cùng lúc; service nào cần dependency chưa có sẽ crash và được khởi động lại cho tới khi thể giới sẵn sàng. Crash-loop *trong lúc* hội tụ là vận hành bình thường, không phải lỗi.
- Xóa khỏi file manifest không xóa khỏi cụm — apply chỉ thêm và cập nhật, nó không dọn (prune). Gỡ Ingress course-service cũ đã cần một lệnh `kubectl delete` tường minh.

14.2 Các bộ phận chuyển động



Hình 14.1: Mọi thứ chỉ nói chuyện với API server.

API server cửa trước duy nhất; `kubectl` là một REST client của nó. Xác thực, phân quyền (RBAC của Chương 22) và kiểm tra hợp lệ diễn ra ở đây.

Kho trạng thái (state store) cơ sở dữ liệu trạng thái mong muốn (`etcd` trong cụm chuẩn; `k3s` nhúng `sqlite` khi chỉ có một node).

Bộ lập lịch (scheduler) gán pod chưa được lập lịch vào node, tôn trọng *requests* tài nguyên của Chương 17.

Controller manager các vòng lặp điều hòa: Deployment → ReplicaSet → pod, endpoints, và hàng chục vòng khác.

kubelet agent trên node: theo dõi các pod được gán cho node mình, điều khiển `containerd` để chạy chúng, thực thi probe, báo cáo trạng thái.

`k3s` đóng gói toàn bộ những thứ này vào một binary — một Kubernetes được CNCF chứng nhận, trừ đi chi phí của một VM, cộng thêm pin sẵn: Traefik làm Ingress controller, lưu trữ local-path, `containerd`. Mọi manifest trong sách này áp dụng nguyên xi lên EKS hay OpenShift; chính chứng nhận đó là lý do học trên `k3s` chuyển giao được.

14.3 Object, label, namespace

Mọi object là một tài liệu YAML với cùng bộ khung — `apiVersion`, `kind`, `metadata` (tên, namespace, label), `spec` (mong muốn), `status` (quan sát được, do controller ghi). Hai cơ chế tổ chức:

Namespace phân vùng cụm; dự án này đặt mọi thứ trong `ts`, giữ nó tách biệt với `kube-system` và cho phép giới hạn phạm vi RBAC — quyền của Jenkins dừng lại ở ranh giới namespace (Chương 22).

Label cặp khóa/giá trị tùy ý; *selector* truy vấn chúng. Mọi mối nối trong Phần V đều là một label selector: Service tìm pod theo label, Deployment sở hữu pod theo label. Kustomization đóng dấu `app.kubernetes.io/part-of: teacher-supporter` lên mọi thứ để một selector có thể liệt kê — hoặc xóa — cả ứng dụng.

14.4 Đưa kubectl tới cụm

`kubectl` đọc `~/.kube/config`: URL của cụm, chứng chỉ CA và một thông tin đăng nhập. Từ Windows, dự án này với tới `https://100.85.66.43:6443` qua Tailscale — điều đòi hỏi cài `k3s` với `--tls-san 100.85.66.43`, vì chứng chỉ của API server chỉ hợp lệ cho những tên được nướng vào lúc cài đặt.

THỰC HÀNH — CẢM NHẬN VÒNG LẶP

```
kubectl -n ts get pods
kubectl -n ts delete pod -l app=course-service
kubectl -n ts get pods -w
```

Xóa pod, rồi quan sát: một pod thay thế xuất hiện trong vài giây, được tạo không phải bởi lệnh của bạn mà bởi ReplicaSet controller nhận ra thiếu một replica. Bạn chưa hề ra lệnh khởi động lại — bạn làm trạng thái thực tế lệch khỏi mong muốn, và vòng lặp đã khép lại khoảng cách đó.

CẠM BÃY — CERTIFICATE IS VALID FOR... NOT 100.85.66.43

Triệu chứng: kubectl từ Windows thất bại khi kiểm tra TLS trong khi chạy tốt trên server. Nguyên nhân: chứng chỉ API server thiếu địa chỉ tailnet trong SAN — nó được cài mà không có `--tls-san`. Sửa lúc cài đặt chỉ là một cớ; sửa sau đó nghĩa là xóa thư mục TLS của server và khởi động lại k3s. Một trường hợp cụ thể của quy tắc chung: chứng chỉ gắn với *tên*, nên hãy quyết định mọi tên mà server sẽ được gọi *trước* khi cấp.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Giải thích “khai báo”: bạn viết gì, ai hành động trên nó, và vì sao apply cùng một manifest hai lần không thay đổi gì?
2. Vì sao Kubernetes cố ý không có `depends_on`, và điều gì thay cho thứ tự khởi động trong lúc hội tụ?
3. Lần theo `kubectl -n ts delete pod ...` qua các thành phần: ai nhận, ai nhận ra, ai hành động, trên máy nào?
4. Vì sao gỡ Ingress cũ khỏi git không gỡ nó khỏi cụm, và hai lệnh nào đều có thể gỡ được nó?

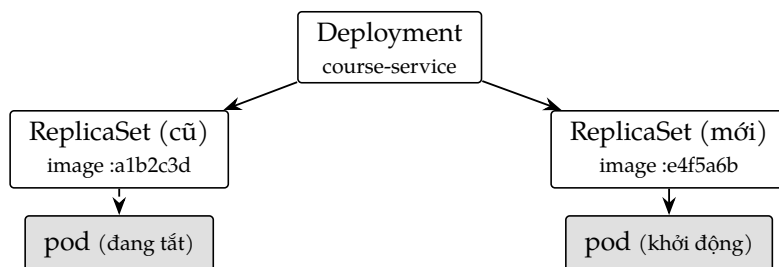
Workload: Pod, Deployment, StatefulSet

15.1 Pod

Pod là đơn vị mà Kubernetes lập lịch: một hoặc nhiều container dùng chung network namespace (một IP, localhost giữa chúng) và volume. Hầu như mọi pod ở đây chỉ có một container; các mẫu nhiều container (sidecar làm proxy hay đẩy log) mới quan trọng khi bạn gặp service mesh. Pod *sinh ra để chết* — chúng không bao giờ được chữa, chỉ được thay, và IP của chúng chết theo. Mọi thứ còn lại trong Phần V — controller, Service, PVC — tồn tại để xây hành vi bền vững từ những pod dùng xong bỏ.

15.2 Deployment và rollout

Bạn không bao giờ tạo pod trần. Một *Deployment* khai báo “N replica của mẫu pod này”; nó quản lý một *ReplicaSet* (bộ đếm), và *ReplicaSet* quản lý các pod. Ba tầng gián tiếp là thứ làm cho việc cập nhật trở nên thanh lịch: đổi mẫu (một tag image mới), *Deployment* tạo một *ReplicaSet mới*, tăng dần nó lên trong khi cái cũ giảm xuống:



Hình 15.1: Một rolling update đang giữa chừng: hai *ReplicaSet*, lưu lượng đang dịch chuyển.

Rollout chỉ tiến tới khi pod mới trở nên *Ready* (probe của Chương 17 — đây là nơi chúng chứng tỏ giá trị), nên một image hỏng làm rollout dừng lại thay vì kéo sập service; và `kubectl rollout undo` trở ngược về *ReplicaSet* trước, thứ được giữ lại chính là vì mục đích này. `kubectl rollout status` của pipeline (Chương 23) là CI đang chờ bộ máy này kết thúc.

Với một replica — cụm này — rolling update vẫn nghĩa là khởi-động-mới-rồi-tắt-cũ, một khoảng chổng lẩn hai pod ngăn ngửi tổn một thoáng gấp đôi bộ nhớ. `maxUnavailable/maxSurge` tinh chỉnh điều nhảy đó ở quy mô lớn.

15.3 StatefulSet: khi danh tính quan trọng

Với cơ sở dữ liệu, tính hoán đổi được là hoàn toàn sai: pod nào lên cũng phải là *đúng Postgres đó với đúng dữ liệu đó*. StatefulSet cung cấp thứ mà Deployment từ chối: tên ổn định (`postgres-course-0`, không phải hậu tố ngẫu nhiên), PVC riêng cho từng replica qua `volumeClaimTemplates` (Chương 19) sống lâu hơn pod và gắn lại vào pod kế nhiệm, và thay thế có thứ tự, từng cái một. Từ `postgres-course.yaml`:

```
kind: StatefulSet
spec:
  serviceName: postgres-course
  replicas: 1
  volumeClaimTemplates:      # a PVC per replica, not per manifest
  - metadata: { name: data }
    spec:
      accessModes: [ReadWriteOnce]
      resources: { requests: { storage: 5Gi } }
```

Quy tắc quyết định: *tiến trình* có giữ trạng thái mà tiến trình kế tiếp phải thừa kế không? Cơ sở dữ liệu thì có; Spring service thì không — trạng thái của chúng nằm trong các cơ sở dữ liệu kia, điều làm chúng thành Deployment, và điều khiến chúng mở rộng và rollout được. (Tính phi trạng thái của JWT ở Chương 5 chính là thuộc tính này ở tầng trên.)

CẠM BÃY — MẤT DỮ LIỆU KIỂU CƠ-SỞ-DỮ-LIỆU-TRONG-DEPLOYMENT

Triệu chứng: ai đó chạy cơ sở dữ liệu như một Deployment với volume thường; chạy tốt hàng tháng; rồi một sự cố node tạo ra replica *thứ hai* trở vào cùng bộ file, và cơ sở dữ liệu hỏng. Nguyên nhân: hợp đồng của Deployment là “các replica hoán đổi được” và rolling update của nó chạy cũ và mới *đồng thời* trong chốc lát — chí mạng khi cả hai cùng mở một thư mục dữ liệu. Thứ tự của StatefulSet + claim riêng từng replica tồn tại để điều này bất khả. Bài học khái quát: chọn controller theo *hợp đồng* của nó, không phải theo file YAML nào bạn thấy trước.

15.4 Các loại workload khác

Cho đầy đủ, các controller còn lại, chưa cái nào được dùng ở đây: **DaemonSet** (một pod mỗi node — bộ gom log, metric của node), **Job** (chạy tới khi xong — Flyway-như-bước-CI của Chương 7 sẽ là một Job), **CronJob** (Job theo lịch — sao lưu, mảnh ghép còn thiếu của Chương 19).

THỰC HÀNH — XEM MỘT ROLLOUT DIỄN RA

```
kubectl -n ts get rs -l app=course-service          # one ReplicaSet
kubectl -n ts set image deployment/course-service \
  course-service=localhost:5000/course-service:1.0.0-SNAPSHOT
kubectl -n ts rollout status deployment/course-service
kubectl -n ts get rs -l app=course-service          # two -- old kept at 0
kubectl -n ts rollout undo deployment/course-service
```

Lệnh `get rs` thứ hai là điểm mấu chốt: ReplicaSet cũ vẫn tồn tại, scale về không — đích của `undo`, mặc định được giữ cho mười bản sửa đổi gần nhất.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Vì sao Deployment quản lý ReplicaSet thay vì quản lý pod trực tiếp? Tính năng nào rơi ra từ tầng gián tiếp đó?
2. Phát biểu khác biệt về hợp đồng giữa Deployment và StatefulSet, mỗi cái một câu, rồi phân loại: Eureka server, một cache Redis, một Kafka broker, api-gateway.
3. Trong rolling update, điều kiện nào quyết định việc tắt pod cũ, và bộ máy của chương nào cung cấp điều kiện đó?
4. Giải thích chính xác cơ chế khiến chạy Postgres dưới Deployment rước lấy hỏng dữ liệu trong khi StatefulSet thì không.

Cấu hình và Secret trong cụm

16.1 Hai cơ chế tiêm cấu hình

Phần I đã xác lập rằng biến môi trường thắng mọi file. Kubernetes cung cấp hai cách để đặt chúng, và các manifest dùng cả hai một cách có chủ đích:

```
env:                                     # literal, visible in git
- { name: SPRING_DATASOURCE_URL,
    value: "jdbc:postgresql://postgres-course:5432/ts_course" }
envFrom:                                # every key of a Secret
- secretRef: { name: postgres-course-app }
- secretRef: { name: jwt }
```

Chính sự tách bạch này là ranh giới bảo mật: topology (hostname, cổng, feature flag) không phải bí mật và thuộc về git, nơi có thể review; thông tin đăng nhập (credential) không bao giờ xuất hiện trong git. Khi đọc một Deployment ở đây, mọi mục `env:` là cấu hình công khai và mọi dòng `envFrom:` là con trỏ tới thứ được tạo ngoài luồng.

16.2 Secret: là gì và không là gì

Secret là một object thuộc namespace, chứa các cặp key/value được encode base64. Hai điều thắng thắn về nó:

- Base64 là *encoding*, không phải mã hóa. `echo cm9vdA== | base64 -d` lộ ngay mật khẩu. Thứ bảo vệ nó là kiểm soát truy cập (RBAC quyết định ai được đọc Secret — để ý Chương 22 không cấp cho Jenkins verb nào như vậy), vòng đời tách khỏi code, và tùy chọn mã hóa khi lưu (encryption at rest).
- Nó vẫn là chỗ chứa đúng: mọi cơ chế cao hơn nó (Sealed Secrets, Vault, secret manager của cloud) cuối cùng đều vật chất hóa thành một Secret để pod tiêu thụ. Học object thuần trước là học chính giao diện.

16.3 `create-secrets.sh`, có chú giải

Secret ở đây được tạo bằng một script lười đăng, không phải manifest:

```
kubectl -n "$NS" create secret generic postgres-course \
  --from-literal=POSTGRES_USER="$PG_USER" \
  --from-literal=POSTGRES_PASSWORD="$PG_PASSWORD" \
  --from-literal=POSTGRES_DB=ts_course \
  --dry-run=client -o yaml | kubectl apply -f - ①

kubectl -n "$NS" create secret generic postgres-course-app \
  --from-literal=SPRING_DATASOURCE_USERNAME="$PG_USER" \
  --from-literal=SPRING_DATASOURCE_PASSWORD="$PG_PASSWORD" \
  --dry-run=client -o yaml | kubectl apply -f - ②

kubectl -n "$NS" create secret generic jwt \
  --from-literal=JWT_SECRET="$JWT_SECRET" ③
```

- ① Thành ngữ tạo-lũy-đăng: create đơn thuần sẽ thất bại nếu Secret đã tồn tại; render bằng `--dry-run` rồi pipe sang `apply` khiến script chạy lại an toàn — tạo mới hay cập nhật, đăng nào cũng hội tụ. Một bài học nhỏ về khai báo-hơn-mệnh-lệnh, dưới dạng shell.
- ② Cùng một credential viết hai lần, theo cách mỗi bên tiêu thụ muốn: image Postgres cần `POSTGRES_USER`; Spring cần `SPRING_DATASOURCE_USERNAME`. Hai Secret nghĩa là `envFrom` của mỗi pod chỉ nhập những tên nó hiểu — không pod nào phải ôm một mớ biến lạ.
- ③ Một Secret được hai Deployment tiêu thụ — `auth-service` dùng nó để ký JWT, `api-gateway` dùng để xác minh. Hợp đồng khóa-chia-sẻ của Chương 5, thể hiện dưới dạng hạ tầng: xoay vòng (rotate) khóa là một lần chạy script cộng hai lần khởi động lại pod, và hai bên tiêu thụ *không thể* lệch nhau.

QUYẾT ĐỊNH — SCRIPT, KHÔNG PHẢI MANIFEST — VÀ KHÔNG NẪM TRONG KUSTOMIZATION

Manifest Secret trong git sẽ đặt credential đã base64 chỉ cách bất kỳ ai có quyền truy cập repo một lệnh `base64 -d`, mãi mãi, trong lịch sử. Script giữ giá trị trong môi trường của người vận hành (`JWT_SECRET=... ./create-secrets.sh`), giá trị mặc định khớp với dev nên hành vi giống hệt, và git chỉ giữ *hình dạng*. Comment trong `kustomization` nói cùng một điều từ phía bên kia: secret cố tình vắng mặt trong `resources`:

16.4 ConfigMap

Người anh em không-bí-mật của Secret — cùng cơ chế, không giả vờ nhạy cảm, cộng thêm một tài năng mà Secret cũng có nhưng ConfigMap dùng nhiều hơn:

mount thành *file*. Dự án này chưa có ConfigMap nào, vì một lý do thẳng thắn: cấu hình không-bí-mật của nó đã chảy qua config-server và biến môi trường. ConfigMap đầu tiên sẽ đến một cách tự nhiên cùng việc cho config-server nghỉ hưu ở Chương 27 — các file configurations/*.yaml được mount vào pod, để Spring đọc trực tiếp và xóa hẳn một service.

CẠM BẦY — ĐỔI SECRET RỒI, CHẴNG CÓ GÌ XẢY RA

Triệu chứng: bạn chạy lại create-secrets.sh với mật khẩu mới; các service vẫn dùng mật khẩu cũ, cho tới khi vài ngày sau một pod nào đó khởi động lại và bắt đầu lỗi xác thực — một sự cố hên giờ. Nguyên nhân: biến môi trường được *tiêm lúc container khởi động*; pod đang chạy không bao giờ thấy Secret cập nhật. Vì vậy xoay vòng theo thiết kế là hai bước: cập nhật Secret, rồi `kubectl -n ts rollout restart deployment ...` cho mọi bên tiêu thụ, ngay lập tức — trong lúc credential cũ và mới đều còn dùng được, nếu hệ thống phía sau cho phép.

16.5 Ngoài phạm vi dự án

Khoảng cách giữa thiết lập này và thực hành công nghiệp là một khái niệm: làm cho secret *giao được qua pipeline* mà không đọc được trong git. Ba nấc trên chiếc thang đó: **Sealed Secrets** (mã hóa bằng khóa công khai của cụm; commit khối đã niêm phong; một controller giải mã trong cụm — tương thích GitOps với một bộ phận chuyển động); **External Secrets Operator** (một manifest Secret là *con trỏ* vào Vault/AWS Secrets Manager, được controller đồng bộ); và **Vault injection** (secret không bao giờ trở thành object Kubernetes). Mỗi cách đều thay thế script; không cách nào thay đổi cách pod tiêu thụ — `envFrom` sống sót qua mọi cuộc di cư, và đó là lý do các manifest đã sẵn sàng cho tương lai.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Base64 không phải mã hóa — vậy hãy kể ba thứ thực sự bảo vệ một Secret, và dự án này đứng ở đâu với từng thứ.
2. Vì sao hai Secret cho một credential Postgres? Việc tách này ngăn được lỗi gì?
3. Mô tả trọn vẹn việc xoay vòng JWT secret trên cụm này: lệnh, thứ tự, và kiểu lỗi khi chỉ làm một nửa.
4. Mục cấu hình nào trong dự án sẽ chuyển sang ConfigMap trước tiên, và điều đó cho service nào nghỉ hưu?

Probe sức khỏe và quản lý tài nguyên

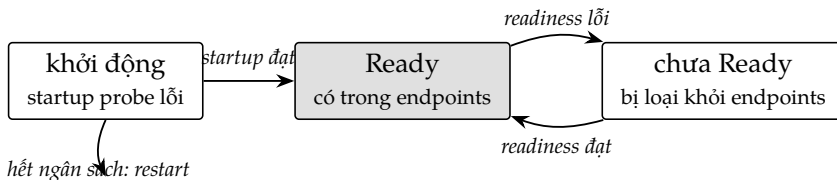
Chương này là trái tim vận hành của Phần V: kubelet quyết định một pod còn sống và sẵn sàng như thế nào, và bộ lập lịch cùng kernel chia hai nhân CPU cho một tá JVM ra sao. Gần như mỗi dòng ở đây đều học được từ việc nhìn một pod thật cứ xử sai.

17.1 Ba loại probe

startupProbe “vẫn đang boot — kiên nhẫn.” Khi nó còn thất bại, các probe khác bị tạm hoãn. Cạn ngân sách thì container bị khởi động lại.

readinessProbe “có thể nhận traffic *ngay bây giờ* không?” Thất bại thì pod bị rút khỏi endpoints của Service — không restart, không kích tính, chỉ là không nhận request mới cho tới khi qua lại.

livenessProbe “tiến trình đã hết cứu được chưa?” Thất bại thì container bị *khởi động lại*. Cái nguy hiểm.



Hình 17.1: Vòng đời của pod dưới con mắt kubelet.

Từ `course-service.yaml`:

```
startupProbe:
  httpGet: { path: /v3/api-docs, port: http } ①
  periodSeconds: 10
  failureThreshold: 12 ②
readinessProbe:
  httpGet: { path: /v3/api-docs, port: http }
  periodSeconds: 5
  failureThreshold: 3
# no livenessProbe ③
```

- ① Quyết định về đường dẫn probe. `/actuator/health` là endpoint sinh ra cho việc này — nhưng `course-service` và `auth-service` không cho nó qua Spring Security, nên nó trả 401 và kubelet tính là thất bại. Đường dẫn không cần xác thực duy nhất chỉ trả 200 *sau khi toàn bộ context đã lên* — Flyway đã migrate, bean đã dựng — là `/v3/api-docs` của `springdoc`. `Dictionary-service` và `gateway` có cho phép `actuator` và probe nó đúng cách; còn `notification-service` không có Spring Security nào cả, nên endpoint `health` của nó mặc định mở — ba service, ba câu trả lời cho cùng một câu hỏi. Quy tắc: đường dẫn probe phải không cần xác thực *và* là tín hiệu readiness thật; một kiểm tra `tcpSocket` qua ngay khi Tomcat bind cổng thì chẳng đạt điều nào.
- ② $12 \times 10\text{ s}$ = hai phút ân hạn khởi động — đo cho một JVM khởi động nguội trên hai nhân bị tranh chấp.
- ③ Cổ tình vắng mặt. Liveness giết; thứ duy nhất nó bắt được thêm so với readiness là một JVM treo cứng thật sự — hiếm — trong khi một đợt dừng GC dài trên máy 2 nhân đang tải nặng có thể làm probe thất bại và giết một pod *khỏe mạnh*. Hãy thêm liveness khi bạn đã quan sát được cái treo mà nó sinh ra để bắt.

CẠM BẦY — STARTUP PROBE FAILED: STATUSCODE 404 — HOẶC 403

Triệu chứng (milestone 6, build #6): log của `notification-service` cho thấy Kafka đã gán partition và ứng dụng khởi động hoàn toàn, vậy mà pod không bao giờ Ready; probe báo 404. `dictionary-service` cũng vậy, nhưng 403. Nguyên nhân: các file config-server `expose health, info, metrics, prometheus` cho mọi service — nhưng chỉ config-server, discovery-server và api-gateway có `spring-boot-starter-actuator` trên classpath. Cấu hình `expose` cho một thư viện không tồn tại thì lặng lẽ vô hiệu; URL đơn giản là 404. Biến thể 403 là đóng góp của Spring Security: 404 forward sang `/error`, `/error` không nằm trong danh sách `permitAll`, và một chuỗi filter `stateless` không có entry point thì trả 403 — nên triệu chứng chỉ về phân quyền trong khi nguyên nhân là thiếu jar. Sửa: thêm starter vào cả bốn POM của service. Bài học: đường dẫn probe phải được kiểm chứng bằng `curl` từ trong pod *trước khi* tin nó trong manifest — khẳng định trước đó của sách rằng `dictionary-service` “probe health đúng cách” được viết từ `SecurityConfig` của nó, không phải từ một request thật.

CẠM BẦY — VÒNG LẶP RESTART TỰ SINH RA CHÍNH NÓ

Triệu chứng: một service khởi động lại vài phút một lần khi tải nặng, mỗi lần restart làm tải nặng thêm. Nguyên nhân: một liveness probe quá gắt bị timeout trong lúc GC dừng hoặc gọi downstream chậm — probe *gây ra* chính

sự cố nó sinh ra để phát hiện. Kiểu lỗi này là lý do “không có liveness probe” là mặc định có thể biện hộ, và là lý do liveness, khi được thêm, chỉ nên kiểm tra “event loop có còn phản hồi không,” không bao giờ kiểm tra phụ thuộc downstream.

17.2 Requests, limits và JVM

```
resources:
  requests: { memory: 512Mi, cpu: 200m } ①
  limits: { memory: 768Mi } ②
env:
  - { name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0" } ③
```

- ① *Requests* là sổ sách của bộ lập lịch: pod chỉ được đặt vào nơi còn 512 Mi và 0,2 CPU chưa ai nhận. Requests *giữ chỗ*, không giới hạn.
- ② *Limits* là trần do kernel thực thi. Vượt limit bộ nhớ là tiến trình bị OOM-kill — trạng thái `OOMKilled`, exit code 137, không Java exception, không dòng log. Để ý sự bất đối xứng: có limit bộ nhớ, nhưng *không có limit CPU* — xem hộp quyết định tiếp theo.
- ③ JVM nhận biết container định cỡ heap theo tỷ lệ của limit bộ nhớ *của container*: 75% của 768 Mi cho heap, phần còn lại cho metaspace, thread và off-heap. Không có dòng này, giá trị mặc định để lại hoặc lãng phí, hoặc biên an toàn OOM-kill bằng không.

QUYẾT ĐỊNH — VÌ SAO KHÔNG CÓ LIMIT CPU

Limit CPU bóp nghẹt (throttle): container vượt hạn mức bị tạm dừng tới chu kỳ lập lịch tiếp theo — và JVM đang khởi động chính là lúc service muốn từng chu kỳ CPU (nạp class, JIT). Khởi động bị bóp biến 60 giây thành 200 và làm startup probe thất bại: chính limit chế tạo ra thất bại. Khác bộ nhớ, CPU là tài nguyên nén được — tranh chấp làm mọi thứ chậm lại thay vì giết — nên mẫu dùng ở đây là lời khuyên chuẩn: **luôn request CPU, hiếm khi limit**. Bộ nhớ có limit vì kiểu lỗi của bộ nhớ không giới hạn là kernel giết *một ai đó*, và bạn muốn ranh giới ấy đoán trước được.

TRÊN PHẦN CỨNG NÀY

Tổng requests của mọi service xấp xỉ phần còn lại của 19 GB sau khi trừ Jenkins, k3s và các datastore — một cách có chủ đích. Requests là lời hứa với bộ lập lịch; hứa quá tay trên một node duy nhất nghĩa là pod không bao giờ được lập lịch (Pending mãi mãi), hứa thiếu nghĩa là chơi roulette OOM. Trên một node, bạn đang hoạch định dung lượng bằng tay; autoscaler trên cloud biến cùng phép tính đó thành định cỡ node pool.

THỰC HÀNH — XEM MỘT LẦN KHỞI ĐỘNG QUA CON MẮT CỦA PROBE

```
kubectl -n ts delete pod -l app=course-service
kubectl -n ts get pods -w
# Running 0/1 <- startup probe still failing; no traffic yet
# Running 1/1 <- /v3/api-docs answered; endpoints updated
kubectl -n ts describe pod -l app=course-service | tail -n 12
```

Khoảng cách giữa 0/1 và 1/1 là Flyway cộng khởi động context — cửa sổ mà, nếu không có probe, request sẽ đập vào một service mới lên được một nửa.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Với từng probe: thất bại thì chuyện gì xảy ra, và probe nào an toàn để đặt gắt?
2. Vì sao /v3/api-docs là tín hiệu readiness trung thực cho course-service còn tcpSocket: 8082 là lời nói dối? Điều gì khiến /actuator/health không dùng được ở đó, mà dùng được trên dictionary-service?
3. Một pod hiện OOMKilled, exit 137, log sạch. Giải thích cơ chế và vì sao không có Java exception nào. Hai thứ đầu tiên cần kiểm tra?
4. Requests và limits: bộ lập lịch đọc cái nào, kernel thực thi cái nào, và vì sao dự án này giới hạn bộ nhớ mà không giới hạn CPU?

Mạng: Service, DNS, Ingress

18.1 Vấn đề mà pod tạo ra

Pod có IP định tuyến được trong cụm — và mất nó mỗi lần bị thay thế. Chương 15 đã biến pod thành thứ dùng xong bỏ; phải có gì đó ổn định đứng phía trước. Thứ đó là *Service*: một IP ảo và tên DNS cố định, phía sau là danh sách sống các endpoint pod được chọn theo label:

```
apiVersion: v1
kind: Service
metadata: { name: course-service }
spec:
  selector: { app: course-service }    ①
  ports:
    - { port: 8082, targetPort: 8082 } ②
```

① Endpoint controller liên tục liệt kê các pod *Ready* khớp selector này — readiness (Chương 17) là bài kiểm tra tư cách thành viên. ② Traffic tới port của Service được chuyển tiếp tới targetPort của một endpoint được chọn.

DNS của cụm cho mỗi Service một cái tên: `course-service` trong namespace, `course-service.ts.svc.cluster.local` từ bất cứ đâu. Đây là lý do các override biến môi trường trên Kubernetes trông giống hệt bên Compose — cả hai nền tảng đều phân giải tên service trên một mạng riêng; dự án cố tình đặt tên Service k8s theo tên service Compose để các override khớp nhau.

Kiểu Service mặc định, *ClusterIP*, chỉ với tới được từ bên trong cụm — một thuộc tính bảo mật mà Phần II đã dựa vào cho Mongo không mật khẩu. *NodePort* và *LoadBalancer* tồn tại để expose Service trực tiếp; dự án này thay vào đó dồn mọi thứ qua một Ingress.

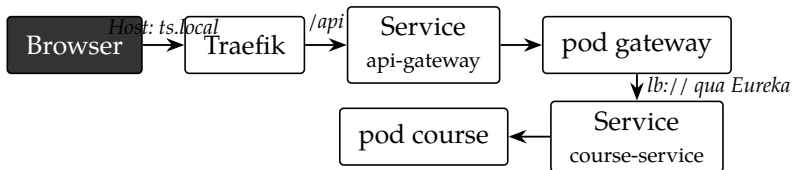
18.2 Ingress và Traefik

Một Service expose một thứ; *Ingress* là cửa trước ở tầng L7: định tuyến HTTP theo host và path, sau này là kết thúc TLS, được khai báo dưới dạng dữ liệu và thực thi bởi một *Ingress controller* — trong k3s là Traefik đi kèm, theo dõi các object Ingress. Từ `api-gateway.yaml`:

```
kind: Ingress
spec:
  rules:
    - host: ts.local                                ①
      http:
        paths:
          - path: /api                              ②
            pathType: Prefix
            backend:
              service:
                name: api-gateway
                port: { number: 8080 }
# /oauth2 and /login/oauth2 likewise -> api-gateway
```

- ① Quy tắc theo host: áp dụng cho request có header Host là ts.local — một cái tên chỉ tồn tại trong file hosts của Windows, trỏ tới IP tailnet của server. Nhiều host có thể dùng chung cặp cổng 80/443 duy nhất của cụm.
- ② Tỏa theo path. Chỉ ba prefix được vào, tất cả tới gateway; / để trống cho Ingress riêng của frontend (Chương 26), mà Traefik gộp vào bảng định tuyến của cùng host.

18.3 Đường đi trọn vẹn của một request



Hình 18.1: Hai bộ định tuyến, hai hệ thống khám phá, một request.

Đáng dừng lại một chút: tra cứu Eureka của gateway trả về IP pod (prefer-ip-address của Chương 3), nên chẳng thứ tư về kỹ thuật đi vòng qua Service course-service. Hai hệ thống khám phá chạy song song — Eureka cho gateway-tới-service, DNS của Service cho mọi thứ còn lại (config-server, cơ sở dữ liệu, Kafka). Bước đơn giản hóa ở Chương 27 gộp chúng thành một.

CẠM BẦY — SERVICE KHÔNG CHỌN ĐƯỢC GÌ

Triệu chứng: curl tới tên Service bị timeout; pod đang Running và khỏe mạnh. Nguyên nhân #1: lệch label — selector của Service và label trong pod template khác nhau bởi một lỗi gõ; danh sách endpoint rỗng và không ai kêu

ca. Kiểm tra: `kubectl -n ts get endpoints course-service` — cột ENDPOINTS trống chính là lỗi này. Nguyên nhân #2: endpoint có nhưng pod chưa Ready — cùng lệnh đó hiện chúng dưới `NotReadyAddresses`. Lớp trừu tượng Service thất bại *trong im lặng*; object endpoints là nơi nó lộ ra.

QUYẾT ĐỊNH — MỘT HOST INGRESS + FILE HOSTS, KHÔNG DÙNG NODEPORT

NodePort (mỗi service một cổng lạ lẫm) sẽ chạy được qua Tailscale mà không cần trò DNS nào — và tạo ra URL kiểu `100.85.66.43:30082` vừa lộ topology, vừa né gateway, vừa không làm TLS tử tế được. Một host Ingress nghĩa là một địa chỉ cần bảo vệ và trải nghiệm trình duyệt giống hết production; dòng `ts.local` trong file hosts là chi phí duy nhất phía client. Đường nâng cấp — một domain thật cộng cert-manager, hoặc Cloudflare Tunnel — chỉ đổi host của Ingress và không gì khác.

18.4 Ngoài phạm vi dự án

Trên EKS, cùng object Ingress đó, thêm annotation cho controller của AWS, sẽ cấp phát một ALB thật; trên OpenShift nó trở thành Route — cùng ý tưởng, khác danh từ. Mảnh ghép cụm này bỏ qua: **NetworkPolicy** — quy tắc tường lửa cho traffic pod-tới-pod (“chỉ gateway được chạm tới `course-service:8082`”), tức là sự thực thi trong cụm mà mẫu biên-tin-cây của Chương 5 gọi là điểm mềm còn lại của nó. Tin cây mạng phẳng là bình thường với homelab và là cẩu thả với một ngân hàng.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. IP pod đổi mỗi lần thay thế, vậy mà `course-service:8082` luôn chạy. Kể tên từng cơ chế nằm giữa cái tên và socket của pod.
2. Readiness tương tác với endpoints của Service ra sao, và điều đó nghĩa là gì trong một rolling update?
3. Chẩn đoán: Service tồn tại, pod Ready, `get endpoints` hiện danh sách rỗng. Sai ở đâu, và vì sao không có gì báo lỗi?
4. Vì sao `ts.local` cần một dòng trong file hosts, và thứ gì thay thế dòng đó trong hai đường nâng cấp lên production?

Lưu trữ: Volume, Claim và trạng thái trên một node

19.1 Chồng trừu tượng

Hệ thống file của container chết cùng nó. Kubernetes tách *yêu cầu* lưu trữ khỏi việc *cung cấp* nó qua ba object:

PersistentVolume (PV) một mẫu lưu trữ thật — một thư mục, một volume EBS, một export NFS.

PersistentVolumeClaim (PVC) yêu cầu của workload: “5 Gi, ReadWriteOnce.” Claim là thứ pod mount; nó gắn (bind) vào một PV.

StorageClass nhà máy: cách tạo PV theo nhu cầu khi một claim xuất hiện (*cấp phát động*).

Sự gián tiếp này là tính khả chuyển: `postgres-course.yaml` chỉ nói “tôi cần 5 Gi”; còn *làm thế nào* nó thành đĩa là việc của cụm. Cùng manifest đó nhận một thư mục cục bộ trên k3s và một volume EBS trên EKS.

19.2 local-path trên k3s

k3s đi kèm StorageClass `local-path`: một PVC trở thành một thư mục dưới `/var/lib/rancher/k3s/storage/` trên SSD của node. Đơn giản, nhanh, và thẳng thắn về giới hạn — dữ liệu nằm trên *đĩa của node đó*. Trên cụm nhiều node, điều đó ghim pod vào node; trên cụm một node này, nó chỉ có nghĩa đúng như nó nói.

TRÊN PHẦN CỨNG NÀY

`local-path` cung cấp *bền vững* (persistence: dữ liệu sống sót qua khởi động lại pod và node), không phải *bền chắc* (durability: một SSD hỏng là mất tất cả) và không phải *sao lưu* (một lệnh `DELETE` sai lầm không có bản sao nào để quay về). Mảnh còn thiếu, nói thẳng, là một CronJob chạy `pg_dump` lên MinIO hoặc ra ngoài máy. Quy tắc ngón tay cái đáng ghi nhớ: cơ sở dữ liệu không có khôi phục đã kiểm chứng chỉ là một cái cache.

19.3 Hai kiểu claim trong manifest

MinIO (một Deployment với một replica) dùng PVC độc lập; các cơ sở dữ liệu dùng volumeClaimTemplates bên trong StatefulSet — một PVC *đúc riêng cho từng replica* và, quan trọng nhất, *gắn lại vào pod thay thế* chứ không phải vào một pod sống thứ hai. Sự phân biệt đó là nửa lưu trữ của cạm bẫy hồng dữ liệu ở Chương 15.

```
volumeClaimTemplates:
- metadata: { name: data }
  spec:
    accessModes: [ReadWriteOnce] ①
    resources: { requests: { storage: 5Gi } }
# in the pod template:
volumeMounts:
- { name: data, mountPath: /var/lib/postgresql/data }
env:
- { name: PGDATA, value: /var/lib/postgresql/data/pgdata } ②
```

- ① ReadWriteOnce = mount đọc-ghi bởi một *node* tại một thời điểm — vừa khít cho cơ sở dữ liệu. (RWX — nhiều node — cần lưu trữ kiểu NFS và là chế độ người ta muốn cho upload dùng chung, trước khi phát hiện lưu trữ đối tượng, Chương 9, là câu trả lời tốt hơn.)
- ② Mẹo PGDATA, một kinh điển thực thụ: image Postgres từ chối khởi tạo vào thư mục không rỗng, mà volume ext4 sinh ra đã có lost+found. Trỏ PGDATA sâu thêm một thư mục là né được. Bạn sẽ gặp đúng mẫu này trong hàng năm trời StackOverflow về Postgres-trên-Kubernetes.

CẠM BÃY — XÓA STATEFULSET, DỮ LIỆU VẪN CÒN — HOẶC MẤT

Hai bất ngờ trong một. Thứ nhất: xóa StatefulSet *không* xóa PVC của nó — theo thiết kế, dữ liệu sống lâu hơn sai lầm điều phối; tạo lại StatefulSet là nó gắn lại. Thứ hai, bẫy ngược: xóa PVC khi đang thử nghiệm *có* xóa PV và thư mục của nó (reclaimPolicy Delete, mặc định của cấp phát động). Sự bất đối xứng này đáng thuộc lòng trước khi bạn dọn một namespace: object workload là thứ dùng xong bỏ; claim mới là nơi dữ liệu nằm.

THỰC HÀNH — NHÌN XUYÊN QUA LỚP TRỮ TƯỢNG

```
kubectl -n ts get pvc           # claims and their PVs
kubectl get pv                  # cluster-scoped, note the paths
ssh ts-server 'sudo ls /var/lib/rancher/k3s/storage/'
```

```
# pvc-... one directory per PV -- your databases, as plain files
```

Giải ảo chính là mục đích: một PVC trên cụm này là một thư mục. Trên EKS, cùng hai lệnh kubectl đó sẽ hiện ID volume EBS — cùng object, khác nhà máy.

19.4 Ngoài phạm vi dự án

Trên Kubernetes có quản lý, StorageClass ánh xạ sang đĩa cloud (EBS/gp3 trên EKS, Persistent Disk trên GKE) với snapshot, mã hóa và đổi cỡ — object VolumeSnapshot đưa sao lưu vào cùng mô hình khai báo. Nhưng mẫu công nghiệp mà chương này hướng tới lại là phép trừ: các đội ngày càng giữ *cơ sở dữ liệu ngoài cụm* (RDS, Cloud SQL) và để Kubernetes chỉ chạy những thứ phi trạng thái — đổi sự chậm chạp StatefulSet ở Chương 15 lấy một hóa đơn dịch vụ có quản lý. Giờ bạn đã thấy cả hai mặt của cuộc đánh đổi đó.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. PV, PVC, StorageClass: pod mount cái nào, cái nào thuộc phạm vi cụm, và sự gián tiếp mua được gì cho manifest của dự án trên một EKS tương lai?
2. Vì sao ReadWriteOnce là đủ cho Postgres, và loại workload nào đẩy người ta về phía RWX — thường là sai?
3. Giải thích cả hai chiều của sự bất đối xứng khi xóa: StatefulSet mất/PVC còn, PVC mất/dữ liệu mất.
4. Bền vững, bền chắc và sao lưu mỗi thứ nghĩa là gì, và local-path cung cấp thứ nào?

Kustomize: Manifest theo bộ

20.1 Vấn đề

Phần V sinh ra cả tá file YAML. Apply từng file một thì dễ quên mất một file; và ngay khi có môi trường thứ hai (“prod, nhưng 2 replica và secret thật”), copy-paste manifest cho từng môi trường bảo đảm sẽ trôi (drift) cấu hình. Hai công cụ thống trị mảng này; dự án dùng công cụ được tích hợp sẵn trong kubectl.

20.2 Kustomization, có chú giải

`kubectl apply -k deploy/k8s/base` đọc một file duy nhất:

```
# deploy/k8s/base/kustomization.yaml
namespace: ts                                ①
resources:                                  ②
- postgres-course.yaml
- postgres-auth.yaml
- mongodb.yaml
- minio.yaml
# kafka.yaml requires the Strimzi operator (one-time admin step:
# ../install-strimzi.sh) -- without its CRDs, apply fails on kind Kafka.
- kafka.yaml
- kafka-topics.yaml
- maildev.yaml
- config-server.yaml
- discovery-server.yaml
- course-service.yaml
- auth-service.yaml
- dictionary-service.yaml
- notification-service.yaml
- api-gateway.yaml
commonLabels:                               ③
  app.kubernetes.io/part-of: teacher-supporter
```

- ① Đóng dấu lên mọi resource — không manifest nào cần dòng `namespace:` riêng, và chuyển cả ứng dụng sang namespace khác chỉ là sửa một dòng.
- ② Danh sách chính thức. Hai sự vắng mặt có chủ ý dạy nhiều hơn những gì có mặt: `namespace.yaml` bị loại vì nó thuộc phạm vi cluster và Role theo names-

pace của Jenkins không thể apply nó (Chương 22); các Secret bị loại vì những lý do của Chương 16. Một đối tượng thứ ba cũng ở diện chỉ-admin thì thậm chí không phải là file: Strimzi operator và các CRD của nó, được cài một lần bằng `install-strimzi.sh` (Chương 25) — `kafka.yaml` có trong danh sách nhưng không thể apply chừng nào chúng chưa tồn tại. Thứ tự liệt kê chỉ mang tính tài liệu — API server nhận tất cả, còn quá trình reconcile lo phần thực tế.

- ③ Một label đóng lên mọi object, biến cả ứng dụng thành một selector: `kubectl get all -l app.kubernetes.io/part-of=teacher-supporter` — hoặc, trong một buổi diễn tập thảm họa, một lệnh `delete`.

20.3 Base và overlay

Tính năng thật sự của Kustomize xuất hiện cùng môi trường thứ hai. *Base* giữ sự thật; các *overlay* vá lên nó:

```
# deploy/k8s/overlays/prod/kustomization.yaml
resources:
- ../../base
patches:
- target: { kind: Deployment, name: course-service }
  patch: |-
    - op: replace
      path: /spec/replicas
      value: 2
images:
- name: localhost:5000/course-service
  newTag: v1.4.2
```

Không template, không biến — overlay là *phép gộp YAML thuần*. Mỗi lớp đều là YAML Kubernetes hợp lệ mà bạn đọc được, diff được, và đẩy qua `kubectl kustomize` để xem kết quả cuối cùng trước khi bất cứ thứ gì chạm vào cluster. Bộ cục thư mục đã sẵn dạng overlay (*base/*), nên ngày cần overlay *prod*, đó là một lệnh `mkdir`, không phải một cuộc di dời.

QUYẾT ĐỊNH — KUSTOMIZE, KHÔNG PHẢI HELM

Helm là câu trả lời còn lại: manifest dưới dạng template Go (`replicas: {{ .Values.replicaCount }}`) với file giá trị cho từng môi trường, đóng gói thành các *chart* có phiên bản, chia sẻ được. Template mạnh hơn hẳn — vòng lặp, điều kiện, một chart triển khai được ứng dụng của bất kỳ ai — và đó chính là điểm rõ: Helm tỏa sáng khi bạn *phân phối* phần mềm lên cluster của người

lạ; còn để triển khai ứng dụng của chính mình, template khiến mọi manifest không đọc nổi cho đến khi được render. Kustomize giữ YAML thuần đúng là YAML thuần, đi kèm trong kubectl, và phủ hết việc vá replica/image/label — toàn bộ nhu cầu ở đây. Bạn vẫn sẽ dùng Helm chart cho phần mềm bên thứ ba (Strimzi, kube-prometheus-stack); còn viết một chart thì có thể đợi đến khi bạn có người dùng.

CẠM BÃY — SPEC.SELECTOR: FIELD IS IMMUTABLE

Triệu chứng: kubectl apply -k trong pipeline thất bại ở một Deployment với lỗi này, dù cùng file đó apply bằng tay thì ổn. Nguyên nhân: commonLabels đóng label của nó không chỉ vào metadata mà vào cả *selector* của mọi Deployment — và selector là bất biến sau khi tạo. Một object được tạo lần đầu *bên ngoài* Kustomize (kubectl apply -f maildev.yaml lúc thiết lập cột mốc 6, selector {app: maildev}) sẽ không bao giờ được Kustomize cập nhật về sau (selector {app: maildev, part-of: teacher-supporter}). Cách sửa: xóa object tạo bằng tay và để pipeline tạo lại. Quy tắc: object do một kustomization quản lý thì *chỉ* apply thông qua nó — muốn thử một lần, hãy render bằng kubectl kustomize rồi pipe kết quả đó, không bao giờ dùng file thô. (labels: với includeSelectors: false của Kustomize hiện đại tránh được việc chèn vào selector — nhưng đổi sang lúc này sẽ phá mọi selector đã tạo theo cách cũ.)

CẠM BÃY — HÔM QUA CÒN CHẠY: ẢO GIÁC PRUNE

Triệu chứng: bạn xóa một manifest khỏi resources:, apply lại, và object vẫn đang chạy — hàng tháng trời, cho đến khi nó xung đột với thứ gì đó. Nguyên nhân: apply-cả-bộ vẫn là apply — quy tắc của Chương 14 rằng apply không bao giờ xóa vẫn đúng với Kustomize. Ingress bị gỡ ở Chương 4 đã cần một lệnh kubectl delete thủ công. Các cách sửa có hệ thống: kubectl apply -k --prune (vẫn bị chặn sau cờ, và có lý do chính đáng — một selector gắn nhầm label có thể xóa hàng loạt), hoặc các controller GitOps (Chương 27) mà toàn bộ công việc là làm git và cluster hội tụ, kể cả phần xóa.

THỰC HÀNH — RENDER TRƯỚC KHI TIN

```
kubectl kustomize deploy/k8s/base | less      # the final YAML, merged
kubectl kustomize deploy/k8s/base | grep -c 'kind:' # object count
kubectl apply -k deploy/k8s/base --dry-run=server # validate, no changes
```

kubectl kustomize thuần túy là biến đổi văn bản — đọc đầu ra của nó là cách

bạn xem xét một overlay thật sự thay đổi gì, và `--dry-run=server` là thứ gần nhất với một unit test cho manifest.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Một kustomization giải quyết hai vấn đề gì so với `kubectl apply -f` từng file, ngay cả khi không có overlay nào?
2. Vì sao namespace và các Secret cố ý vắng mặt trong `resources:`? (Hai lý do khác nhau.)
3. Helm so với Kustomize: nêu sân nhà của mỗi công cụ trong một câu, và kể tên phần mềm bên thứ ba trong tương lai của dự án sẽ đến dưới dạng chart.
4. Một đồng nghiệp gỡ `mongodb.yaml` khỏi danh sách để “cho nghỉ hưu” nó. Điều gì thật sự xảy ra, và hai quy trình đúng là gì?

Phần VI

CI/CD

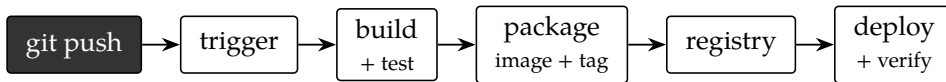
CI/CD: Các khái niệm

21.1 Hai thực hành, một pipeline

Continuous Integration mọi thay đổi được merge vào branch chung thường xuyên và được *kiểm chứng tự động* — biên dịch, test, đóng gói. Sản phẩm của CI là sự tự tin: branch chính luôn ở trạng thái được biết là tốt, và một hỏng hóc bị bắt vài phút sau commit gây ra nó, khi tác giả vẫn còn nhớ ngữ cảnh.

Continuous Delivery/Deployment mọi thay đổi đã kiểm chứng được tự động chuẩn bị để phát hành (Delivery) hoặc thật sự được phát hành (Deployment — không có cổng chặn bởi con người). Dự án này thực hành dạng mạnh hơn: một lần push lên `main` kết thúc bằng các pod đang chạy, không ai bấm nút.

Pipeline là cỗ máy cưỡng chế cả hai:



Hình 21.1: Hình dạng pipeline phổ quát. Tên công cụ thay đổi; các stage thì không.

Dù là Jenkins+Jib+k3s (dự án này), GitHub Actions+Docker+EKS, hay GitLab CI+Kaniko+OpenShift — mã nguồn đi vào, artifact đã test đi vào registry, bộ điều phối được bảo hộ tự về nó, rồi kiểm chứng rằng nó đã làm vậy. Học hình dạng này một lần và pipeline của mọi nhà tuyển dụng chỉ còn là bài tập đổi tên.

21.2 Từ vựng

Trigger thứ khởi động một lần chạy. *Webhook*: máy chủ git gọi CI server khi có push — tức thì, nhưng đòi hỏi CI server phải truy cập được từ internet. *Polling*: CI hỏi repo “có gì mới không?” theo lịch — không cần ai truy cập được, đổi lại là độ trễ. Dự án này dùng polling (H/5 * * * *: khoảng năm phút một lần) chính vì Jenkins nằm sau NAT ở nhà, không có địa chỉ công khai — topology quyết định trigger, một mẫu hình bạn sẽ gặp ở mọi mạng doanh nghiệp bị khóa chặt.

Stage / step các đơn vị của pipeline: stage là các pha có tên (hiện trên UI, chặn tiến trình); step là các lệnh bên trong.

Artifact bất cứ thứ gì build sinh ra mà đáng giữ lại: file jar, image, báo cáo test. Artifact phải *bất biến và truy vết được* — vì thế image được tag bằng SHA của git, để một pod đang chạy có thể truy ngược về đúng commit mà nó đang chạy.

Test gate tính chất rằng các stage sau chỉ chạy nếu các stage trước đã qua. Một pipeline triển khai dù test thất bại chỉ là một script triển khai với vài bước thừa.

Environment promotion cùng một artifact đi từ dev → staging → prod. Quy tắc sắt: *thăng cấp artifact, không bao giờ build lại* — một lần build lại “cho prod” là một binary khác với cái bạn đã test. Bộ máy biến môi trường của Chương 1 tồn tại chính xác để một image vừa với mọi môi trường.

Rollback lối thoát hiểm. Vì registry giữ các image tag theo SHA và Kubernetes giữ các ReplicaSet cũ (Chương 15), rollback là trở lại một artifact trước đó — vài giây, không phải một lần build lại.

QUYẾT ĐỊNH — TRIỂN KHAI Ở MỌI LẦN PUSH LÊN MAIN — CHO MỘT LẬP TRÌNH VIÊN

Continuous Deployment không cần phê duyệt là lựa chọn đúng ở đây: một lập trình viên, bán kính ảnh hưởng cỡ homelab, và vòng phản hồi nhanh nhất có thể. Cùng lựa chọn đó ở một ngân hàng sẽ sai, không phải vì tự động hóa rủi ro mà vì các thay đổi *chịu quản lý* cần bằng chứng đã được xem xét. Con đường lên cấp giữ nguyên pipeline và thêm cổng: một stage phê duyệt thủ công trước prod, hoặc — Chương 27 — GitOps, nơi “deploy” trở thành một pull request có thể xem xét vào repo cấu hình. Hình dạng pipeline sống sót; chỉ chính sách cổng chặn thay đổi.

CẠM BẦY — PIPELINE XANH MÀ NÓI DỐI

Triệu chứng: lần chạy nào cũng xanh; production vẫn hồng. Các nguyên nhân kinh điển: test bị bỏ qua trong lệnh build (`-DskipTests` — điều mà pipeline *này* làm ở stage build, chấp nhận một cách có ý thức vì `mvn verify` chạy trong CI trên GitHub Actions — hãy biết rõ các cổng của chính mình); stage deploy không kiểm chứng rollout (pipeline này có — `rollout status` làm build thất bại nếu pod không bao giờ Ready); hoặc thành công được đo bằng “các lệnh thoát với mã 0” thay vì “service trả lời request.” Một pipeline xanh là một lời khẳng định; hãy biết chính xác nó khẳng định điều gì.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. CI và CD đưa ra những lời hứa khác nhau. Nêu mỗi lời hứa trong một câu, và chỉ ra stage nào trong pipeline của dự án này thực hiện lời hứa nào.
2. Webhook so với polling: nêu ràng buộc quyết định trong dự án này, và một bối cảnh doanh nghiệp có cùng ràng buộc đó.
3. Vì sao một artifact được thăng cấp không bao giờ được build lại cho từng môi trường, và cơ chế nào từ Chương 1 làm cho việc thăng cấp một-artifact trở nên khả thi?
4. Pipeline của bạn xanh và prod sập. Liệt kê ba cách khác nhau mà một pipeline có thể nói dối, và cách sửa cho mỗi cách.

Jenkins: CI server

22.1 Kiến trúc

Jenkins là một ứng dụng web Java — *controller* — lưu job, credential, plugin và lịch sử build trong một thư mục, JENKINS_HOME, và thực thi build hoặc trên chính nó (qua các *executor*, các khe build song song của nó) hoặc trên các *agent* từ xa. Mẫu hình doanh nghiệp là một controller không build gì cả, đẩy việc ra các agent dùng xong bỏ (thường là pod, qua plugin Kubernetes).

QUYẾT ĐỊNH — BUILD TRÊN CONTROLLER, JENKINS NGOÀI CLUSTER

Hai sự đảo ngược thực hành doanh nghiệp kiểu homelab, cả hai đều là phép tính CPU. Agent pod sẽ tốn chi phí lập lịch và kéo image cho mỗi lần build trên cùng hai nhân mà chính build cần; hai executor của controller dành những chu kỳ đó cho Maven thay vì thế. Và Jenkins chạy trong Docker trên máy chủ, không phải trong k3s: nếu bạn làm hỏng cluster — và trong lúc học, bạn sẽ làm hỏng — thứ có thể dựng lại cluster không được chết cùng nó. Cả hai đều đảo ngược gọn gàng ở quy mô doanh nghiệp: Jenkins trong cluster với pod agent là nội dung của Chương 27, và bản thân cuộc di dời chính là bài học.

22.2 Image của controller, có chú giải

Thay vì bấm chuột cài công cụ vào một Jenkins nguyên bản, dự án nướng toolchain vào một image — `deploy/jenkins/Dockerfile`:

```
FROM jenkins/jenkins:lts-jdk21      ①
USER root
RUN ... temurin-25-jdk ...           ②
RUN ... apache-maven-3.9.9 ...      ③
RUN wget -qO /usr/local/bin/kubect1 ... ④
ENV JDK25_HOME=/usr/lib/jvm/temurin-25-jdk-amd64 ⑤
USER jenkins
RUN jenkins-plugin-cli --plugins \
    git workflow-aggregator ... credentials-binding ⑥
```

① Jenkins LTS chạy trên JDK 21 — phiên bản mà *chính Jenkins* chạy trên đó.

- ② Dự án nhắm tới Java 25, nên một JDK thứ hai được cài cho việc *build*. Lưu ý JAVA_HOME *không* bị đổi toàn cục ⑤ — Jenkinsfile đặt nó cho từng pipeline. Runtime của controller và toolchain build là hai mối quan tâm độc lập; Dockerfile này giữ chúng tách bạch rõ ràng.
- ③ Maven ghim ở 3.9.9 — gói của distro thường lạc hậu.
- ④ kubectl: stage deploy chỉ là binary này cộng với một kubeconfig.
- ⑥ Plugin ghim trong image, không bấm chọn trên UI: hỗ trợ pipeline, git, báo cáo JUnit, và credentials-binding (dùng bởi withCredentials ở Chương 23). Toàn bộ ý nghĩa: *bản thân CI server giờ tái tạo được* — hủy container, build lại từ Dockerfile, và chỉ JENKINS_HOME (một volume có tên) mang trạng thái.

Một cờ lúc chạy trong run.sh xứng đáng có đoạn riêng: --network host. Jib bên trong Jenkins push tới localhost:5000 — và bên trong một container dùng mạng bridge, localhost sẽ là *container*. Mạng host làm Jenkins dùng chung namespace mạng của máy chủ, nên localhost:5000 của nó chính là registry mà k3s kéo image từ đó (hợp đồng đặt tên của Chương 13). Không phải đường tắt — mà là yêu cầu của cách đặt tên image.

22.3 Credential

Jenkins lưu secret trong một kho mã hóa, tham chiếu bằng ID; pipeline gắn chúng vào biến môi trường chỉ trong phạm vi một khối (withCredentials), và bộ che log xóa giá trị của chúng khỏi đầu ra console. Dự án này lưu một credential: kubeconfig-ts, một credential dạng file chứa kubeconfig mà Jenkins dùng để triển khai — và đó là phần thú vị.

22.4 Danh tính triển khai: một ServiceAccount, không phải admin

Con đường lười biếng là đưa cho Jenkins kubeconfig admin của k3s — cluster-admin, được phép xóa mọi namespace. Thay vào đó, deploy/k8s/jenkins-rbac.yaml dựng một danh tính tối thiểu:

```
kind: ServiceAccount          # the identity
metadata: { name: jenkins, namespace: ts }
---
kind: Role                   # namespaced permissions
metadata: { name: jenkins-deployer, namespace: ts }
rules:
  - apiGroups: ["apps"]
```

```

resources: ["deployments", "statefulsets"]
verbs: ["get", "list", "watch", "create", "patch", "update"] ①
- apiGroups: ["apps"]
  resources: ["replicasets"]
  verbs: ["get", "list", "watch"] ②
# + services, PVCs, secrets, configmaps, ingresses, pods/log ...
- apiGroups: ["kafka.strimzi.io"] ③
  resources: ["kafkas", "kafkanodepools", "kafkatopics"]
  verbs: ["get", "list", "create", "patch", "update"]
---
kind: RoleBinding                                # identity <- permissions

```

- ① Các verb được suy ra từ chính các lệnh của pipeline: `kubectl apply` cần `get+create+patch`; `set image` cần `get+patch`. *Không có delete ở bất cứ đâu*: CI chỉ tiến tới; con người mới xóa. Một pipeline bị chiếm quyền vì thế có thể làm hỏng các Deployment trong ts nhưng không thể quét sạch chúng, và không thể nhìn ra ngoài namespace — Role, không phải ClusterRole, là quyết định về bán kính ảnh hưởng.
- ② Phát hiện theo cách trung thực: `rollout status` theo dõi các ReplicaSet của Deployment, nên cần quyền đọc chúng — loại quyền bạn tìm ra bằng cách chạy pipeline với Role đó và đọc lỗi Forbidden đầu tiên.
- ③ Thêm cùng cột mốc 6: pipeline được phép khai báo cluster và topic Kafka (custom resource theo namespace), nhưng cài operator và các CRD của nó — phạm vi cluster — vẫn là việc của admin (Chương 25). RBAC lớn thêm một rule cho mỗi API group mà pipeline chạm tới; file Role là nhật ký thay đổi của những gì CI đã được tin cậy.

Kubeconfig lưu trong Jenkins đơn giản ghép địa chỉ cluster với token của ServiceAccount này thay vì chứng chỉ admin.

CẠM BẤY — FORBIDDEN: CANNOT CREATE RESOURCE "NAMESPACES"

Triệu chứng: stage deploy chết với lỗi Forbidden trên namespaces dù file RBAC trông đã đầy đủ. Nguyên nhân: namespace thuộc *phạm vi cluster*, và một Role theo namespace không thể cấp quyền đó — chính xác là lý do namespace.yaml bị loại khỏi kustomization (Chương 20) và namespace được admin tạo một lần. Gỡ lỗi RBAC nói chung: `kubectl auth can-i patch deployments -n ts --as system:serviceaccount:ts:jenkins` trả lời “việc này có được phép không” mà không cần chạy pipeline.

THỰC HÀNH — CHỨNG MINH RANH GIỚI TỒN TẠI

```
kubectl auth can-i patch deployments -n ts \
  --as system:serviceaccount:ts:jenkins      # yes
kubectl auth can-i delete deployments -n ts \
  --as system:serviceaccount:ts:jenkins      # no
kubectl auth can-i list pods -n kube-system \
  --as system:serviceaccount:ts:jenkins      # no
```

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Những gì nằm trong JENKINS_HOME, và vì sao nướng plugin vào image cộng với một volume cho home làm CI server trở thành thứ dùng xong bỏ được?
2. Vì sao controller chạy hai JDK, và mỗi JDK được chọn ở đâu?
3. Dựng lại danh sách verb RBAC cho một pipeline chỉ chạy kubectl apply -k và rollout status — verb nào trong Role của dự án khi đó có thể bỏ đi?
4. Giải thích --network host trên container Jenkins theo hợp đồng đặt tên image của Chương 13.
5. ServiceAccount, Role, RoleBinding: cái nào là danh tính, cái nào là năng lực, cái nào là mối nối — và vì sao Role-chứ-không-phải-ClusterRole là tiêu đề bảo mật?

Pipeline, từng dòng một

Mọi thứ trong cuốn sách này hội tụ về một file Groovy 70 dòng. Một Jenkinsfile dạng khai báo sống trong chính repository mà nó build — pipeline dưới dạng mã, có phiên bản và được xem xét như mọi thứ khác.

23.1 Phần mở đầu và environment

```

pipeline {
  agent any                                ❶
  options {
    timestamps()
    disableConcurrentBuilds()              ❷
    buildDiscarder(logRotator(numToKeepStr: '10'))
  }
  triggers { pollSCM('H/5 * * * *') }      ❸
  environment {
    JAVA_HOME = "${env.JDK25_HOME}"         ❹
    PATH       = "${env.JDK25_HOME}/bin:${env.PATH}"
    TAG        = "${env.GIT_COMMIT.take(7)}"  ❺
    NS         = 'ts'
    REGISTRY   = 'localhost:5000'
    SERVICES   = 'config-server discovery-server course-service api-gateway auth-service
    ↪ dictionary-service notification-service'
    MODULES    = 'common,config-server,discovery-server,course-service,api-gateway,auth-
    ↪ service,dictionary-service,notification-service'
  }
}

```

- ❶ Bất kỳ executor nào trên controller — thỏa hiệp homelab của Chương 22 gói trong một từ.
- ❷ Hai build cùng lúc trên hai nhân sẽ giẫm đạp lên nhau; các lần chạy xếp hàng cũng được gộp lại (một commit mới trong lúc đang build thay thế những lần đang chờ).
- ❸ Polling vì NAT chặn webhook. Chữ H là phân bố theo hash: “một phút cố định nào đó trong mỗi 5 phút,” để nhiều job không cùng poll vào phút :00.
- ❹ Công tắc JDK theo từng pipeline mà image của controller đã chuẩn bị: build thấy Java 25, bản thân Jenkins vẫn ở 21.
- ❺ Tag triển khai là SHA git rút gọn — mọi image truy vết được về commit của

nó, mọi lần deploy ghim vào một tag bất biến (bài học tag-so-với-digest của Chương 13 được áp dụng).

SERVICES/MODULES sổ đăng ký của pipeline về những gì tồn tại: **MODULES** cấp cho Maven (phân cách bằng dấu phẩy, gồm cả **common** được build nhưng không sinh image); **SERVICES** cấp cho kubectl (phân cách bằng dấu cách, chỉ những thứ triển khai được). Thêm một service vào cluster nghĩa là mở rộng cả hai — bước thủ công duy nhất cho mỗi service mới.

23.2 Ba stage

```
stage('Build') {
  steps {
    dir('Project/TeacherSupporter') {           ①
      sh 'mvn -B -DskipTests -pl ${MODULES} -am verify' ②
    } } }
stage('Push images') {
  steps {
    dir('Project/TeacherSupporter') {
      sh 'mvn -B -DskipTests -pl ${MODULES} -am package jib:build -Djib.to.tags=${TAG}'
      ↪ ③
    } } }
stage('Deploy') {
  steps {
    dir('Project/TeacherSupporter') {
      withCredentials([file(credentialsId: 'kubecfg-ts',
        variable: 'KUBECONFIG')]) {           ④
        sh '''
          set -eu                               ⑤
          kubectl apply -k deploy/k8s/base       ⑥
          for s in ${SERVICES}; do
            kubectl -n ${NS} set image deployment/${s} \
              ${s}=${REGISTRY}/${s}:${TAG}       ⑦
          done
          for s in ${SERVICES}; do
            kubectl -n ${NS} rollout status \
              deployment/${s} --timeout=300s     ⑧
          done
        '''
      } } } }
```

- ① Gốc repository là một monorepo học tập; dự án nằm trong một thư mục con, nên mọi stage đều `dir()` vào đó.
- ② `-pl` (project list) + `-am` (also make dependencies): chỉ build các module được liệt kê và những gì chúng cần — kinh tế học monorepo trên hai nhân. `-B` là

- chế độ batch (không có đầu ra tương tác). `-DskipTests` là sự làm yếu cồng kềnh được chấp nhận có ý thức từ hộp cam bẫy của Chương 21.
- ③ Một lệnh build và push cả bảy image — Chương 13 gói trong một dòng. `-Djib.to.tags` thêm tag SHA bên cạnh tag SNAPSHOT. Chữ package đứng trước `jib:build` không phải để trang trí — xem hộp cam bẫy bên dưới; nó đã tồn tại hai lần build thất bại trong ngày cột mốc 5 và 6 ra lò.
 - ④ Credential dạng file hiện hình thành một file tạm; `KUBECONFIG` là biến môi trường mà `kubectl` đọc. Ngoài khối này file không tồn tại; trong log nội dung của nó không bao giờ xuất hiện.
 - ⑤ Thất bại khi gặp bất kỳ lỗi nào và khi gặp biến chưa đặt — thiếu nó, một lần apply thất bại sẽ bị lướt qua và “đã triển khai” được báo cáo.
 - ⑥ Hội tụ cluster về các manifest — lũy đẳng (Chương 14), nên chạy lại an toàn ở mỗi build dù có gì thay đổi hay không.
 - ⑦ Trỏ lại từng Deployment vào image tag theo SHA. Đây mới là thứ thật sự kích hoạt rollout — và lưu ý bản thân manifest vẫn giữ tag SNAPSHOT, nên chỉ apply thôi sẽ không triển khai lại; việc ghim của pipeline và giá trị mặc định của manifest cùng tồn tại.
 - ⑧ Cổng kiểm chứng: chặn cho đến khi mỗi rollout hoàn tất; làm build thất bại nếu có pod nào không bao giờ Ready trong vòng năm phút. Dòng này là thứ khiến màu xanh của pipeline có nghĩa “đang chạy,” chứ không phải “các lệnh đã thực thi” — nó ủy thác sự thật cho các probe của Chương 17.

CẠM BÃY — COULD NOT FIND ARTIFACT COM.TS:COMMON:JAR:1.0.0-SNAPSHOT

Triệu chứng: stage Build qua, rồi stage Push thất bại ở module đầu tiên phụ thuộc vào `common`, không phân giải được một file jar mà stage trước vừa build xong. Nguyên nhân: `mvn jib:build` là một *goal trần* — Maven chạy thẳng nó, không có pha vòng đời nào, nên không có gì trong reactor được biên dịch hay đóng gói trong phiên đó, và các phụ thuộc anh em quay về kho cục bộ, nơi `common` chưa bao giờ được cài (`verify` dừng trước `install`). Các module không có phụ thuộc anh em (`config-server`, `discovery-server`) qua trót lọt, khiến lỗi trông như đặc thù của từng module. Cách sửa: đặt một pha lên trước — package `jib:build` — để reactor đóng gói `common` trước khi Jib hỏi đến nó. Quy tắc chung: một goal của plugin cần artifact của module khác phải đi cùng một pha vòng đời, không được gọi một mình.

Hai vòng lặp thay vì một là một điểm đúng đắn nhỏ: mọi rollout khởi động trước, rồi mới đợi tất cả — nếu không, việc chờ service N sẽ tuần tự hóa việc khởi động service N+1, kéo dài tổng thời gian triển khai mà chẳng được gì.

23.3 Xử lý thất bại

```
post {
  success { echo "Deployed ${TAG} to namespace ${NS}." }
  failure {
    dir('Project/TeacherSupporter') {
      withCredentials([...]) {
        sh 'kubectl -n ${NS} get pods || true'
      } } }
}
```

Khi thất bại, log build kết thúc bằng một danh sách pod — lệnh đầu tiên mà một con người sẽ chạy, đã được chạy sẵn. Phần `|| true` giữ cho chính bước chẩn đoán không che mất trạng thái thoát thật của build.

23.4 Cùng pipeline đó, ở nơi khác

Để hiệu chỉnh, đây là hình dạng y hệt dưới dạng GitHub Actions:

```
on: { push: { branches: [main] } }      # webhook, not polling
jobs:
  deploy:
    runs-on: ubuntu-latest                # disposable agent, not controller
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: 25 }
      - run: mvn -B -pl $MODULES -am verify
      - run: mvn -B -pl $MODULES -am jib:build -Djib.to.tags=${GITHUB_SHA::7}
      - run: kubectl apply -k deploy/k8s/base && ...
      env: { KUBECONFIG_DATA: ${ secrets.KUBECONFIG } }
```

Cùng các stage, cùng cách đánh tag, cùng ý tưởng gắn secret. Khác biệt nằm ở sự đánh đổi hạ tầng: runner của GitHub là các VM dùng xong bỏ (agent làm đúng cách, miễn phí) nhưng chúng sống trên internet — nên biến thể này sẽ cần registry và cluster truy cập được từ bên ngoài, đúng rằng buộc đã định hình thiết kế homelab. Pipeline thì khả chuyển; *topology mạng mới là thứ thật sự khác nhau giữa các nơi.*

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Vì sao tag các lần deploy bằng SHA git khi tag SNAPSHOT cũng tồn tại? Nếu chế độ lỗi mà deploy chỉ-SNAPSHOT mời gọi (Chương 13) và khả năng truy vết mà SHA bổ sung.

2. Giải thích `-pl ... -am` và nó tiết kiệm gì trong một monorepo. Điều gì hỏng nếu bạn quên mở rộng `MODULES` cho một service mới?
3. Dòng đơn lẻ nào khiến thành công của pipeline này có nghĩa “các pod đã Ready,” và nó dựa vào bộ máy của chương nào?
4. Vì sao dùng hai vòng lặp trong stage `deploy` thay vì `set-image-rồi-đợi` cho từng service?
5. Liệt kê ba thứ thay đổi và ba thứ không đổi khi Jenkinsfile này được chuyển sang GitHub Actions.

Phần VII

Vận hành và chặng đường phía trước

Vận hành stack: Cẩm nang gỡ lỗi thực địa

Đây là chương để đọc lại khi có một cluster hỏng trước mặt bạn. Nó tập hợp quy trình gỡ lỗi mà các chương trước đã tích lũy, sắp xếp theo cách sự cố thật diễn ra.

24.1 Bốn câu hỏi

Gần như mọi sự cố Kubernetes đều được định vị bằng cách hỏi, theo thứ tự:

1. **Pod có đang chạy không?** `kubectl -n ts get pods` — đọc cột STATUS và RESTARTS; bảng phân loại bên dưới.
2. **Nền tảng nói gì về nó?** `kubectl -n ts describe pod <pod>` — mục Events ở cuối là kubelet và scheduler kể lại các quyết định của chúng: kéo image, probe thất bại, OOM kill, từ chối lập lịch.
3. **Ứng dụng nói gì?** `kubectl -n ts logs <pod>` (thêm `--previous` cho container vừa chết — cờ hay bị quên nhất trong ứng phó sự cố).
4. **Tôi có tái hiện được góc nhìn của nó về thế giới không?** `kubectl -n ts exec -it <pod> -- sh` rồi thử kết nối mà ứng dụng khẳng định là đang thất bại.

24.2 Bảng phân loại trạng thái

THỰC HÀNH — CỐ Ý LÀM HỎNG — BA BÀI DIỄN TẬP

Phá hoại có chủ đích, vào một ngày yên tĩnh, dạy nhiều hơn bất kỳ sự cố nào:

```
# 1. wrong image tag -> watch ImagePullBackOff, then roll back
kubectl -n ts set image deployment/course-service \
    course-service=localhost:5000/course-service:nope
kubectl -n ts rollout undo deployment/course-service

# 2. scale the database down -> watch dependents degrade honestly
kubectl -n ts scale statefulset/postgres-course --replicas=0
kubectl -n ts get pods -w      # course-service readiness reacts
kubectl -n ts scale statefulset/postgres-course --replicas=1
```

Trạng thái	Ý nghĩa và bước đi đầu tiên
Pending	không lập lịch được: requests vượt quá tài nguyên còn trống, hoặc PVC chưa được gắn. describe cho biết là gì.
ImagePullBackOff	registry không truy cập được, gõ sai tên, hoặc bẫy HTTPS-so-với-HTTP của Chương 13.
CrashLoopBackOff	tiến trình thoát ngay sau khi khởi động; thời gian chờ nhân đôi giữa các lần thử. logs --previous.
OOMKilled (exit 137)	chạm giới hạn bộ nhớ; không có trace Java nào — cú kill từ kernel của Chương 17. Tăng limit hoặc thu nhỏ heap.
Running 0/1	còn sống nhưng readiness thất bại: bình thường khi khởi động; là vấn đề khi kéo dài — thử chính URL của probe.
Completed/Error	cho Job; với pod service, Error nghĩa là tiến trình đã thoát với mã khác 0 một lần.

Bảng 24.1: Cột STATUS đang cố nói với bạn điều gì.

```
# 3. kill Eureka -> verify existing routes survive on client caches
kubectl -n ts delete pod -l app=discovery-server
```

Mỗi bài diễn tập ghép lý thuyết của một chương với hành vi quan sát được của nó: 13 (đặt tên), 17–18 (probe và endpoint), 3 (cache phía client).

24.3 Đọc thể giới của một service từ bên trong

Khi log khẳng định kết nối thất bại, hãy kiểm chứng lời khẳng định đó từ chính vị trí mạng của pod — DNS và khả năng truy cập khác nhau giữa máy bạn và pod:

```
kubectl -n ts exec -it deploy/course-service -- sh
wget -qO- http://config-server:8888/actuator/health # DNS + service up?
nc -zv postgres-course 5432 # TCP reachable?
env | grep SPRING # what was injected?
```

Lệnh cuối cùng đó dần xếp tranh cãi rất nhanh: nó in ra cấu hình mà pod *thật sự* nhận được, hơn hẳn việc đọc lại manifest.

24.4 Thói quen giữ cho cuối tuần được rảnh

Sau mỗi lần deploy pipeline đã kiểm chứng rollout rồi; dù vậy vẫn liếc qua `kubectl -n ts get pods — RESTARTS` nhích dần lên là sự cố ngày mai đang tự báo trước.

Hằng tuần `kubectl top pods -n ts` (cần `metrics-server`, đi kèm trong `k3s`): so sánh mức dùng thực tế với requests của Chương 17 và sửa lại phần `hư cấu`. Kiểm tra đĩa: `df -h` trên máy chủ — `registry` và file log lớn dần; `registry:2` thỉnh thoảng cần dọn rác.

Trước buổi demo *đừng build* — hộp phần cứng của Chương 5: hai nhân nghĩa là một lần build Maven và buổi demo tranh nhau đúng phần CPU đang giữ cho `readiness probe` trả lời.

TRÊN PHẦN CỨNG NÀY

`ufw` trên máy chủ phải miễn trừ các CIDR của pod và service (`10.42.0.0/16`, `10.43.0.0/16`) — chặn tường lửa chúng sẽ phá DNS và lưu lượng pod-tới-pod theo những cách giả dạng lỗi ứng dụng: timeout chấp chờn, tên lúc phân giải được lúc không. Khi mạng trên `k3s` xảy ra chuyện *kỳ lạ*, hãy nghi ngờ tường lửa máy chủ trước ứng dụng. Và `k3s` chỉ đọc `registries.yaml` lúc khởi động — sau khi sửa, `systemctl restart k3s`, lệnh này khởi động lại mọi pod trong chốc lát: đừng làm giữa buổi demo.

CẠM BẦY — GỠ LỖI SAI TẦNG

Thói quen đắt giá nhất khi gỡ lỗi Kubernetes là bắt đầu từ nơi triệu chứng xuất hiện thay vì nơi nguyên nhân sống. Một lỗi 502 trên trình duyệt có thể là Traefik (không có endpoint), gateway (Eureka lạc hậu), service (readiness thất bại), cơ sở dữ liệu (pod bị đuổi), hay đĩa (đầy). Bốn câu hỏi tồn tại để đi qua các tầng theo thứ tự; hãy cưỡng lại cám dỗ mở log ứng dụng trước chỉ vì đó là tầng quen thuộc nhất. `describe` trước `logs` — nền tảng thường đã biết rồi.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Sắp xếp bốn câu hỏi và nêu lệnh cho mỗi câu. Câu nào dùng một cờ mà mọi người hay quên, và khi nào cờ đó quan trọng?
2. `CrashLoopBackOff` SO VỚI `Running` 0/1: các tầng khác nhau đang thất bại — khái niệm probe nào phân biệt chúng?
3. Một service khẳng định không tới được Postgres; từ laptop của bạn Post-

gres trả lời. Bước chẩn đoán tiếp theo là gì và vì sao thành công trên laptop của bạn không liên quan?

4. Vì sao một thay đổi tường lửa máy chủ có thể giả dạng lỗi ứng dụng trên k3s, và hai CIDR nào phải luôn được miễn trừ?

Kafka trong cụm: Strimzi và luồng gửi mail

Cột mốc 6 chuyển broker của Chương 10 từ Compose vào k3s và triển khai service cuối cùng — notification-service — để luồng *sự kiện* đầu tiên chạy trơn vẹn trong cụm: đăng ký một người dùng, nhận một lá mail. Đây cũng là chương mà ý tưởng điều hòa (reconciliation) của Chương 14 mọc thêm phần mở rộng quan trọng nhất của nó: *operator*.

25.1 Operator và custom resource

Kubernetes có sẵn controller cho các kind dựng sẵn — Deployment, Service. Một *operator* là controller cho những kind bạn tự thêm: một *CustomResourceDefinition* (CRD) dạy API server một danh từ mới (Kafka, KafkaTopic), và vòng lặp điều hòa của operator biến các thể hiện của nó thành những object bình thường — pod, Service, ConfigMap — rồi giữ chúng hội tụ. Kiến thức vận hành trước đây nằm trong runbook (cách roll một broker an toàn, cách tính advertised listener, cách thêm partition) nay được mã hóa trong code của operator. Strimzi là operator cho Kafka; mẫu hình này là phổ quát (operator cho Postgres, Prometheus operator, cert-manager).

25.2 Cài operator: một việc của admin

```
# deploy/k8s/install-strimzi.sh (abridged)
kubectl apply --server-side --force-conflicts \
  -f "https://strimzi.io/install/latest?namespace=${NS}" -n "$NS" ①
kubectl -n "$NS" rollout status deployment/strimzi-cluster-operator
```

- ① Gói này chứa các CRD, Deployment của operator và RBAC của nó — phần lớn ở *phạm vi cụm* (cluster-scoped). Hai hệ quả: nó do admin apply chứ không phải Jenkins (Role của Jenkins, Chương 22, dừng ở namespace), và nó được apply *server-side* vì các tài liệu CRD quá lớn cho annotation last-applied phía client. Lũy đẳng, nên cũng chính là lệnh nâng cấp. Tham số `?namespace=` giới hạn operator chỉ theo dõi ts.

Kustomization (Chương 20) liệt kê `kafka.yaml`, nhưng file đó không thể apply cho đến khi script này đã chạy một lần — thành viên thứ ba của bộ “điều kiện tiên quyết do admin làm”, sau namespace và các secret.

CẠM BẦY — NO MATCHES FOR KIND “KAFKA” IN VERSION v1BETA2

Triệu chứng: apply một manifest Kafka chép từ bất kỳ tutorial nào cũng thất bại với lỗi này dù operator đã cài và khỏe mạnh. Nguyên nhân: Strimzi 1.x chỉ phục vụ `kafka.strimzi.io/v1`; phiên bản `v1beta2` mà cả thập kỷ bài blog dùng đã biến mất. Hãy kiểm tra CRD thật sự phục vụ phiên bản nào trước khi tin một ví dụ: `kubectl get crd kafkas.kafka.strimzi.io -o jsonpath={.spec.versions[*].name}`. Dự án này dính lỗi đó ngay ngày cài đặt.

25.3 Cụm, được khai báo

```
apiVersion: kafka.strimzi.io/v1
kind: KafkaNodePool                                ①
metadata:
  name: dual-role
  labels: { strimzi.io/cluster: ts-kafka } ②
spec:
  replicas: 1
  roles: [controller, broker]
  storage: { type: persistent-claim, size: 5Gi, deleteClaim: false } ③
  resources:
    requests: { memory: 768Mi, cpu: 200m }
    limits: { memory: 1Gi }
---
apiVersion: kafka.strimzi.io/v1
kind: Kafka
metadata:
  name: ts-kafka
spec:
  kafka:
    listeners:
      - { name: plain, port: 9092, type: internal, tls: false } ④
    config:
      offsets.topic.replication.factor: 1 ⑤
      default.replication.factor: 1
      min.insync.replicas: 1
      log.retention.hours: 168
      jvmOptions: { -Xms: 256m, -Xmx: 512m } ⑥
    entityOperator:
      topicOperator: {} ⑦
```

- ① Hai object: *pool* nói Kafka chạy ở đâu (replica, vai trò, lưu trữ, tài nguyên); *resource* Kafka nói nó hành xử thế nào. Một node KRaft hai vai trò là cách Kubernetes viết `process.roles=broker,controller` của Compose.
- ② Label này nối *pool* với *cụm*; operator khớp theo nó. Sai label thì im lặng — *pool* đơn giản là không bao giờ thành hình.
- ③ Một PVC qua *local-path*, y hệt các cơ sở dữ liệu (Chương 19). `deleteClaim:false`: xóa *resource* Kafka vẫn giữ lại log — cùng sự bất đối xứng mà *StatefulSet* có theo mặc định.
- ④ So với khối ba *listener*, hai *advertised address* trong `docker-compose.yml`: biến mất. Bạn khai báo *một listener nội bộ*; operator tính ra *Service bootstrap* và *Service* cho từng broker, các *advertised address*, và *listener* của KRaft controller. Mê cung của Chương 10 là kiến thức vận hành; giờ nó nằm trong operator.
- ⑤ Một broker duy nhất nghĩa là mọi *replication factor* phải bằng 1, nếu không lần tạo topic đầu tiên sẽ thất bại vì đòi những broker không tồn tại.
- ⑥ Heap mặc định của Kafka giả định một máy chủ; nửa GiB đủ cho lưu lượng nhỏ giọt của homelab và để dành bộ nhớ cho các service JVM.
- ⑦ Nửa topic của *entity operator* theo dõi các object *KafkaTopic* và điều hòa chúng thành topic thật. Nửa user bị bỏ qua: không xác thực, không có user nào để quản lý.

Strimzi đặt tên *Service bootstrap* là `<cluster>-kafka-bootstrap`; vì thế mọi *producer* và *consumer* nhận `SPRING_KAFKA_BOOTSTRAP_SERVERS=ts-kafka-kafka-bootstrap:9092` trong *Deployment* của nó — mặc định `localhost:9092` trong cấu hình của Phần III bị ghi đè theo đúng cách mọi địa chỉ khác đã bị ghi đè.

QUYẾT ĐỊNH — OPERATOR, KHÔNG PHẢI STATEFULSET VIẾT TAY

Một *StatefulSet* KRaft thuần (một pod, một PVC, khối *env* của *compose* chép vào) sẽ tốn ít bộ nhớ hơn — không operator, không *entity operator* — và đó là phương án dự phòng thắng thắn nếu tranh chấp CPU có lúc gây khó. Operator thắng vì đó là thứ các công ty dùng Kubernetes chạy (Red Hat bán nó dưới tên AMQ Streams), vì topic trở thành object khai báo, và vì nâng cấp broker cùng khởi động lại cuốn chiếu trở thành code đã được người khác kiểm thử. Học nó chính là mục đích; phương án dự phòng vẫn được ghi lại trong `STACK-K3S-JENKINS.md`.

25.4 Topic dưới dạng object

```
apiVersion: kafka.strimzi.io/v1
kind: KafkaTopic
metadata:
  name: ts.user.registered
  labels: { strimzi.io/cluster: ts-kafka }
spec: { partitions: 1, replicas: 1 }
```

Kafka sẽ tự tạo các topic này ở lần dùng đầu tiên; khai báo chúng biến số partition thành một quyết định được review, loại bỏ cuộc đua ngay đầu giữa producer và topic, và biến `kubectl get kafkatopics` thành bản kiểm kê mọi hợp đồng sự kiện trong hệ thống (bốn, khớp với bảng của Chương 6). Mỗi topic một partition: thứ tự toàn cục, nên đương nhiên giữ được thứ tự theo từng người dùng mà các publisher đặt key cho. Partition có thể thêm về sau, không bao giờ bớt được.

25.5 Service cuối cùng, và hộp thư của nó

Manifest của notification-service là Deployment tiêu chuẩn của Chương 15 với ba điểm riêng: địa chỉ bootstrap ở trên; `SPRING_MAIL_HOST=maildev / PORT=1025` ghi đè mặc định `#{MAIL_HOST:localhost}` của cấu hình qua relaxed binding; và các probe trên `/actuator/health` — service này hoàn toàn không có Spring Security, nên endpoint mở theo mặc định (câu trả lời thứ ba cho câu hỏi về probe của Chương 17). Maildev là một Deployment nhỏ xíu kèm Service (cổng 1025 SMTP, 1080 web); nó không gửi đi đâu cả và hiển thị mọi thứ nó bắt được. Không có Ingress: đây là công cụ của người vận hành, truy cập qua port-forward. Nó cũng cung cấp bài học thực chiến thứ hai của cột mốc này: được apply bằng tay trước khi pipeline chạy, selector của nó thiếu label mà Kustomize thêm vào, và lần deploy đầu tiên của pipeline chết với “field is immutable” — cạm bẫy của Chương 20, học theo cách đặt giá.

CẠM BÃY — NOTREADY — FATALPROBLEM, TRONG KHI POD ĐANG RUNNING

Triệu chứng: vài phút sau khi apply, `kubectl get kafka` cho thấy READY trống và một condition ghi “Pod is unschedulable or is not starting” — thế nhưng `get pods` cho thấy broker 1/1 Running. Nguyên nhân: vòng điều hòa của operator hết timeout trong lúc image Kafka 600 MB vẫn đang tải qua Wi-Fi, và ghi lại một trạng thái thất bại; vòng lặp định kỳ *tiếp theo* (mỗi hai phút) quan sát thấy pod đang chạy và chuyển nó sang Ready. Trạng thái của operator chỉ trung thực về cuối cùng. Trước khi phản ứng với một condition đỏ, hãy kiểm tra pod mà nó đang mô tả — và kiểm tra log của operator, nơi các dòng

“reconciled” cho thấy vòng lặp vẫn đang quay.

TRÊN PHẦN CỨNG NÀY

Operator + broker + entity operator tốn khoảng 1,5 GB ở đây, với heap của broker giới hạn 512 MiB. Lần kéo image đầu tiên là phần đắt nhất — hãy chuẩn bị tinh thần mười phút trên Wi-Fi gia đình, và cạm bẫy trạng thái cũ ở trên. Mọi thứ sau đó đều nhanh: image đã được cache trên node, điều mà bài kiểm tra khối bên dưới tận dụng.

THỰC HÀNH — CHỨNG MINH BROKER, RỒI CHỨNG MINH LUỒNG

Một pod dùng xong vút với Kafka CLI, ở cùng vị trí mạng với các service, dùng image node đã có sẵn:

```
kubectl -n ts run kafka-smoke --rm -i --restart=Never \
  --image=quay.io/strimzi/kafka:1.2.0-kafka-4.3.1 -- sh -c '
  B=ts-kafka-kafka-bootstrap:9092
  bin/kafka-topics.sh --bootstrap-server $B --create --topic smoke \
    --partitions 1 --replication-factor 1
  echo hello | bin/kafka-console-producer.sh --bootstrap-server $B --topic smoke
  bin/kafka-console-consumer.sh --bootstrap-server $B --topic smoke \
    --from-beginning --max-messages 1 --timeout-ms 20000
  bin/kafka-topics.sh --bootstrap-server $B --delete --topic smoke'
```

Dùng một topic nháp, không phải topic thật: một chuỗi “hello” thuần văn bản trên `ts.user.registered` là poison pill cho consumer JSON (Chương 11) — được bỏ qua êm thấm nhờ `ErrorHandlingDeserializer`, nhưng việc gì phải phát hiện ra trong production. Rồi đến luồng thật, khi pipeline đã triển khai xong các service:

```
curl -X POST http://ts.local/api/auth/register -H 'Content-Type: application/json' \
  -d '{"email": "t@example.com", "password": "Passw0rd!", "firstName": "Test",
    "lastName": "User", "role": "TEACHER", "activationMethod": "EMAIL"}'
# -> 201 {"message": "Registration successful. Check your email ..."}
kubectl -n ts logs deploy/notification-service | grep -i registered
# maildev's REST API, from inside the cluster (or port-forward 1080 for the UI):
kubectl -n ts run mailcheck --rm -i --restart=Never \
  --image=curlimages/curl:8.10.1 -- -s http://maildev:1080/email
# -> [{"subject": "Activate your TeacherSupporter account", "to": "t@example.com", ...}
```

Request đó đi qua mọi phần của cuốn sách này: Ingress → gateway → auth-service → Postgres → Kafka → notification-service → SMTP. Nó chạy lần đầu ngày 2026-08-25, pipeline build #7 — sau khi các build #3–#6 mỗi lần dạy một

cạm bẫy của chương này và của các Chương 17, 20 và 23. Một đầu mỗi còn lỏng mà nó phơi ra: link kích hoạt trong mail vẫn ghi `localhost:8080`, mặc định dev của URL gốc cho link — một ghi đề env dành cho cột mốc 7, khi frontend cho cụm một địa chỉ công khai thật sự.

25.6 Ngoài dự án

Kafka được quản lý — AWS MSK, Confluent Cloud, endpoint Kafka của Azure Event Hubs — là operator của operator: cùng một khai báo hình dạng Kafka nhưng thực hiện trong console của cloud, với những broker bạn không bao giờ thấy. Chuyển từ đây sang đó là một chuỗi bootstrap cộng SASL/TLS trên listener; các object `KafkaTopic` sẽ nhường chỗ cho resource Terraform nói đúng điều tương tự. Bài học còn lại để mang theo: mẫu hình operator là cách Kubernetes lưu trữ *mọi* thành phần nền tảng có trạng thái — Postgres (CloudNativePG), Redis, Elasticsearch — và khi đã cài, đã dùng sai phiên bản, đã debug trạng thái của một cái, bạn biết hình dạng của tất cả chúng.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Định nghĩa operator theo cách nói về controller của Chương 14, và nêu hai kind object mà Strimzi đã thêm vào cụm này.
2. Vì sao `install-strimzi.sh` là bước của admin trong khi `kafka.yaml` do Jenkins apply? Khái niệm RBAC nào vạch ra ranh giới đó?
3. Liệt kê ba thứ mà khối Kafka của Compose cấu hình bằng tay nhưng resource Kafka không hề nhắc tới, và nói ai tính chúng bây giờ.
4. Bạn thấy `NotReady` / `FatalProblem` trên resource Kafka. Hai thứ bạn kiểm tra trước khi hành động là gì, và vì sao trạng thái đó có thể sai?
5. Vì sao khai báo topic dưới dạng object khi Kafka tự tạo được chúng, và vì sao mỗi topic chỉ một partition?

Frontend và một host duy nhất

Cột mốc 7 đưa thứ có thể triển khai cuối cùng — frontend React — vào cụm và giành lấy đường dẫn / trên `ts.local` mà Ingress của Chương 18 đã cố ý để trống. Đây là một cột mốc ngắn tính theo số dòng YAML và dài tính theo khái niệm: đó là image duy nhất mà pipeline build từ Dockerfile, pod duy nhất không phải JVM, và là thay đổi biến “các service gọi được bằng `curl`” thành “một ứng dụng dùng được trong trình duyệt”.

26.1 Frontend là gì, dưới mắt người vận hành

Một ứng dụng React đã build không phải là một server. `npm run build` (kiểm tra TypeScript, rồi đóng gói bằng Vite) sinh ra một thư mục file tĩnh: một `index.html`, một bundle JavaScript có mã băm nội dung trong tên (`index-CqZING8N.js`), một file CSS, một favicon. Không có gì trong đó chạy trên server; từng dòng của nó chạy trong trình duyệt của khách, rồi trình duyệt gọi `/api/...` qua HTTP như bất kỳ client nào khác. Vậy “triển khai frontend” nghĩa là triển khai một *file server* có chứa những file đó — ở đây là nginx — và bảo router biên biết request nào là file, request nào là API.

Cách nhìn đó quyết định mọi thứ còn lại trong chương này. Không config server (ứng dụng không có thuộc tính nào để import), không Eureka (không ai gọi nó theo tên), không secret (trình duyệt đọc được mọi thứ frontend nắm giữ, nên không được có gì bí mật ở đó), và một dấu chân tài nguyên nhỏ hơn cả chi phí nạp class của JVM nhỏ nhất.

26.2 Hai giai đoạn, một Dockerfile

Jib (Chương 13) build image JVM từ `target/classes`; nó không chạy được `npm`. Vì thế frontend dùng Dockerfile nhiều giai đoạn (multi-stage) kinh điển — và lý do nó có hai giai đoạn cũng chính là lý do các image JVM dùng image gốc JRE thay vì JDK:

```
FROM node:24-alpine AS build           ①
WORKDIR /app
COPY package.json package-lock.json ./ ②
RUN npm ci --no-audit --no-fund
```

```
COPY . .
RUN npm run build

FROM nginx:1.27-alpine ③
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html ④
EXPOSE 80
```

- ① Giai đoạn build có Node, trình biên dịch TypeScript, và sau npm ci là chừng 300 MB node_modules. Không thứ nào trong đó cần cho việc *phục vụ* kết quả.
- ② Lockfile trước, mã nguồn sau — quy tắc cache theo layer của Chương 12. npm ci cài đúng các phiên bản đã ghim trong package-lock.json và từ chối sửa nó; một chỉnh sửa mã nguồn chỉ làm mất hiệu lực các layer từ COPY . . trở xuống, nên việc tải dependency được bỏ qua ở mọi lần rebuild không đụng tới package.json.
- ③ Một image gốc mới tinh. Image cuối cùng là nginx cộng file tĩnh: khoảng 15 MB, không Node, không trình biên dịch, không trình quản lý gói — bề mặt tấn công nhỏ hơn và kéo nhanh hơn, miễn phí.
- ④ --from=build là phép sao chép multi-stage: chỉ dist/ bằng từ giai đoạn một sang giai đoạn hai. Mọi thứ khác ở giai đoạn một bị vứt bỏ khi build kết thúc.

QUYẾT ĐỊNH — SAO KHÔNG THÊM NODE VÀO IMAGE JENKINS RỒI DỪNG JIB?

Jib có thể gói bất kỳ thư mục nào thành image, nên con đường “Node trong Jenkins, Jib để đóng gói” là có. Nó bị loại vì che giấu kỹ thuật build image phổ biến nhất sau kỹ thuật ít phổ biến nhất. Mọi đội ngũ phát hành frontend đều có Dockerfile này; đọc và viết được nó tự thân đã là một kỹ năng thị trường tuyển dụng cần, và có một image không-Jib trong pipeline cũng chứng minh pipeline không âm thầm mang hình dáng của Jib. Cái giá là một Docker CLI trong image Jenkins và một socket daemon được mount — Mục 26.6.

26.3 nginx: fallback giúp SPA hoạt động

```
server {
    listen 80;
    root /usr/share/nginx/html;
    location / {
        try_files $uri $uri/ /index.html; ①
    }
    location /assets/ {
        add_header Cache-Control "public, max-age=31536000, immutable"; ②
    }
}
```



```

    }
    location = /healthz { return 200 "ok\n"; }
  }
}

```

- ① Dòng mà mọi ứng dụng một trang (SPA) đều cần và mọi lần triển khai đầu tiên đều quên. React Router (BrowserRouter trong `main.tsx`) sở hữu những URL như `/courses/42` hoàn toàn trong trình duyệt: bấm một link không bao giờ hỏi server. Nhưng một lần refresh, một bookmark hay một link dán vào có hỏi server về `/courses/42`, file đó không tồn tại, và một file server thuần trả 404. `try_files` thử đường dẫn nguyên văn, rồi thư mục, rồi rơi về `index.html` — ứng dụng khởi động và router của nó đọc URL. OAuth callback (`/oauth/callback`) cũng dựa vào điều này: redirect của Google là một request lạnh tới một route chỉ tồn tại trong JavaScript.
- ② Vite ghi mã băm nội dung vào tên mọi asset, nên file đổi thì luôn có tên mới. Điều đó khiến “cache vĩnh viễn” an toàn cho `/assets/` — và *không an toàn* cho `index.html`, thứ phải được tải lại để biết các tên mới sau mỗi lần deploy. Cache file HTML là cách người dùng rơi vào cảnh ứng dụng cũ đi đòi những bundle mà server không còn nữa.
- ③ Một đích cho probe không tốn gì và không bao giờ chạm đĩa (các probe của Chương 17 cần thứ gì đó để gọi).

Hãy để ý điều file này *không* làm: proxy `/api`. Trên laptop, `vite.config.ts` proxy `/api` tới gateway ở `localhost:8080` để dev server và API trông như một origin. Trong cụm, việc đó thuộc về Traefik — nginx không bao giờ thấy một request API nào cả.

26.4 Giành lấy /: một origin

```

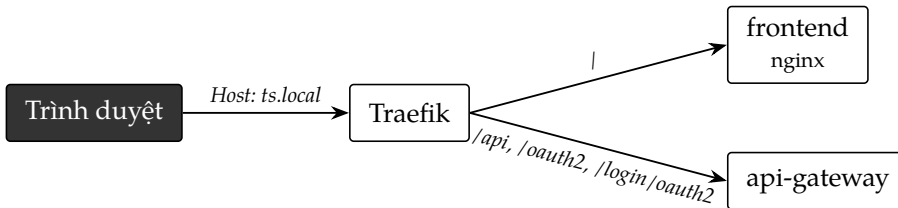
kind: Ingress
metadata: { name: frontend }
spec:
  rules:
    - host: ts.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend: { service: { name: frontend, port: { number: 80 } } }

```

- ① Một quy tắc bắt-tất-cả, trong một object Ingress *thứ hai* đặt cạnh Ingress của gateway. Traefik gộp mọi quy tắc Ingress của một host và định tuyến theo

tiền tố khớp dài nhất: /api/courses khớp cả / lẫn /api, và /api thắng. Ba tiền tố của gateway vì thế tiếp tục hoạt động không đổi, còn mọi thứ khác — /, /login, /courses/42, /assets/... — rơi xuống nginx. Hai object, một bảng định tuyến gộp; quy tắc “apply không bao giờ xóa” của Chương 20 vẫn áp dụng cho từng cái.

Hệ quả chính là lý do cột mốc này tồn tại: **một origin**. Trình duyệt tải trang từ `http://ts.local` và gọi `http://ts.local/api/...` — cùng scheme, host và cổng. So với Compose, nơi trang đến từ `localhost:3000` còn API từ `localhost:8080`: khác cổng là khác origin, nên mọi lời gọi API đều có một CORS preflight đi trước và `CorsConfig` của gateway phải liệt kê rõ `localhost:3000`. Trong cụm, `CorsConfig` đơn giản là không bao giờ được hỏi tới — chẳng có request cross-origin nào để kiểm soát. Cookie, redirect của OAuth, và bất kỳ chính sách `SameSite` nào về sau đều đơn giản đi vì cùng lý do.



Hình 26.1: Tiền tố dài nhất thắng: một host, hai backend.

Một thuộc tính đi theo việc đổi host. `OAuth2LoginSuccessHandler` kết thúc đăng nhập Google bằng cách chuyển hướng trình duyệt tới `app.oauth2.success-redirect-uri`, mà `auth-service.yml` của `config-server` đặt là `localhost:3000/oauth/callback` — đúng trên laptop, sai trong cụm. `auth-service.yml` ghi đè nó bằng biến môi trường `APP_OAUTH2_SUCCESS_REDIRECT_URI=http://ts.local/oauth/callback` — quy tắc ưu tiên của Chương 16 (biến môi trường thắng config server) làm đúng việc nó đã làm cho URL datasource.

26.5 Manifest

Deployment mang hình dáng của Chương 15 với gần như mọi thứ bị lược bỏ: không `envFrom`, không biến `SPRING_*`, không `startupProbe` (nginx lên trong vài mili giây), và `requests: { memory: 32Mi, cpu: 10m }` — một phần mười hai của JVM nhỏ nhất. `imagePullPolicy: Always` với `tag :latest` làm mặc định của manifest, được pipeline ghi đè bằng tag commit y hệt các service JVM.

26.6 Pipeline mộc thêm một stage

```
stage('Frontend image') {
  steps {
    dir('Project/TeacherSupporter') {
      sh '''
        docker build -t ${REGISTRY}/frontend:${TAG} \
          -t ${REGISTRY}/frontend:latest frontend ①
        docker push ${REGISTRY}/frontend:${TAG}
        docker push ${REGISTRY}/frontend:latest
      '''
    }
  }
}
```

- ① Build context là frontend/, được cắt gọn bởi một .dockerignore loại trừ node_modules và dist: các module native của laptop Windows không được gửi lên daemon, và image tự build lấy của mình. frontend gia nhập SERVICES (danh sách của kubectl) nhưng không vào MODULES (của Maven) — các vòng lặp set image / rollout status của stage Deploy xử lý nó mà không cần đổi gì.

Để docker build chạy được bên trong container Jenkins, hai thứ thay đổi trong thiết lập của Chương 22. Image có thêm Docker CLI (docker-ce-cli — chỉ phần client), và run.sh mount /var/run/docker.sock của host vào container. CLI nói chuyện qua socket đó với daemon của host; không có daemon nào bên trong Jenkins và không có Docker-in-Docker. Image tạo ra nằm trong kho image của host, và docker push localhost:5000/... tới được registry qua mạng host mà container vốn đã dùng chung.

CẠM BẤY — PERMISSION DENIED TRÊN /VAR/RUN/DOCKER.SOCK

Triệu chứng: CLI đã cài, socket đã mount, và mọi lệnh docker đều thất bại với lỗi quyền. Nguyên nhân: socket thuộc sở hữu của nhóm docker trên host — GID 984 trên ts-server — và con số đó chẳng có ý nghĩa gì bên trong image, nơi user jenkins thuộc một nhóm cùng tên nhưng GID khác (hoặc không có). Cách sửa trong run.sh: --group-add "\$(stat -c %g /var/run/docker.sock)" thêm GID số của host vào user của container lúc chạy, không phải nướng một con số đặc thù của máy vào Dockerfile.

QUYẾT ĐỊNH — SOCKET LÀ ROOT

Mount socket Docker trao quyền kiểm soát host tương đương root: ai chạy được `docker run -v /:/host` là sở hữu cả máy. Với một homelab một người dùng mà Jenkins chỉ truy cập được qua tailnet, đây là sự đánh đổi được chấp nhận, nói rõ thành lời. Production build image mà không cần daemon nào cả — Kaniko hoặc BuildKit chạy như một pod không đặc quyền, hoặc Jib cho JVM — và việc chuyển Jenkins vào cụm ở Chương 27 là nơi khoản thỏa hiệp này sẽ được trả lại.

TRÊN PHẦN CỨNG NÀY

Pod frontend rẻ nhất namespace (yêu cầu 32 Mi), nhưng *build* thì không: npm ci cộng tsc cộng Vite chạy gần một CPU-phút và một gigabyte RAM trên controller, trong khi Maven có thể vẫn đang chạy trong cùng container 2 GB đó. Jenkinsfile giữ các stage tuần tự vì lý do đó; trên cái máy này, stage song song chỉ tranh giành nhau.

26.7 Kiểm chứng cột mốc

THỰC HÀNH — DANH SÁCH KIỂM TRA CỘT MỐC 7

1. `kubectl -n ts get ingress` — hai object trên `ts.local`; `kubectl -n ts get pods -l app=frontend` Ready.
2. Mở `http://ts.local/` — trang đăng nhập. Rồi dán thẳng `http://ts.local/courses` vào thanh địa chỉ: một trang, không phải 404 của nginx — fallback SPA hoạt động.
3. Đăng ký, đăng nhập, tạo một khóa học: tab network của trình duyệt cho thấy các request `/api/...` không có preflight OPTIONS nào — cùng origin.
4. `curl -sI http://ts.local/assets/index-*.js | grep Cache-Control` — `immutable`; làm tương tự với `/` — không có header cache.
5. Đổi một đoạn chữ bất kỳ trong một trang, push, và xem pipeline: stage Frontend image dùng lại layer npm ci, và rollout thay vào một image 15 MB.

Build #9 là lần chạy đầu tiên của stage mới: `docker build` hoàn tất trong 90 giây trên controller, rollout thay frontend:f5d9213 vào, và danh sách kiểm tra ở trên đạt đúng như đã viết — `/courses` trả về `index.html`, `/api/courses` trả về JSON 401 của gateway, asset mang `immutable` và `/` không mang header cache nào.

CẠM BẦY — INGRESS SỐNG LÂU HƠN MANIFEST CỦA NÓ

kubect1 get ingress hôm đó liệt kê *ba* object trên ts.local: của gateway, của frontend — và course-service, tám ngày tuổi, định tuyến /courses thẳng tới service mà không kiểm tra JWT. Manifest của nó đã bị gỡ khỏi course-service.yaml từ Chương 18 kèm một comment nói rõ điều đó, nhưng quy tắc của Chương 20 vẫn đúng: apply không bao giờ xóa thứ nó không còn thấy. Phải cần một lệnh tay kubect1 -n ts delete ingress course-service. Bài học không phải là câu lệnh mà là việc kiểm toán: sau mỗi cột mốc, hãy liệt kê thứ cụm *có*, không chỉ thứ *git nói* — cho đến khi việc dọn (prune) của GitOps (Chương 27) biến hai thứ đó thành một.

26.8 Truy cập từ bất cứ đâu: tên trên tailnet

Mọi thứ ở trên trả lời trên ts.local — một cái tên chỉ tồn tại trong file hosts của một chiếc laptop, trên một LAN. “Live” nghĩa là một URL phân giải được trên mọi thiết bị bạn có, và cái máy đã có sẵn một cái: Tailscale cho mỗi node một tên MagicDNS, ts-server.tailbfe002.ts.net, mà mọi peer trên tailnet phân giải được mà không cần sửa file hosts. Traefik định tuyến theo header Host (Chương 18), nên một cái tên nó không liệt kê sẽ nhận 404 dù gói tin có tới nơi. Cách sửa là thêm một mục rules nữa với cùng danh sách đường dẫn trong cả hai object Ingress (Phụ lục D có chúng) — apply bằng tay với một bộ lọc chỉ trích ra các tài liệu Ingress từ đầu ra của kubect1 kustomize, vì một lệnh apply -k đầy đủ sẽ đặt lại tag image của mọi Deployment về 1.0.0-SNAPSHOT của manifest và roll cả namespace (set image của Chương 23 nằm *sau* apply chính vì lý do này).

Cái tên đó rồi phơi ra một lỗi mà LAN đã che giấu. Một lần đăng ký qua host mới sinh ra mail kích hoạt (chuỗi của Chương 25, tròn vện qua tailnet), nhưng link bên trong ghi http://localhost:8080/api/auth/activate?code=...: một host không điện thoại nào tới được, và một endpoint POST mà GET của trình đọc mail đăng nào cũng không dùng nổi. EmailService dựng mọi link — kích hoạt, mời, đăng nhập — từ chuỗi literal. Sửa chữa là quy tắc của Chương 16 áp dụng cho URL: origin công khai là *cấu hình*. app.frontend.base-url mặc định là dev server của Vite trong notification-service.yml của config-server; manifest ghi đề nó bằng APP_FRONTEND_BASE_URL đặt thành tên tailnet, cùng thủ thuật relaxed binding như SPRING_MAIL_HOST; và các link giờ trở tới trang /activate của frontend, nơi thực hiện POST. Hai biến OAuth chuyển sang cùng host, đồng nghĩa với thêm một mục nữa dưới “Authorized redirect URIs” của Google — việc đăng ký mà Chương 5 gọi là thứ duy nhất không manifest nào tự động hóa được.

CẠM BẦY — MỘT LINK LÀ MỘT HỢP ĐỒNG VỚI BÊN NGOÀI

Bất cứ thứ gì rời khỏi hệ thống — một email, một payload webhook, một mã QR — đều mang một URL phải hoạt động từ *bên ngoài*, nên nó không bao giờ được là tên Service và không bao giờ nên là literal. Bài kiểm tra rất rẻ: đăng ký từ một thiết bị không phải laptop build và bấm vào. Trên stack này, chính request đầu tiên qua hostname mới đã phơi ra một lỗi có mặt từ commit đầu tiên.

26.9 Ngoài dự án

Mẫu hình — router biên tách tĩnh khỏi API theo đường dẫn, phần tĩnh đến từ một file server ngớ ngếch — là phổ quát, với file server thường được thay bằng lưu trữ đối tượng cộng CDN (S3 + CloudFront, hoặc một static host như Netlify) và việc tách do quy tắc origin của CDN làm. Giai đoạn build vẫn y nguyên nửa đầu của Dockerfile này. Hai thói quen chuyển sang trực tiếp: bấm-và-cache-vĩnh-viễn cho asset kèm `index.html` không cache, và fallback SPA — theo cách nói của CloudFront là “ánh xạ 403/404 sang `/index.html` với mã 200”, cùng ý tưởng nhưng dùng vùng hơn.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Vì sao image frontend có hai dòng `FROM`, và chính xác thứ gì băng qua giữa chúng?
2. Một người dùng bookmark `http://ts.local/courses/7` và mở nó sáng hôm sau. Lần theo request qua Traefik và nginx, và nói dòng nào khiến nó thành công.
3. Cả hai object Ingress đều khai báo host: `ts.local`. Traefik quyết định thế nào để `/api/courses` đi tới gateway và `/assets/x.js` đi tới nginx?
4. Giải thích vì sao `CorsConfig` của gateway trở nên không liên quan trong cụm trong khi vẫn cần thiết trên laptop.
5. `--group-add "$(stat -c %g /var/run/docker.sock)"` sửa điều gì, và vì sao giá trị được tính ra thay vì viết sẵn?
6. Link trong mail kích hoạt đã sai hàng tháng mà không ai để ý. Nêu hai khiếm khuyết riêng biệt trong `http://localhost:8080/api/auth/activate?code=...`, và nói mỗi cái sẽ hỏng ở môi trường nào (laptop, compose, cụm).

Chặng đường phía trước

Với frontend được phục vụ từ cụm (Chương 26), mọi thứ có thể triển khai của dự án đều đã được triển khai và ứng dụng dùng được trong trình duyệt. Các cột mốc còn lại, mỗi cái thay một thứ tự chế bằng cơ chế công nghiệp — và mỗi cái đều có ý nghĩa nhất vào lúc này, khi bạn đã biết nó thay thế cái gì.

27.1 Khả năng quan sát

Bên ngoài `kubectl logs` là bộ ba tiêu chuẩn: **metric** (Prometheus cào các endpoint /actuator/prometheus mà mọi service đã mở sẵn; dashboard Grafana trên bộ ba RED — rate, errors, duration), **trace** (dây nối Zipkin đã có trong mọi cấu hình, hiện lấy mẫu ở mức không trong cụm — một biến môi trường bật nó lên khi có pod Zipkin), và **alert** (Alertmanager: một cú báo khi tỷ lệ lỗi của một service hay độ đầy của một PVC vượt ngưỡng). Việc này đến qua Helm chart `kube-prometheus-stack` — khoảnh khắc đã hứa khi bạn dùng Helm mà không tự viết chart (hộp quyết định của Chương 20, được giữ lời). Strimzi có thể xuất metric của broker vào cùng Prometheus đó với vài dòng `metricsConfig` trên resource Kafka — độ trễ của consumer group là alert Kafka đầu tiên đáng có.

TRÊN PHẦN CỨNG NÀY

Bộ đầy đủ tốn ~1,5 GB và không ít CPU — ngân sách nói là vừa, nhưng đây là thứ đầu tiên phải cắt khi tải cao. Khả năng quan sát có thể bỏ đi là món xa xỉ của homelab; các đội production tính kích cỡ cho nó trước tiên.

27.2 GitOps: Argo CD

Pipeline *đẩy* các lần triển khai (Jenkins giữ một `kubeconfig` và chạy `kubectl`). GitOps đảo chiều mũi tên: một agent trong cụm (Argo CD) *kéo* — liên tục so sánh manifest của một kho git với cụm và hội tụ. Điều đó mua được gì, bằng vốn từ bạn đã có: thông tin xác thực của cụm không bao giờ rời khỏi cụm (nỗi lo RBAC của Chương 22 tan biến — CI chỉ push image và commit một lần nâng tag); việc xóa cuối cùng cũng lan truyền (cạm bẫy `prune` của Chương 20, được giải quyết từ thiết kế); trôi (drift) cấu hình trở nên hữu hình (một Deployment bị sửa tay

hiện “OutOfSync”); và rollback là `git revert`. Jenkins giữ CI — build, test, push, cập nhật tag trong git — Argo CD nhận CD. Stage deploy của Jenkinsfile, tức Chương 23 của cuốn sách này, trở thành một commit.

27.3 Cho stack cổ điển về hưu

Sự dư thừa có chủ đích của Chương 2 và 3 nay có thể được thanh toán, từng service một:

- **Bỏ Eureka:** đổi các route của gateway từ `lb://course-service` sang `http://course-service:8082` — DNS của Service và các endpoint gác bằng readiness làm mọi thứ registry từng làm ở đây. Xóa một thứ triển khai, giải phóng 512 Mi.
- **Bỏ Config-server:** mount các file `configurations/` thành một ConfigMap và trở `spring.config.import` vào thư mục đã mount (`optional:file:/config/`). Cùng file, bớt một service, và thay đổi cấu hình trở thành thay đổi manifest chảy qua GitOps.

Cả hai cuộc di dời đều là điểm cộng cho CV chính vì chúng chạy *về phía* nền tảng: biết vì sao Eureka tồn tại *và* vì sao Kubernetes khiến nó lỗi thời là khác biệt giữa dùng một trong hai và hiểu cả hai.

27.4 Jenkins vào cụm

Phép đảo ngược kiểu homelab của Chương 22 — build trên controller, Jenkins nằm ngoài k3s — đảo lại gọn gàng một khi pipeline đã ổn định: plugin Kubernetes sinh một pod agent cho mỗi build, và bản thân Jenkins trở thành một StatefulSet với `JENKINS_HOME` trên một PVC. Cuộc di dời chạm tới mọi khái niệm của Phần V cùng lúc, đó là lý do nó được xếp cuối cùng.

27.5 OpenShift, nói ngắn

Các manifest được giữ khả chuyển có chủ đích: image không chạy bằng root của Jib (Chương 13) thỏa mãn mô hình bảo mật UID tùy ý của OpenShift, nên triển khai lên một sandbox OpenShift miễn phí hẳn chỉ cần bước dịch Ingress sang Route (Chương 18). Chạy thí nghiệm đó một lần sẽ kiểm chứng những tuyên bố về tính khả chuyển mà cuốn sách này vẫn đưa ra — và vốn từ của OpenShift (Route, SCC, BuildConfig) ánh xạ một-một lên các chương bạn đã đọc.

27.6 Bạn đang đứng ở đâu

Bỏ tên dự án đi, bản kiểm kê đọc như một bản mô tả công việc platform: bộ máy phân giải thuộc tính giúp một artifact chạy ở bất cứ đâu; log, consumer group và ngữ nghĩa giao nhận; image, layer và registry; điều hòa, probe, RBAC, storage class và operator; pipeline, danh tính deploy giới hạn bằng RBAC, và phép đảo chiều GitOps. Những ràng buộc của homelab không phải là giới hạn — chúng là giáo trình: mỗi hộp “trên phần cứng này” đều ép ra một quyết định mà tiền bạc thường che đi.

TỰ KIỂM TRA (đáp án: Phụ lục A)

1. Bạn sẽ đặt alert trên metric nào trước tiên cho đường Kafka, và nó đo cái gì theo cách nói của Chương 10?
2. Triển khai kiểu đẩy so với kiểu kéo: chỉ ra thông tin xác thực của cụm nằm ở đâu trong mỗi kiểu, và giải thích việc dọn (prune) của GitOps giải quyết cạm bẫy của chương nào.
3. Phác thảo việc cho Eureka về hưu: cái gì đổi trong cấu hình của gateway, hai cơ chế Kubernetes nào thay cho đăng ký và liveness, và cái gì có thể xóa?
4. Vì sao image của Jib khiến thí nghiệm OpenShift rẻ, và phép dịch object nào vẫn còn lại?

Đáp án tự kiểm tra

Chương 1 — Spring Boot

1. Biến môi trường thẳng: biến môi trường đứng trên tài liệu của config-server, và tài liệu đó đứng trên YAML đóng gói trong jar. Đúng với container vì image phải bất biến giữa các môi trường; những gì khác nhau do runtime bơm vào.
2. @Configuration, @ComponentScan, @EnableAutoConfiguration. Cái cuối cùng: auto-configuration phụ thuộc vào classpath, nên gỡ một dependency sẽ âm thầm gỡ luôn các bean mà nó kích hoạt.
3. Nó được giải thành chuỗi ký tự placeholder theo đúng nghĩa đen. Vì thế service vẫn *khởi động* được dù không có thông tin xác thực Google — OAuth chỉ hỏng lúc người dùng bấm, chứ không hỏng lúc boot.
4. Việc mở (exposure) khiến endpoint tồn tại; Spring Security vẫn phân quyền từng request. Nếu /actuator/** không được cho phép, probe nhận 401, và kubelet tính đó là thất bại.

Chương 2 — Config Server

1. Biến môi trường > config-server > YAML đóng gói.
2. Native phục vụ file từ thư mục classpath bên trong jar; git thêm khả năng kiểm toán, review, hoàn tác, và thay đổi cấu hình mà không cần build lại config-server.
3. optional: cho phép service boot khi config-server sập — nhưng khi đó nó boot với các giá trị mặc định localhost và hỏng về sau, ở rất xa nguyên nhân.
4. Trong application.yml của chính nó, đóng gói trong jar của nó. Nguyên tắc bootstrap: nguồn cấu hình không thể tự cấu hình cho mình; mọi chuỗi như vậy đều cần một gốc tự chứa.

Chương 3 — Eureka

1. course-service đăng ký (POST) lúc khởi động; discovery client của gateway làm mới cache registry của nó (mặc định tối đa 30 s); rồi việc định tuyến cân

-
- bằng tải từ cache. Trường hợp xấu nhất để một phiên bản mới nhận được lưu lượng: khoảng đăng ký + làm mới cache, tức hàng chục giây.
2. Compose khiến *tên* service phân giải được, và hostname trùng với tên đó; trong Kubernetes hostname của pod là tên pod, thứ không ai phân giải được — nên phải đăng ký IP thay vào đó.
 3. Ưu tiên tính sẵn sàng của dữ liệu registry hơn tính chính xác. Có lợi: một switch khởi động lại làm im bật mọi heartbeat nhưng các phiên bản vẫn đang chạy. Có hại: sập thật hàng loạt để lại các phiên bản đã chết vẫn nằm trong registry.
 4. Danh sách endpoint của Service thay cho registry; readiness probe thay cho heartbeat; kube-proxy/DNS thay cho tra cứu phía client; ReplicaSet thay cho việc phiên bản mới tự đăng ký.

Chương 4 — API Gateway

1. Trình duyệt → ts.local (file hosts) → Traefik (Ingress: /api → Service api-gateway) → pod gateway (predicate /api/auth/**, StripPrefix=2, không đòi JWT với /api/auth) → pod auth-service do Eureka phân giải, đến nơi dưới dạng /login → controller.
2. Google kiểm tra redirect URI từng byte một; nó đã được đăng ký với đường dẫn đầy đủ /login/oauth2/..., nên phải đến nơi nguyên vẹn. Các route API thì cắt tiền tố vì controller được viết không có tiền tố.
3. Mọi request mang token do auth-service ký đều trượt bước kiểm tra của gateway — 401 đồng loạt trên các route được bảo vệ trong khi bản thân việc đăng nhập có vẻ vẫn chạy.
4. Proxy là bài toán I/O với độ đồng thời cao — event loop rất hợp. course-service làm việc JDBC kiểu blocking; reactive sẽ làm nó phức tạp mà không có lợi ích gì cho đến khi chứng minh được điều ngược lại.

Chương 5 — Bảo mật

1. CSRF bám vào một thông tin xác thực có sẵn trong môi trường (cookie phiên được gửi tự động). JWT trong header Authorization được code gắn vào một cách tường minh; một request giả mạo xuyên site không thể thêm nó vào.
2. Mười lăm phút (thời hạn của nó). Không gì cả — chữ ký không thể thu hồi. /refresh: refresh token được đối chiếu với cơ sở dữ liệu, nơi việc thu hồi được lưu.

3. Bất biến: mọi đường mạng đến một service đều đi qua filter JWT của gateway. Bị phá khi: một Ingress định tuyến thẳng đến service (route /courses đã xóa), hoặc bất kỳ bên gọi nào trong cụm giả mạo header trên mạng pod phẳng.
4. Client-secret không bao giờ rời khỏi server; authorization code thì đi qua trình duyệt. Code chỉ dùng một lần, sống ngắn, và vô giá trị nếu không có client-secret lúc đổi lấy token.
5. HS256: một secret dùng chung; ai kiểm tra được thì cũng ký được. RS256: khóa riêng ký, khóa công khai kiểm tra — nên một IdP có thể cho hàng nghìn service kiểm tra token mà không service nào trong số đó đúc được token.

Chương 6 — Các service và module common

1. (a) HTTP — bên gọi cần dữ liệu ngay. (b) Sự kiện — một sự thật mà bên khác phản ứng theo; việc chấm điểm không được chờ mail. (c) Không cái nào: kiểm tra token là phép mật mã cục bộ với secret dùng chung; không cần gọi đi đâu.
2. auth-service chỉ *sản xuất* — từng lần gửi hổng riêng lẻ, boot vẫn thành công. notification-service chỉ *tiêu thụ*; không có broker thì nó chỉ là chi phí đứng không, nên nó đi kèm Kafka.
3. Thuộc về common: các record sự kiện — hai bên phải thống nhất hình dạng. Không bao giờ: logic nghiệp vụ/entity — hành vi dùng chung trói chu kỳ phát hành của các service vào nhau và che mất ranh giới.
4. Consumer triển khai trước producer, và thay đổi chỉ được thêm vào (trường mới, tùy chọn). Khi đó consumer cũ bỏ qua trường mới và consumer mới chịu được sự kiện cũ.

Chương 7 — Postgres, JPA, Flyway

1. Schema của hai lập trình viên âm thầm lệch nhau (update áp dụng theo từng cơ sở dữ liệu, không có lịch sử); và thay đổi phá hủy là bất khả thi — update không bao giờ drop/đổi tên, nên refactor entity lạng lẽ lệch khỏi schema.
2. Mỗi dòng lịch sử lưu checksum của script tương ứng; script bị sửa sẽ lệch checksum và Flyway từ chối khởi động — lịch sử là dấu vết kiểm toán, và việc sửa sẽ làm giả nó.
3. Phát hành 1: thêm cột mới, ghi cả hai (hoặc dùng trigger). Migration backfill.

Phát hành 2: đọc cột mới, ngừng ghi cột cũ. Phát hành 3: drop cột cũ. Mỗi bước tương thích với phiên bản ngay trước nó.

4. Ứng hộ: cô lập lỗi và phạm vi ảnh hưởng — một server sập, nâng cấp, hay cấu hình sai không thể chạm tới dữ liệu của server kia. Phản đối: gấp đôi bộ nhớ và bề mặt vận hành với một homelab. Cloud có quản lý thường chọn cô lập bằng thông tin xác thực trên một instance vì server là đơn vị tính tiền.

Chương 8 — MongoDB

1. Mục từ điển: tự chứa, đọc/ghi như một khối, lồng nhau tự nhiên, không cần nhất quán giữa các mục. Enrollment: tham chiếu hai aggregate khác và đòi hỏi nhất quán giữa chúng — join và transaction chính là điểm mấu chốt.
2. Vào code ứng dụng (và validator của collection nếu muốn). Kỷ luật: lớp document đọc ngược được (trường mới là tùy chọn), và script sửa dữ liệu tường minh khi đổi hình dạng.
3. Postgres từ chối kết nối đến cơ sở dữ liệu không tồn tại; Mongo tạo cơ sở dữ liệu ở lần ghi đầu tiên, nên tên sai cho ra kết quả rỗng.Ồn ào tốt hơn — lỗi bạn thấy lúc triển khai hơn hẳn lỗi bạn thấy lúc kiểm toán.
4. Chỉ ClusterIP + không có Ingress nghĩa là chỉ bên gọi trong cụm mới chạm tới được; sự tương đồng với compose cũng giữ ma sát khi dev bằng không. Mất hiệu lực ngay khi nó bị lộ ra ngoài cụm hoặc dùng chung cụm với workload không đáng tin.

Chương 9 — MinIO / S3

1. Service ký (micro giây) thay vì stream (giây); storage phục vụ byte trực tiếp. Kiểu lỗi mới: URL nhúng sẵn endpoint và thời hạn — endpoint sai hay lệch đồng hồ làm hỏng tải xuống chỉ ở trình duyệt, service không nhìn thấy.
2. Service ký chạm tới MinIO qua `minio:9000` (DNS của cụm); trình duyệt không phân giải được tên đó. Chữ ký bao trùm cả host, nên viết lại sau khi ký sẽ làm chữ ký vô hiệu — vì thế phải ký với một endpoint thứ hai, công khai.
3. Thời hạn (15 phút) giới hạn cửa sổ thiệt hại; URL chỉ cấp một object, không phải cả bucket; và không thể ký lại nếu không có khóa.
4. Thay đổi: endpoint, thông tin xác thực (lý tưởng là chuyển sang IAM role), bảo đảm về độ bền. Không đổi: mọi lời gọi SDK, logic bucket/key/presign — API chính là chuẩn.

Chương 10 — Kafka cốt lõi

1. Key theo `userId` — thứ tự theo key chính là thứ tự theo partition. Số partition $\geq$ số consumer tối đa bạn từng muốn có trong group; độ song song bị chặn trên bởi số partition.
2. Các group độc lập nhau: mỗi group giữ offset riêng cho từng partition, nên cả hai group đều duyệt hết toàn bộ log; tiêu thụ là đọc không phá hủy cộng với một con trỏ theo từng group.
3. `LISTENERS` = socket được bind; `ADVERTISED_LISTENERS` = địa chỉ trả về cho client trong metadata. Bootstrap dùng địa chỉ bạn cung cấp; mọi kết nối sau đó dùng địa chỉ được quảng bá — nên lỗi chỉ bắt đầu sau một cái bắt tay thành công.
4. Retry và sập sau khi xử lý đều gây gửi lại; broker không thể biết bạn đã hành động chưa. Notification: khứ trùng lặp theo id sự kiện (hoặc chấp nhận một mail trùng hiểm hoi — lũy đẳng bằng cách chịu đựng).

Chương 11 — Spring Kafka

1. `send()` đệm ở phía client và trả về một future; không cần broker cho đến lúc giao. Sự cố lộ ra theo từng lần gửi: future thất bại sau timeout, và sự kiện đang đệm mất trắng nếu tiến trình chết.
2. Khi chưa có offset đã commit, `earliest` bắt đầu từ record cũ nhất còn được giữ — lịch sử được xử lý lại. Một khi group đã commit offset, thiết lập này không bao giờ được hỏi đến nữa.
3. Không có: giải tuần tự ném lỗi ngay trong poll, offset không bao giờ tiến, partition kẹt mãi ở một record. Có: lỗi được vật chất hóa thành một thất bại có kiểu, error handler bỏ qua (hoặc retry rồi chuyển tiếp) — ở production đầy đủ, chuyển đến `<topic>.DLT`.
4. Giải tuần tự JSON khởi tạo các lớp được nêu tên trong header; danh sách tin cậy không giới hạn cho phép một message bắt consumer dựng lên kiểu bất kỳ trên classpath — một vector tiêm gadget.
5. Khi “ghi DB + phát sự kiện” không được phép lệch nhau. Nó làm cho thay đổi trạng thái và việc ghi lại ý định phát trở thành nguyên tử — một transaction cục bộ; một relay đảm bảo việc phát cuối cùng sẽ diễn ra.

Chương 12 — Docker

1. Namespace (góc nhìn cô lập) và cgroup (giới hạn tài nguyên). Không có kernel khách: khởi động tính bằng mili giây và không tốn chi phí hệ điều hành cho từng instance — nhưng đồng thời: kernel dùng chung (cô lập yếu hơn VM) và JVM nhìn thấy tài nguyên của host trừ khi được bảo khác đi.
2. Bên trong container, `localhost` là chính container đó. Compose sửa bằng DNS theo tên service trên mạng bridge; Kubernetes bằng DNS của Service — cả hai qua biến môi trường ghi đè các giá trị mặc định `localhost`.
3. Mỗi container có network namespace riêng, nên cả hai đều bind cổng 5432 “của mình”. Xung đột quay lại ở host dùng chung: cổng host được publish (vì thế mới có 5433/5434) — và với `--network host`, nơi không hề có sự tách biệt namespace.
4. `depends_on` thường chờ *đã khởi động*; `service_healthy` chờ healthcheck vượt qua — ở đây là config-server tự `curl /actuator/health` của chính nó, được mở bởi cấu hình management.

Chương 13 — Jib và Registry

1. Không cần daemon trong CI, chia layer theo độ biến động (push lại nhỏ), image không chạy bằng root và sẵn sàng cho OpenShift. Về bảo mật: không có daemon nghĩa là không có docker socket trong CI — socket đó là root trên host.
2. Tên image đồng thời là địa chỉ pull cho cụm. Ở đây chúng trùng nhau vì Jenkins dùng chung network namespace với host; nếu Jenkins chạy trên mạng bridge, `localhost:5000` của nó sẽ là chính nó và push hỏng trong khi pull vẫn chạy.
3. Mỗi lần deploy phải import trên từng node, mất hàng phút tuần tự hóa, và không có lịch sử tag bất biến; lệnh `set image` của pipeline trở đến tên gắn tag SHA sẽ không có gì để trở vào — registry chính là thứ làm cho tag có địa chỉ.
4. Cùng tag, nội dung mới: với `IfNotPresent` image đã cache trên node âm thầm thắng. Cách sửa: `imagePullPolicy: Always` (lựa chọn của manifest), hoặc deploy bằng tag/digest bất biến (lựa chọn của pipeline) — dự án này làm cả hai.

Chương 14 — Giải phẫu cụm

1. Bạn viết trạng thái mong muốn; controller điều hòa trạng thái thực tế về phía đó mãi mãi. Apply lần thứ hai không tạo ra diff, nên không có hành động — lũy đẳng nhờ kiến trúc.
2. Thứ tự là khái niệm thủ tục; hội tụ thay thế nó: mọi thứ cùng khởi động, bên phụ thuộc sập rồi thử lại với backoff cho đến khi thứ nó cần tồn tại. CrashLoop trong lúc hội tụ là cơ chế, không phải bug.
3. kubectl gửi một lệnh REST delete đến API server; kho trạng thái cập nhật; vòng lặp của controller ReplicaSet thấy replica $0 < 1$ và tạo một object pod; scheduler gán nó; kubelet trên node khởi động container.
4. Apply hợp nhất trạng thái mong muốn; nó không bao giờ xóa những gì vắng mặt trong đầu vào. kubectl delete ingress course-service -n ts, hoặc apply với --prune (hoặc một controller GitOps) thì sẽ xóa.

Chương 15 — Workload

1. Mỗi thay đổi template sinh ra một ReplicaSet mới trong khi các ReplicaSet cũ được giữ ở scale 0 — cập nhật cuốn chiếu và rollback một lệnh là hệ quả của việc tách bộ đếm khỏi template có phiên bản.
2. Deployment: N replica thay thế được cho nhau, cái nào chết cũng được, thứ tự không quan trọng. StatefulSet: replica có tên với storage theo từng danh tính và thay thế có thứ tự. Eureka: Deployment (registry được dựng lại nhờ đăng ký lại). Redis cache: Deployment nếu thật sự chỉ là cache. Broker Kafka: StatefulSet. Gateway: Deployment.
3. Pod mới phải vượt qua readiness trước khi pod cũ bị kết thúc — probe của Chương 17 là cổng gác cho rollout của Chương 15.
4. Cập nhật Deployment chạy pod cũ và mới đồng thời; hai tiến trình postgres cùng mở một thư mục dữ liệu sẽ làm hỏng nó. StatefulSet thay thế từng cái một và gán lại claim cho pod kế nhiệm, không bao giờ hai cái cùng lúc.

Chương 16 — Cấu hình và Secret

1. RBAC về việc ai được đọc object Secret; tách vòng đời khỏi git; mã hóa khi lưu (tùy chọn). Dự án này: RBAC có (Role của Jenkins bỏ qua quyền đọc secret... nó cấp create/patch nhưng kiểm soát thực tế là giới hạn theo namespace), tách khỏi git có (script), mã hóa khi lưu không.

-
2. Image server đọc `POSTGRES_*`; Spring đọc `SPRING_DATASOURCE_*`. Tách Secret nghĩa là mỗi pod chỉ nhập những tên nó hiểu — không có biến lạ che mắt lỗi gõ hay rò rỉ phạm vi.
 3. Cập nhật: `JWT_SECRET=new ./create-secrets.sh`; rồi ngay lập tức `kubectl -n ts rollout restart deployment/auth-service deployment/api-gateway`. Làm nửa chừng: bên ký và bên kiểm tra giữ khóa khác nhau — mọi lần đăng nhập đều 401, hoặc tệ hơn, chỉ sau lần restart không liên quan tiếp theo.
 4. Copy `configurations/*.yaml` của `config-server`, mount dưới dạng file — và cho `config-server` nghỉ hưu luôn.

Chương 17 — Probe và tài nguyên

1. Startup thất bại quá ngân sách: restart. Readiness thất bại: bị gỡ khỏi endpoint, không restart — có thể mạnh tay. Liveness thất bại: restart — cái này phải dè dặt.
2. `/v3/api-docs` chỉ trả lời sau khi toàn bộ context (kể cả Flyway) đã lên và được `permitAll`; một cổng đã bind chỉ chứng minh Tomcat tồn tại. `/actuator/health` trả 401 ở nơi security không cho phép (`course/auth`) và trung thực ở nơi cho phép (`dictionary, gateway`).
3. OOM killer của kernel đã kết thúc cgroup vì vượt giới hạn bộ nhớ — JVM không hề tham gia, nên không có exception. Kiểm tra: phép tính giữa `limit` và `MaxRAMPercentage`, và xem tải hay rò rỉ đã làm heap phình lên.
4. Scheduler đọc requests (để xếp chỗ); kernel thực thi limits. Bộ nhớ không nén được — vượt là có kẻ bị giết, nên hãy chặn nó một cách dự đoán được; CPU nén được — limit sẽ bóp nghẹt lúc khởi động đúng khi JVM cần bứt tốc.

Chương 18 — Mạng

1. DNS phân giải tên Service thành một IP ảo ổn định; endpoint controller giữ danh sách pod Ready luôn cập nhật; kube-proxy (hoặc thứ tương đương của k3s) viết lại lưu lượng đến IP ảo sang IP của một pod được chọn; readiness quyết định tư cách thành viên.
2. Chỉ pod Ready mới là endpoint, nên cập nhật cuốn chiếu chuyển lưu lượng đúng lúc pod mới vượt probe và pod cũ kết thúc — không gián đoạn chính là tương tác giữa probe và endpoint.
3. Selector/label lệch nhau — Service khớp với không pod nào. Không có gì báo lỗi vì Service không có endpoint là một trạng thái hợp lệ; triệu chứng

chỉ nhìn thấy trong `get endpoints`.

4. `ts.local` không phải DNS thật; file hosts ghim nó vào IP của tailnet. Thay thế: một domain thật với DNS thực + TLS bằng `cert-manager`, hoặc một Cloudflare Tunnel công bố hostname đó.

Chương 19 — Lưu trữ

1. Pod mount PVC; PV (và StorageClass) có phạm vi toàn cụm; lớp gián tiếp này cho phép cùng một claim nhận local-path ở đây và EBS trên EKS — manifest không đổi.
2. Postgres chạy trên một node tại một thời điểm — RWO là đủ. Upload dùng chung đẩy người ta sang RWX; lưu trữ đối tượng thường mới là câu trả lời đúng.
3. Xóa StatefulSet không đụng đến PVC nên dữ liệu sống sót qua sai lầm điều phối; xóa PVC (`reclaim Delete`) gỡ luôn PV và dữ liệu — dữ liệu sống ở claim, không phải ở workload.
4. Bền vững (persistence): sống qua restart (có). Độ bền (durability): sống qua hỏng phần cứng (không — một SSD). Sao lưu: sống qua sai lầm của chính bạn (không — chưa có, khoảng trống thành thật).

Chương 20 — Kustomize

1. Một tập hợp chuẩn (không sót gì, một lệnh) và các chỉnh sửa xuyên suốt (namespace, label) được đóng dấu nhất quán.
2. Namespace: phạm vi toàn cụm — Role theo namespace của Jenkins không thể apply nó. Secret: giá trị tuyệt đối không được tồn tại trong git — hình dạng nằm trong script, giá trị nằm trong môi trường của người vận hành.
3. Kustomize: triển khai ứng dụng của chính bạn dưới dạng YAML dễ đọc. Helm: phân phối phần mềm có tham số đến cụm của người khác. Đến dưới dạng chart: Strimzi và kube-prometheus-stack.
4. Không gì cả — apply không dọn; mongodb vẫn chạy, giờ không còn ai quản lý. Đúng cách: `kubectl delete -f mongodb.yaml` (rồi gỡ khỏi danh sách), hoặc cơ chế prune/GitOps điều hòa cả việc xóa.

Chương 21 — Khái niệm CI/CD

1. CI: main luôn được xác nhận là build được — stage Build. CD: mọi thay đổi đã xác nhận tự đến production không cần ai đỡ — stage Deploy cộng với

cổng gác rollout của nó.

2. Jenkins nằm sau NAT, không có địa chỉ công khai — GitHub không gọi vào được, nên Jenkins phải poll. Cùng ràng buộc: CI bên trong mạng doanh nghiệp mà git host trên cloud không chạm tới được.
3. Build lại là một binary khác — phân giải dependency, timestamp, toolchain khác — nên test đã chứng nhận một thứ khác. Thứ tự ưu tiên biến môi trường của Chương 1 cho phép một artifact nhận cấu hình theo môi trường lúc chạy.
4. Test bị bỏ qua (bật lại, hoặc gác ở chỗ khác và biết rõ điều đó); deploy không được xác minh (rollout status/smoke test); thành công được định nghĩa bằng exit code chứ không bằng hành vi (xác minh dựa trên probe, alert).

Chương 22 — Jenkins

1. Job, credential, trạng thái plugin, lịch sử build. Image = binary+plugin tái tạo được; volume = trạng thái không thay thế được; gộp lại, server là gia súc với một thư mục thú cưng.
2. Jenkins chạy trên JDK21 của image gốc; build export `JAVA_HOME=JDK25_HOME` trong khối environment của Jenkinsfile — theo từng pipeline, không phải toàn cục.
3. apply -k cần get/list + create/patch/update trên mọi kind được apply; rollout status cần get/list/watch trên deployment và replicaset. Bỏ được nếu không có set image: không gì mới — get+patch của set image đã được bao phủ sẵn; các quy tắc đọc pods/log chỉ tồn tại cho trình xử lý thất bại.
4. Jib push đến localhost:5000, và tên đó phải chỉ cùng một máy với cả bên push lẫn bên pull; host networking khiến localhost của Jenkins chính là localhost của host, đúng như tên image đã hứa với k3s.
5. ServiceAccount = danh tính; Role = danh sách năng lực; RoleBinding = mối nối. Role chứ không phải ClusterRole giới hạn một pipeline bị xâm nhập trong một namespace — phạm vi ảnh hưởng như một lựa chọn về kiểu.

Chương 23 — Pipeline

1. SNAPSHOT là khả biến — một node có thể phục vụ bất kỳ build lịch sử nào dưới cùng một tên; SHA ghim đúng commit và làm cho câu hỏi “cái gì đang chạy?” trả lời được từ `kubectl describe`.
2. Chỉ build các module được liệt kê cộng với dependency của chúng. Quên một mục MODULES: service mới không bao giờ có image; vòng lặp deploy

sau đó thất bại (hoặc deploy một image cũ nếu có) — điểm ghép nối thủ công duy nhất của pipeline.

3. `kubectl rollout status --timeout=300s` — nó chuyển kết quả probe (Chương 17) thành exit code của pipeline.
4. Mọi rollout nên tiến hành đồng thời; chờ theo từng service tuần tự hóa sự hội tụ mà Kubernetes sẵn sàng làm song song — cùng tổng công việc, thời gian thực dài hơn.
5. Thay đổi: cách kích hoạt (webhook), nơi build chạy (runner tạm thời), cơ chế secret (secret của repo). Không đổi: thứ tự stage, lệnh Maven, gắn tag SHA, bước xác minh rollout.

Chương 24 — Vận hành

1. `get pods` → `describe pod` → `logs` (`--previous` — thiết yếu với CrashLoop, khi container hiện tại chưa có lịch sử gì) → exec vào và thăm dò từ bên trong.
2. CrashLoop: tiến trình thoát — tăng ứng dụng. `Running 0/1`: tiến trình sống nhưng readiness thất bại — đích của probe (thường là một dependency) mới là tăng cần xem.
3. Exec vào pod và thử kết nối từ đó — DNS, đường mạng, và biến môi trường được bơm vào của pod khác với laptop của bạn; laptop chỉ chứng minh Postgres còn sống, không chứng minh pod chạm tới được nó.
4. Lưu lượng pod/service của k3s đi qua firewall của host; chặn `10.42.0.0/16` hoặc `10.43.0.0/16` làm hỏng DNS và các lời gọi pod-đến-pod một cách chập chờn — triệu chứng không phân biệt được với ứng dụng thiếu ổn định.

Chương 25 — Kafka trong cụm

1. Một controller có vòng lặp điều hòa nhắm đến các kind tài nguyên tùy chỉnh do CRD định nghĩa. Strimzi thêm `Kafka/KafkaNodePool` (cụm) và `KafkaTopic` (các hợp đồng) — cộng thêm các kind `user`, `connect` và `bridge` mà dự án này không dùng.
2. Gói operator chứa các object phạm vi toàn cụm (CRD, `ClusterRole`) mà Role theo namespace không thể cấp; `kafka.yaml` chỉ chứa tài nguyên tùy chỉnh theo namespace, thứ Role giờ đã cho phép. Role hay `ClusterRole` chính là ranh giới.
3. `Advertised listener`, các voter của controller quorum, bản đồ giao thức bảo mật theo từng listener (thêm cả cluster ID và Service cho từng broker). Operator tính chúng từ khai báo listener và node pool.

-
4. Trạng thái thực của pod broker (`get pods`) và các dòng điều hòa trong log của operator. Condition có thể là một lần ghi cũ từ một vòng điều hòa đã timeout trong lúc pull image kéo dài; vòng lặp định kỳ tiếp theo sẽ sửa lại.
 5. Số partition được review, không có cuộc đua producer/topic ngay đầu, và một bảng kê các hợp đồng sự kiện dưới dạng object. Một partition: thứ tự toàn phần theo topic, đủ dùng chừng nào mỗi consumer group chỉ có một instance; partition có thể thêm về sau nhưng không bao giờ bớt được.

Chương 26 — Frontend và host đơn

1. Giai đoạn một (Node) biên dịch và đóng gói; giai đoạn hai (nginx) phục vụ. Chỉ `dist/` được chép bằng `COPY --from=build; Node, node_modules` và trình biên dịch bị bỏ đi, để lại một image ~15 MB với bề mặt tấn công nhỏ hơn.
2. Traefik so `/courses/7` với cả hai Ingress; `/` là tiền tố khớp dài nhất (không có quy tắc `/courses` nào), nên nginx nhận nó. Không có file như vậy; `try_files $uri $uri/ /index.html` rơi về `index.html`, ứng dụng khởi động và React Router hiển thị route từ URL.
3. Các quy tắc cho cùng một host được hợp nhất vào một bảng định tuyến và tiền tố đường dẫn khớp dài nhất thẳng: `/api/courses` khớp `/api` (dài hơn) thay vì `/`; `/assets/x.js` chỉ khớp `/`.
4. CORS chỉ áp dụng cho request xuyên origin. Trong cụm, trang và API dùng chung scheme, host và cổng (`http://ts.local`), nên trình duyệt không bao giờ gửi preflight; trên laptop trang là `localhost:3000` còn API là `localhost:8080` — cổng khác là origin khác.
5. Socket thuộc sở hữu của group docker trên host (GID 984 trên `ts-server`); user jenkins bên trong container không thuộc group nào có ID số đó, nên bị từ chối ghi. `--group-add` gán GID của host vào lúc chạy; tính toán nó giữ cho script đúng trên mọi host mà không cần cứng hóa một con số đặc thù của máy.

Chương 27 — Chặng đường phía trước

1. Độ trễ tiêu thụ (consumer-group lag): khoảng cách giữa offset mới nhất trong một partition và offset group đã commit — lag tăng nghĩa là notification-service đang tụt lại hoặc đã sập.
2. Đầy: thông tin xác thực nằm ở CI (kubernetes của Jenkins). Kéo: nó không bao giờ rời khỏi cụm; CI chỉ ghi vào git. Prune giải quyết cái bẫy apply-không-bao-giờ-xóa của Chương 20.

3. Route của gateway đổi lb://name thành http://name:port; endpoint của Service thay cho đăng ký, readiness probe thay cho heartbeat; xóa discovery-server (và các dependency eureka client), giải phóng tài nguyên của nó.
4. Chúng đã chạy không bằng root với quyền của group 0, nên SCC UID-tùy-ý của OpenShift chấp nhận chúng nguyên vẹn; chỉ còn Ingress → Route.

Thuật ngữ

Actuator Các endpoint vận hành của Spring Boot (`/actuator/health`, `/metrics`, `/prometheus`); mở ra và phân quyền là hai quyết định riêng (Ch. 1).

Advertised listener Địa chỉ mà broker Kafka báo client dùng sau bước bootstrap — nguyên nhân thường gặp của hiện tượng kết nối được rồi timeout (Ch. 10).

Artifact Bất cứ thứ gì một build sinh ra đáng giữ lại (jar, image, báo cáo); phải bất biến và truy nguyên được (Ch. 21).

Auto-configuration (tự cấu hình) Spring Boot áp dụng cấu hình có điều kiện dựa trên classpath, thuộc tính và bean của người dùng (Ch. 1).

Bootstrap servers Các địa chỉ Kafka ban đầu mà client liên hệ để lấy metadata; mọi lưu lượng sau đó dùng advertised listener (Ch. 10).

ClusterIP Kiểu Service mặc định: một IP ảo ổn định, chỉ tới được từ bên trong cụm (Ch. 18).

Consumer group (consumer group) Một nhóm consumer Kafka có tên, chia nhau các partition của một topic; mỗi nhóm giữ offset riêng (Ch. 10).

Controller (k8s) Một vòng lặp điều hòa hội tụ trạng thái thực tế về trạng thái mong muốn cho một kind object (Ch. 14).

CrashLoopBackOff Trạng thái pod: container liên tục thoát; độ trễ khởi động lại tăng gấp đôi mỗi lần (Ch. 24).

CRD / Operator Các kind resource tùy biến cộng một controller cho chúng — cách Strimzi quản lý Kafka theo kiểu khai báo (Ch. 25).

Dead-letter topic (dead-letter topic) Nơi consumer chuyển những thông điệp xử lý thất bại nhiều lần (Ch. 11).

Deployment Controller workload cho các replica thay thế được cho nhau; quản lý ReplicaSet; cho phép cập nhật cuốn chiếu và rollback (Ch. 15).

Digest Mã băm nội dung bất biến của một image (`@sha256:...`); tag dịch chuyển, digest thì không (Ch. 13).

Endpoint (k8s) Một pod Ready đứng sau một Service; danh sách sống được duy trì theo label selector và readiness (Ch. 18).

Eureka Service registry của Netflix/Spring Cloud: đăng ký, heartbeat, cache phía client (Ch. 3).

Flyway Các migration SQL có phiên bản, bất biến, áp dụng lúc khởi động và ghi

vào một bảng lịch sử (Ch. 7).

GitOps Mô hình triển khai trong đó một agent trong cụm kéo trạng thái mong muốn từ git và hội tụ cụm về đó (Ch. 26).

Ingress Object định tuyến HTTP tầng 7 (host/đường dẫn → Service), được thực thi bởi một Ingress controller như Traefik (Ch. 18).

Jib Plugin Maven build image OCI theo layer mà không cần Docker daemon (Ch. 13).

JWT Token danh tính tự chứa, có chữ ký: header.payload.signature; kiểm tra được mà không cần cơ sở dữ liệu (Ch. 5).

KRaft Cơ chế đồng thuận Raft tích hợp sẵn của Kafka thay cho ZooKeeper (Ch. 10).

kubelet Agent trên mỗi node, chạy pod và thực thi probe (Ch. 14).

Kustomize Ghép manifest không dùng template: base, overlay, patch; tích hợp sẵn trong kubectl (Ch. 20).

Liveness probe “Khởi động lại tôi nếu cái này hỏng” — probe cần đề đặt nhất (Ch. 17).

Namespace Phân vùng của cụm; phạm vi cho tên và RBAC (Ch. 14).

OOMKilled Exit 137: kernel giết container vì vượt giới hạn bộ nhớ; không có trace nào từ phía ứng dụng (Ch. 17).

Outbox pattern Làm cho ghi DB + phát sự kiện trở nên nguyên tử trên thực tế bằng cách ghi sự kiện vào một bảng trong cùng giao dịch (Ch. 11).

Partition (phân vùng) Đơn vị thứ tự và song song của Kafka; một chuỗi bản ghi bất biến, có thứ tự (Ch. 10).

Poison pill Một thông điệp giải tuần tự thất bại mãi mãi, làm kẹt partition của nó trừ khi xử lý lỗi bỏ qua nó (Ch. 11).

Presigned URL (presigned URL) Một link có chữ ký, giới hạn thời gian, cho phép trình duyệt truy cập trực tiếp một object trên S3/MinIO (Ch. 9).

Probe (startup/readiness/liveness) Các kiểm tra sức khỏe của kubelet, lần lượt gác việc khởi động lại, nhận lưu lượng, và thời gian ân hạn lúc boot (Ch. 17).

PV / PVC / StorageClass Lưu trữ đã cấp phát, yêu cầu của một workload lên nó, và nhà máy tạo PV theo nhu cầu (Ch. 19).

RBAC Kiểm soát truy cập theo vai trò: ServiceAccount (danh tính) + Role (động từ trên resource) + RoleBinding (mối nối) (Ch. 22).

Reconciliation (điều hòa) Vòng lặp điều khiển: so sánh trạng thái mong muốn và thực tế, hành động trên phần chênh lệch, mãi mãi (Ch. 14).

Registry Kho lưu image nói giao thức Docker/OCI; giao diện giữa bên build và cụm (Ch. 13).

Relaxed binding Cách Spring ánh xạ `SPRING_DATASOURCE_URL` thành `spring.datasource.url` (Ch. 1).

ReplicaSet Bộ đếm giữ N pod của một template luôn sống; do Deployment quản lý, giữ lại để rollback (Ch. 15).

Rolling update (cập nhật cuốn chiếu) Thay pod từng phần, gác bằng readiness, để service không bao giờ dừng hẳn (Ch. 15).

Secret (k8s) Object key/value theo namespace cho thông tin xác thực; mã hóa base64, được bảo vệ bởi RBAC chứ không phải mật mã học (Ch. 16).

Service (k8s) Tên ổn định + IP ảo trên một tập pod chọn theo label, lọc theo readiness (Ch. 18).

ServiceAccount Một danh tính phi con người để truy cập API — thứ Jenkins dùng để deploy (Ch. 22).

StatefulSet Controller workload cho các replica có trạng thái: tên ổn định, PVC riêng từng replica, thay thế theo thứ tự (Ch. 15).

Trusted edge (biên tin cậy) Gateway kiểm tra JWT một lần và chuyển tiếp các header danh tính; các service tin chúng — chỉ đúng khi không có đường nào đi vòng qua gateway (Ch. 5).

TOTP Mật khẩu một lần theo thời gian: bí mật chia sẻ + đồng hồ, RFC 6238 (Ch. 5).

Bảng tra nhanh

kubectl — hằng ngày

```
kubectl -n ts get pods           # the first command, always
kubectl -n ts get pods -w        # ...and watch it change
kubectl -n ts describe pod <pod> # platform's view; read Events
kubectl -n ts logs <pod>         # app's view
kubectl -n ts logs <pod> --previous # the container that just died
kubectl -n ts exec -it <pod> -- sh # see the world from inside
kubectl -n ts get endpoints <svc> # is anyone behind the Service?
kubectl -n ts get events --sort-by=.lastTimestamp | tail
```

kubectl — deploy và rollback

```
kubectl apply -k deploy/k8s/base # converge on the manifests
kubectl kustomize deploy/k8s/base | less # render without applying
kubectl -n ts set image deployment/<s> <s>=localhost:5000/<s>:<tag>
kubectl -n ts rollout status deployment/<s> --timeout=300s
kubectl -n ts rollout undo deployment/<s> # back to previous ReplicaSet
kubectl -n ts rollout restart deployment/<s> # re-read Secrets, re-pull
kubectl -n ts get rs -l app=<s> # revision history, live
```

kubectl — dung lượng và RBAC

```
kubectl top pods -n ts # actual vs requested
kubectl -n ts get pvc; kubectl get pv # storage and its backing
kubectl auth can-i <verb> <resource> -n ts \
  --as system:serviceaccount:ts:jenkins # test permissions dry
```

docker — các dịch vụ trên host ts-server

```

docker ps                                # jenkins, registry up?
docker logs -f jenkins
docker restart jenkins
curl -s localhost:5000/v2/_catalog      # what the registry holds
curl -s localhost:5000/v2/<name>/tags/list
docker system df; docker system prune   # disk hygiene (careful)

```

maven — máy local và CI

```

mvn -B -pl <modules> -am verify        # build listed + deps
mvn -B -pl course-service -am jib:build # image for one service
mvn -B jib:build -Djib.to.tags=<sha>    # extra tag, as CI does
mvn dependency:tree -pl course-service   # who pulled that jar in
mvn spring-boot:run -pl course-service   # run one service raw

```

compose — stack dev

```

docker compose up -d postgres-course kafka minio # infra only
docker compose up --no-deps course-service        # one service, own terminal
docker compose logs -f --tail=100 <svc>
docker compose down                               # keep volumes
docker compose down -v                            # DESTROY volumes too

```

k3s — bản thân cụm

```

sudo systemctl status k3s
sudo systemctl restart k3s          # rereads registries.yaml; pods bounce
sudo ls /var/lib/rancher/k3s/storage/ # PVs as directories
kubectl get nodes -o wide

```

kafka — broker trong cụm (Ch. 25)

```

kubectl -n ts get kafka,kafkanodepools,kafkatopics # operator's view
kubectl -n ts logs ts-kafka-dual-role-0 --tail=50  # the broker itself
# a throwaway pod with the Kafka CLI, same network position as services:

```

```
kubect1 -n ts run kcli --rm -it --restart=Never \  
  --image=quay.io/strimzi/kafka:1.2.0-kafka-4.3.1 -- sh  
B=ts-kafka-kafka-bootstrap:9092  
bin/kafka-topics.sh --bootstrap-server $B --list  
bin/kafka-console-consumer.sh --bootstrap-server $B \  
  --topic ts.user.registered --from-beginning  
bin/kafka-consumer-groups.sh --bootstrap-server $B \  
  --describe --group notification-group      # lag per partition  
kubect1 -n ts port-forward svc/maildev 1080:1080    # caught mail UI
```

Các file cấu hình

Các file đúng như chúng đang nằm trong repository tại thời điểm build — được chèn thẳng từ mã nguồn, nên phụ lục này không thể trôi khỏi thực tế. Bình luận từng dòng nằm ở các chương; đây là bản tham chiếu để đọc khi không có máy.

Jenkinsfile

```

pipeline {
    agent any

    options {
        timestamps()
        disableConcurrentBuilds()
        buildDiscarder(logRotator(numToKeepStr: '10'))
    }

    triggers { pollSCM('H/5 * * * *') }

    environment {
        JAVA_HOME = "${env.JDK25_HOME}"
        PATH      = "${env.JDK25_HOME}/bin:${env.PATH}"
        TAG       = "${env.GIT_COMMIT.take(7)}"
        NS        = 'ts'
        REGISTRY  = 'localhost:5000'
        // frontend is in SERVICES (kubect1) but not MODULES (Maven): it is built by
        // the 'Frontend image' stage with docker, not by Jib.
        SERVICES = 'config-server discovery-server course-service api-gateway auth-service dictionary-
        ↪ service notification-service frontend'
        MODULES  = 'common,config-server,discovery-server,course-service,api-gateway,auth-service,
        ↪ dictionary-service,notification-service'
    }

    stages {
        stage('Build') {
            steps {
                dir('Project/TeacherSupporter') {
                    sh 'mvn -B -DskipTests -pl ${MODULES} -am verify'
                }
            }
        }

        stage('Push images') {
            steps {
                dir('Project/TeacherSupporter') {
                    // 'package' before jib:build is load-bearing: a bare goal runs no

```

```

    // lifecycle, so sibling modules (common) are never built in this
    // session and Jib cannot resolve com.ts:common from the reactor.
    sh 'mvn -B -DskipTests -pl ${MODULES} -am package jib:build -Djib.to.tags=${TAG}'
  }
}
}

stage('Frontend image') {
  steps {
    dir('Project/TeacherSupporter') {
      // Multi-stage Dockerfile: node builds dist/, nginx ships it. The
      // registry is localhost:5000 from the host's point of view and the
      // container shares the host network, so push needs no extra config.
      sh '''
        set -eu
        docker build -t ${REGISTRY}/frontend:${TAG} -t ${REGISTRY}/frontend:latest frontend
        docker push ${REGISTRY}/frontend:${TAG}
        docker push ${REGISTRY}/frontend:latest
      '''
    }
  }
}

stage('Deploy') {
  steps {
    dir('Project/TeacherSupporter') {
      withCredentials([file(credentialsId: 'kubeconfig-ts', variable: 'KUBECONFIG')]) {
        sh '''
          set -eu
          kubectl apply -k deploy/k8s/base
          for s in ${SERVICES}; do
            kubectl -n ${NS} set image deployment/${s} ${s}=${REGISTRY}/${s}:${TAG}
          done
          for s in ${SERVICES}; do
            kubectl -n ${NS} rollout status deployment/${s} --timeout=300s
          done
        '''
      }
    }
  }
}

post {
  success { echo "Deployed ${TAG} to namespace ${NS}." }
  failure {
    dir('Project/TeacherSupporter') {
      withCredentials([file(credentialsId: 'kubeconfig-ts', variable: 'KUBECONFIG')]) {
        sh 'kubectl -n ${NS} get pods || true'
      }
    }
  }
}
}

```

deploy/jenkins/Dockerfile

```
# Jenkins controller for ts-server, with the build toolchain baked in.
#
# Homelab compromise, stated openly: builds run ON the controller (2 executors),
# not on agent pods. On a 2-core box, agent pods would cost more CPU than they
# save. Migrating to the Kubernetes plugin is a later, self-contained exercise.
#
#   docker build -t ts-jenkins:1 deploy/jenkins
#   see run.sh for the docker run line
FROM jenkins/jenkins:lts-jdk21

USER root

# --- JDK 25 (Temurin) -- the project targets Java 25 -----
RUN apt-get update && apt-get install -y --no-install-recommends wget gpg apt-transport-https \
  && wget -qO- https://packages.adoptium.net/artifactory/api/gpg/key/public \
    | gpg --dearmor -o /usr/share/keyrings/adoptium.gpg \
  && echo "deb [signed-by=/usr/share/keyrings/adoptium.gpg] https://packages.adoptium.net/artifactory/
    ↪ deb $(. /etc/os-release && echo $VERSION_CODENAME) main" \
    > /etc/apt/sources.list.d/adoptium.list \
  && apt-get update && apt-get install -y --no-install-recommends temurin-25-jdk \
  && rm -rf /var/lib/apt/lists/*

# --- Maven 3.9 (Debian's apt maven is 3.8, too old for some plugins) -----
ARG MAVEN_VERSION=3.9.9
RUN wget -qO /tmp/mvn.tgz https://archive.apache.org/dist/maven/maven-3/${MAVEN_VERSION}/binaries/
  ↪ apache-maven-${MAVEN_VERSION}-bin.tar.gz \
  && tar -xzf /tmp/mvn.tgz -C /opt && ln -s /opt/apache-maven-${MAVEN_VERSION} /opt/maven \
  && rm /tmp/mvn.tgz

# --- kubectl: the deploy stage talks to k3s with this -----
RUN wget -qO /usr/local/bin/kubectl \
  "https://dl.k8s.io/release/$(wget -qO- https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/
  ↪ kubectl" \
  && chmod +x /usr/local/bin/kubectl

# --- Docker CLI (client only): the frontend image is a classic Dockerfile ---
# build, which Jib cannot do. Only the CLI is installed; it talks to the
# HOST's daemon through /var/run/docker.sock mounted by run.sh. No daemon
# inside the container, no docker-in-docker.
RUN install -m 0755 -d /etc/apt/keyrings \
  && wget -qO- https://download.docker.com/linux/debian/gpg | gpg --dearmor -o /etc/apt/keyrings/
  ↪ docker.gpg \
  && echo "deb [signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/debian $(. /
  ↪ etc/os-release && echo $VERSION_CODENAME) stable" \
  > /etc/apt/sources.list.d/docker.list \
  && apt-get update && apt-get install -y --no-install-recommends docker-ce-cli \
  && rm -rf /var/lib/apt/lists/*

# Jenkins itself keeps running on the bundled JDK 21; only *builds* see JDK 25.
# JAVA_HOME is therefore NOT changed here -- the Jenkinsfile sets it per stage.
ENV MAVEN_HOME=/opt/maven \
  JDK25_HOME=/usr/lib/jvm/temurin-25-jdk-amd64 \
  PATH=/opt/maven/bin:${PATH}
```

USER jenkins

Plugins pre-installed so the first boot needs no wizard clicks beyond unlock.

RUN jenkins-plugin-cli --plugins \
git workflow-aggregator pipeline-stage-view junit timestamp credentials-binding

deploy/k8s/jenkins-rbac.yaml

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: jenkins
  namespace: ts
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: jenkins-deployer
  namespace: ts
rules:
  # How to read a rule: "may perform <verbs> on <resources> in <apiGroups>".
  # Rules are additive; anything not listed is Forbidden. Namespace-scoped
  # (this is a Role, not a ClusterRole), so nothing here reaches kube-system.
  #
  # Verb cheat-sheet for the pipeline's commands:
  #   kubectl apply          -> get + create (new) or patch (existing)
  #   kubectl set image      -> get + patch
  #   kubectl rollout status -> get/list/watch on deployments AND replicaset
  #   kubectl logs / get pods -> get/list on pods, get on pods/log

  # -- What `kubectl apply -k deploy/k8s/base` creates, by API group -----
  # Core group is spelled "" (empty string). Namespace is deliberately absent:
  # it is cluster-scoped and cannot be granted by a Role, so the pipeline must
  # not try to apply namespace.yaml (ts already exists).
  - apiGroups: [""]
    resources: ["services", "persistentvolumeclaims", "secrets", "configmaps"]
    verbs: ["get", "list", "create", "patch", "update"]

  - apiGroups: ["apps"]
    resources: ["deployments", "statefulsets"]
    verbs: ["get", "list", "watch", "create", "patch", "update"]

  - apiGroups: ["networking.k8s.io"]
    resources: ["ingresses"]
    verbs: ["get", "list", "create", "patch", "update"]

  # Strimzi custom resources (milestone 6). Namespaced like everything
  # else here -- the pipeline may declare Kafka clusters and topics in ts,
  # but installing the operator/CRDs stays an admin act (install-strimzi.sh).
  - apiGroups: ["kafka.strimzi.io"]
    resources: ["kafkas", "kafkanodepools", "kafkatopics"]
```



```

    verbs: ["get", "list", "create", "patch", "update"]

# -- Read-only extras for `rollout status` and post-failure debugging -----
# A Deployment rolls out via ReplicaSets it owns; rollout status watches them.
- apiGroups: ["apps"]
  resources: ["replicasets"]
  verbs: ["get", "list", "watch"]

- apiGroups: [""]
  resources: ["pods", "pods/log", "events"]
  verbs: ["get", "list", "watch"]

# NOT granted, on purpose:
#   delete          -- CI should roll forward, never wipe; a human deletes.
#   nodes, namespaces -- cluster-scoped, out of a Role's reach anyway.
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: jenkins-deployer
  namespace: ts
subjects:
- kind: ServiceAccount
  name: jenkins
  namespace: ts
roleRef:
  kind: Role
  name: jenkins-deployer
  apiGroup: rbac.authorization.k8s.io

```

deploy/k8s/create-secrets.sh

```

#!/usr/bin/env bash
#
# Create the Secrets the ts namespace needs. Deliberately NOT a manifest:
# secrets do not belong in git. Re-runnable -- it replaces what is there.
#
#   ./deploy/k8s/create-secrets.sh
#
# Values match docker-compose.yml so behaviour is identical to local dev.
# Override any of them from the environment before running.

set -euo pipefail

NS="${NS:-ts}"
PG_USER="${PG_USER:-postgres}"
PG_PASSWORD="${PG_PASSWORD:-root}"
MINIO_USER="${MINIO_USER:-minioadmin}"
MINIO_PASSWORD="${MINIO_PASSWORD:-minioadmin}"
# Default matches the fallback baked into config-server's api-gateway.yml /
# auth-service.yml so dev behaviour is unchanged. Override in anything public.
JWT_SECRET="${JWT_SECRET:-mySecretKeyThatIsAtLeast256BitsLongForHs256Algorithm123456}"

```

```
kubectrl get namespace "$NS" >/dev/null 2>&1 || kubectrl create namespace "$NS"

# Four secrets, not two: the same credential is spelled differently by the
# server that owns it and the client that consumes it. Postgres wants
# POSTGRES_USER; Spring wants SPRING_DATASOURCE_USERNAME. Keeping them as
# separate secrets means each pod's envFrom pulls in only names it understands,
# rather than a grab-bag it has to ignore half of.
#
# --dry-run=client | apply is the idempotent-create idiom: `kubectrl create secret`
# alone fails if the secret exists, and there is no `kubectrl create --force`.

# --- postgres, server side ---
kubectrl -n "$NS" create secret generic postgres-course \
  --from-literal=POSTGRES_USER="$PG_USER" \
  --from-literal=POSTGRES_PASSWORD="$PG_PASSWORD" \
  --from-literal=POSTGRES_DB=ts_course \
  --dry-run=client -o yaml | kubectrl apply -f -

# --- postgres, client side (course-service) ---
kubectrl -n "$NS" create secret generic postgres-course-app \
  --from-literal=SPRING_DATASOURCE_USERNAME="$PG_USER" \
  --from-literal=SPRING_DATASOURCE_PASSWORD="$PG_PASSWORD" \
  --dry-run=client -o yaml | kubectrl apply -f -

# --- minio, server side ---
kubectrl -n "$NS" create secret generic minio \
  --from-literal=MINIO_ROOT_USER="$MINIO_USER" \
  --from-literal=MINIO_ROOT_PASSWORD="$MINIO_PASSWORD" \
  --dry-run=client -o yaml | kubectrl apply -f -

# --- minio, client side (course-service) ---
kubectrl -n "$NS" create secret generic minio-app \
  --from-literal=APP_S3_ACCESS_KEY="$MINIO_USER" \
  --from-literal=APP_S3_SECRET_KEY="$MINIO_PASSWORD" \
  --dry-run=client -o yaml | kubectrl apply -f -

# --- postgres-auth, server side ---
kubectrl -n "$NS" create secret generic postgres-auth \
  --from-literal=POSTGRES_USER="$PG_USER" \
  --from-literal=POSTGRES_PASSWORD="$PG_PASSWORD" \
  --from-literal=POSTGRES_DB=ts_auth \
  --dry-run=client -o yaml | kubectrl apply -f -

# --- postgres-auth, client side (auth-service) ---
kubectrl -n "$NS" create secret generic postgres-auth-app \
  --from-literal=SPRING_DATASOURCE_USERNAME="$PG_USER" \
  --from-literal=SPRING_DATASOURCE_PASSWORD="$PG_PASSWORD" \
  --dry-run=client -o yaml | kubectrl apply -f -

# --- google oauth (auth-service) ---
# Real values come from .env / the environment; the placeholders keep the pod
# bootable when they are absent -- Google login then fails at click time, not
# the whole service at startup. Mirrors the ${GOOGLE_CLIENT_ID:placeholder}
# defaults in config-server's auth-service.yml.
kubectrl -n "$NS" create secret generic google-oauth \
```

```

--from-literal=GOOGLE_CLIENT_ID="${GOOGLE_CLIENT_ID:-placeholder}" \
--from-literal=GOOGLE_CLIENT_SECRET="${GOOGLE_CLIENT_SECRET:-placeholder}" \
--dry-run=client -o yaml | kubectl apply -f -

# --- jwt, shared by api-gateway (validates) and auth-service (signs) ---
kubectl -n "$NS" create secret generic jwt \
  --from-literal=JWT_SECRET="${JWT_SECRET}" \
  --dry-run=client -o yaml | kubectl apply -f -

echo
kubectl -n "$NS" get secrets

```

deploy/k8s/install-strimzi.sh

```

#!/usr/bin/env bash
#
# One-time ADMIN step: install the Strimzi operator + CRDs into the ts
# namespace, watching that namespace only. Run before the first
# `kubectl apply -k deploy/k8s/base` that includes kafka.yaml.
#
# Same category as namespace creation: CRDs and the operator's RBAC are
# cluster-scoped, so the Jenkins ServiceAccount cannot (and should not)
# install them. Jenkins only applies the namespaced Kafka/KafkaTopic
# resources afterwards.
#
# ./deploy/k8s/install-strimzi.sh

set -euo pipefail

NS="${NS:-ts}"

kubectl get namespace "$NS" >/dev/null 2>&1 || kubectl create namespace "$NS"

# Server-side apply, because the Strimzi bundle's CRDs are too large for
# the client-side last-applied-configuration annotation. Idempotent: safe
# to re-run, also how you take operator upgrades later.
kubectl apply --server-side --force-conflicts \
  -f "https://strimzi.io/install/latest?namespace=${NS}" -n "$NS"

kubectl -n "$NS" rollout status deployment/strimzi-cluster-operator --timeout=300s

echo
echo "Operator ready. Kafka clusters in '${NS}' will now be reconciled."

```

deploy/k8s/base/kustomization.yaml

```

apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

```

```
namespace: ts

# Applied in dependency order for readability only -- Kubernetes starts pods in
# parallel regardless. Ordering here is documentation, not orchestration.
resources:
  # namespace.yaml is intentionally NOT listed. Namespaces are cluster-scoped,
  # and the Jenkins ServiceAccount only holds a namespaced Role, so `apply`
  # would be Forbidden. The namespace is created once by create-secrets.sh
  # (or `kubectl apply -f namespace.yaml` by an admin).
  - postgres-course.yaml
  - postgres-auth.yaml
  - mongodb.yaml
  - minio.yaml
  # kafka.yaml requires the Strimzi operator (one-time admin step:
  # ../install-strimzi.sh) -- without its CRDs, apply fails on kind Kafka.
  - kafka.yaml
  - kafka-topics.yaml
  - maildev.yaml
  - config-server.yaml
  - discovery-server.yaml
  - course-service.yaml
  - auth-service.yaml
  - dictionary-service.yaml
  - notification-service.yaml
  - api-gateway.yaml
  # Non-JVM deployable: nginx + React build, claims / on ts.local.
  - frontend.yaml

# Stamped onto every object, so `kubectl delete -l part-of=teacher-supporter`
# cleans up the whole slice without touching kube-system.
commonLabels:
  app.kubernetes.io/part-of: teacher-supporter

# Secrets are NOT here on purpose -- see ../create-secrets.sh.
```

deploy/k8s/base/namespace.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  name: ts
```

deploy/k8s/base/postgres-course.yaml

```
# StatefulSet rather than Deployment: a database needs a stable identity and a
# volume that is never handed to a second replica. volumeClaimTemplates gives
# each pod its own PVC, provisioned by k3s' local-path on the node's SSD.
```

```

apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres-course
spec:
  serviceName: postgres-course
  replicas: 1
  selector:
    matchLabels: { app: postgres-course }
  template:
    metadata:
      labels: { app: postgres-course }
    spec:
      containers:
        - name: postgres
          image: postgres:17
          ports:
            - { name: postgres, containerPort: 5432 }
          envFrom:
            - secretRef: { name: postgres-course }
          env:
            # The official image refuses to initialise into a non-empty directory,
            # and local-path volumes arrive with a lost+found. A subdirectory
            # sidesteps it -- the standard fix, not a workaround.
            - { name: PGDATA, value: /var/lib/postgresql/data/pgdata }
          volumeMounts:
            - { name: data, mountPath: /var/lib/postgresql/data }
          readinessProbe:
            exec:
              command: ["pg_isready", "-U", "postgres", "-d", "ts_course"]
              initialDelaySeconds: 10
              periodSeconds: 5
              failureThreshold: 6
          resources:
            requests: { memory: 256Mi, cpu: 100m }
            limits: { memory: 512Mi }
      volumeClaimTemplates:
        - metadata:
            name: data
          spec:
            accessModes: [ReadWriteOnce]
            resources:
              requests:
                storage: 5Gi
---
apiVersion: v1
kind: Service
metadata:
  name: postgres-course
spec:
  selector: { app: postgres-course }
  ports:
    - { name: postgres, port: 5432, targetPort: 5432 }

```

deploy/k8s/base/postgres-auth.yaml

```
# Second Postgres, same shape as postgres-course.yaml -- see the comments
# there for why StatefulSet, PGDATA subdirectory, and volumeClaimTemplates.
#
# Deliberately a SEPARATE server rather than a second database in the same
# one: database-per-service is the microservice contract. auth and course
# must not be able to join each other's tables, today or by accident later.
# On this box it costs ~256Mi -- cheap insurance.
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres-auth
spec:
  serviceName: postgres-auth
  replicas: 1
  selector:
    matchLabels: { app: postgres-auth }
  template:
    metadata:
      labels: { app: postgres-auth }
    spec:
      containers:
        - name: postgres
          image: postgres:17
          ports:
            - { name: postgres, containerPort: 5432 }
          envFrom:
            - secretRef: { name: postgres-auth }
          env:
            - { name: PGDATA, value: /var/lib/postgresql/data/pgdata }
          volumeMounts:
            - { name: data, mountPath: /var/lib/postgresql/data }
          readinessProbe:
            exec:
              command: ["pg_isready", "-U", "postgres", "-d", "ts_auth"]
              initialDelaySeconds: 10
              periodSeconds: 5
              failureThreshold: 6
          resources:
            requests: { memory: 256Mi, cpu: 100m }
            limits: { memory: 512Mi }
      volumeClaimTemplates:
        - metadata:
            name: data
          spec:
            accessModes: [ReadWriteOnce]
            resources:
              requests:
                storage: 5Gi
---
apiVersion: v1
kind: Service
metadata:
  name: postgres-auth
```

```
spec:
  selector: { app: postgres-auth }
  ports:
    - { name: postgres, port: 5432, targetPort: 5432 }
```

deploy/k8s/base/mongodb.yaml

```
# Same reasoning as postgres-course.yaml: databases get a StatefulSet for the
# stable identity + a volumeClaimTemplate PVC that is never shared.
#
# No credentials, matching docker-compose.yml -- the official mongo image only
# enables auth when MONGO_INITDB_ROOT_USERNAME/PASSWORD are set. Acceptable
# here because the Service is ClusterIP: only pods inside the cluster can
# reach it, and there is no Ingress to it. If this ever went multi-tenant or
# network policies were the goal, auth would come first.
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mongodb
spec:
  serviceName: mongodb
  replicas: 1
  selector:
    matchLabels: { app: mongodb }
  template:
    metadata:
      labels: { app: mongodb }
    spec:
      containers:
        - name: mongodb
          image: mongo:8
          ports:
            - { name: mongo, containerPort: 27017 }
          volumeMounts:
            # Unlike postgres, mongod is happy with a lost+found in its data
            # dir, so no PGDATA-style subdirectory trick is needed.
            - { name: data, mountPath: /data/db }
      readinessProbe:
        # mongosh ping: same idea as pg_isready -- "is the server
        # accepting commands", not just "is the port open".
        exec:
          command: ["mongosh", "--quiet", "--eval", "db.adminCommand('ping')"]
          initialDelaySeconds: 10
          periodSeconds: 5
          failureThreshold: 6
      resources:
        requests: { memory: 256Mi, cpu: 100m }
        limits: { memory: 512Mi }
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
```

```
    accessModes: [ReadWriteOnce]
    resources:
      requests:
        storage: 5Gi
---
apiVersion: v1
kind: Service
metadata:
  name: mongodb
spec:
  selector: { app: mongodb }
  ports:
    - { name: mongo, port: 27017, targetPort: 27017 }
```

deploy/k8s/base/minio.yaml

```
# MinIO is a hard startup dependency for course-service: S3StorageService's
# @PostConstruct calls headBucket and only catches NoSuchBucketException, so an
# unreachable MinIO fails bean creation. Kubernetes has no depends_on -- course
# service will CrashLoopBackOff until this pod is serving, then recover.
apiVersion: apps/v1
kind: Deployment
metadata:
  name: minio
spec:
  replicas: 1
  strategy: { type: Recreate } # RWO volume: never two pods holding it at once
  selector:
    matchLabels: { app: minio }
  template:
    metadata:
      labels: { app: minio }
    spec:
      containers:
        - name: minio
          image: minio/minio:latest
          args: ["server", "/data", "--console-address", ":9001"]
          ports:
            - { name: s3, containerPort: 9000 }
            - { name: console, containerPort: 9001 }
          envFrom:
            - secretRef: { name: minio }
          volumeMounts:
            - { name: data, mountPath: /data }
          readinessProbe:
            httpGet: { path: /minio/health/ready, port: s3 }
            initialDelaySeconds: 10
            periodSeconds: 5
          resources:
            requests: { memory: 256Mi, cpu: 100m }
            limits: { memory: 512Mi }
      volumes:
```



```

    - name: data
      persistentVolumeClaim: { claimName: minio-data }
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: minio-data
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests:
      storage: 5Gi
---
apiVersion: v1
kind: Service
metadata:
  name: minio
spec:
  selector: { app: minio }
  ports:
    - { name: s3, port: 9000, targetPort: 9000 }
    - { name: console, port: 9001, targetPort: 9001 }

```

deploy/k8s/base/kafka.yaml

```

# Kafka via the Strimzi operator (KRaft mode -- no ZooKeeper).
#
# PREREQUISITE (one-time, admin): the operator + CRDs must be installed
# first -- ../install-strimzi.sh. Without it, applying this file fails with
# "no matches for kind Kafka". Same class of admin step as namespace.yaml.
#
# We do not create StatefulSets or Services for Kafka ourselves. We declare
# a Kafka cluster as data; the operator (Chapter 14's reconciliation idea,
# extended to custom kinds) computes the pods, Services, listener config,
# and rolling restarts from it. What we hand-tuned in docker-compose.yml --
# the advertised-listeners maze -- the operator now derives.
#
# Client address inside the cluster: ts-kafka-kafka-bootstrap:9092
# (<cluster-name>-kafka-bootstrap -- Strimzi's naming convention)
---
# Node pool: WHERE Kafka runs and with what storage/resources.
# One node holding both KRaft roles -- the k8s equivalent of the compose
# broker's process.roles=broker,controller. RF=1 everywhere: a homelab
# accepts zero fault tolerance; production starts at 3 nodes / RF=3.
apiVersion: kafka.strimzi.io/v1
kind: KafkaNodePool
metadata:
  name: dual-role
  labels:
    # Joins this pool to the Kafka resource below. The operator matches on
    # this label -- it must equal the Kafka resource's metadata.name.
    strimzi.io/cluster: ts-kafka

```

```
spec:
  replicas: 1
  roles: [controller, broker]
  storage:
    type: persistent-claim      # local-path PVC, like the databases
    size: 5Gi
    deleteClaim: false        # deleting the Kafka CR keeps the log data
  resources:
    requests: { memory: 768Mi, cpu: 200m }
    limits: { memory: 1Gi }
---
apiVersion: kafka.strimzi.io/v1
kind: Kafka
metadata:
  name: ts-kafka
  annotations:
    strimzi.io/kraft: enabled
    strimzi.io/node-pools: enabled
spec:
  kafka:
    # No version pinned: the operator's default (its newest supported
    # Kafka) applies, and operator upgrades roll the broker for us.
    listeners:
      # One internal plaintext listener. No TLS/auth for the same reason
      # Mongo has none: ClusterIP-only, nothing routes here from outside.
      - name: plain
        port: 9092
        type: internal
        tls: false
    config:
      # Single broker: every replication setting must be 1, or topic
      # creation fails asking for brokers that do not exist.
      offsets.topic.replication.factor: 1
      transaction.state.log.replication.factor: 1
      transaction.state.log.min.isr: 1
      default.replication.factor: 1
      min.insync.replicas: 1
      log.retention.hours: 168      # matches docker-compose.yml
    jvmOptions:
      # Kafka's default heap assumes a server; half a GiB serves a homelab
      # event trickle comfortably and leaves RAM for the Spring services.
      -Xms: 256m
      -Xmx: 512m
  entityOperator:
    # Reconciles KafkaTopic resources (kafka-topics.yaml) into real topics.
    # userOperator is omitted: no authentication, so no users to manage.
    topicOperator: {}
```

deploy/k8s/base/kafka-topics.yaml

```
# Topics as declared objects, reconciled by Strimzi's topic operator.
#
```

```

# Kafka would auto-create these on first use (auto.create.topics.enable
# defaults to true), but declared topics beat auto-created ones: partition
# count and retention are decisions, not defaults; they exist before the
# first producer runs (no race on day one); and 'kubectl get kafkatopics'
# becomes the inventory of every event contract in the system.
#
# partitions: 1 -- total order per topic, which trivially preserves the
# per-user ordering the publishers key for. Raise partitions later only
# with a consumer that scales past one instance; note partitions can be
# added but never removed.
apiVersion: kafka.strimzi.io/v1
kind: KafkaTopic
metadata:
  name: ts.user.registered
  labels:
    strimzi.io/cluster: ts-kafka
spec:
  partitions: 1
  replicas: 1
---
apiVersion: kafka.strimzi.io/v1
kind: KafkaTopic
metadata:
  name: ts.user.activated
  labels:
    strimzi.io/cluster: ts-kafka
spec:
  partitions: 1
  replicas: 1
---
apiVersion: kafka.strimzi.io/v1
kind: KafkaTopic
metadata:
  name: ts.user.admin-provisioned
  labels:
    strimzi.io/cluster: ts-kafka
spec:
  partitions: 1
  replicas: 1
---
apiVersion: kafka.strimzi.io/v1
kind: KafkaTopic
metadata:
  name: ts.assignment.created
  labels:
    strimzi.io/cluster: ts-kafka
spec:
  partitions: 1
  replicas: 1

```

deploy/k8s/base/maildev.yaml

```
# Maildev: an SMTP server that delivers nothing -- it catches every mail
# and shows it in a web UI. The dev/homelab stand-in for a real relay;
# swapping to real mail later is only MAIL_HOST/credentials env changes
# on notification-service.
apiVersion: apps/v1
kind: Deployment
metadata:
  name: maildev
spec:
  replicas: 1
  selector:
    matchLabels: { app: maildev }
  template:
    metadata:
      labels: { app: maildev }
    spec:
      containers:
        - name: maildev
          image: maildev/maildev:2.2.1
          ports:
            - { name: web, containerPort: 1080 }
            - { name: smtp, containerPort: 1025 }
          readinessProbe:
            tcpSocket: { port: smtp }
            periodSeconds: 5
          resources:
            requests: { memory: 64Mi, cpu: 25m }
            limits: { memory: 128Mi }
---
apiVersion: v1
kind: Service
metadata:
  name: maildev
spec:
  selector: { app: maildev }
  ports:
    - { name: web, port: 1080, targetPort: 1080 }
    - { name: smtp, port: 1025, targetPort: 1025 }
# No Ingress on purpose: the UI is an operator tool, not part of the app.
# To read caught mail: kubectl -n ts port-forward svc/maildev 1080:1080
# then open http://localhost:1080 on Windows.
```

deploy/k8s/base/config-server.yaml

```
# Config Server holds no state: its YAML is baked into the jar under
# classpath:/configurations (spring.profiles.active=native). No PVC, no DB.
# Every other pod blocks on this one, so it starts first in practice by virtue
# of being the only thing with no dependencies of its own.
apiVersion: apps/v1
kind: Deployment
metadata:
  name: config-server
```

```

spec:
  replicas: 1
  selector:
    matchLabels: { app: config-server }
  template:
    metadata:
      labels: { app: config-server }
    spec:
      containers:
        - name: config-server
          image: localhost:5000/config-server:1.0.0-SNAPSHOT
          # SNAPSHOT tags are overwritten on every push; the default IfNotPresent
          # would keep serving whatever the node cached. Always re-check the
          # registry (cheap: only the manifest is fetched if digests match).
          imagePullPolicy: Always
          ports:
            - { name: http, containerPort: 8888 }
          env:
            - { name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0" }
          # startupProbe buys a slow JVM time to boot without forcing a long
          # liveness initialDelay. Until it passes, liveness is not evaluated.
          startupProbe:
            httpGet: { path: /actuator/health, port: http }
            periodSeconds: 5
            failureThreshold: 30      # up to 150s to start
          readinessProbe:
            httpGet: { path: /actuator/health, port: http }
            periodSeconds: 10
          resources:
            requests: { memory: 384Mi, cpu: 100m }
            limits: { memory: 640Mi }
---
apiVersion: v1
kind: Service
metadata:
  name: config-server
spec:
  selector: { app: config-server }
  ports:
    - { name: http, port: 8888, targetPort: 8888 }

```

deploy/k8s/base/discovery-server.yaml

```

# Eureka. Kept deliberately -- this project is the classic Spring Cloud
# showcase, and running it on Kubernetes alongside Service DNS is the point.
#
# Note the two service-discovery systems now coexist: Kubernetes resolves
# `discovery-server` to a ClusterIP, and Eureka resolves `course-service` to a
# pod IP that the clients registered themselves. Only the first survives a pod
# reschedule instantly; Eureka takes up to 90s to evict a dead instance.
apiVersion: apps/v1
kind: Deployment

```

```
metadata:
  name: discovery-server
spec:
  replicas: 1
  selector:
    matchLabels: { app: discovery-server }
  template:
    metadata:
      labels: { app: discovery-server }
    spec:
      containers:
        - name: discovery-server
          image: localhost:5000/discovery-server:1.0.0-SNAPSHOT
          # SNAPSHOT tags are overwritten on every push; the default IfNotPresent
          # would keep serving whatever the node cached. Always re-check the
          # registry (cheap: only the manifest is fetched if digests match).
          imagePullPolicy: Always
          ports:
            - { name: http, containerPort: 8761 }
          env:
            - { name: SPRING_CONFIG_IMPORT, value: "configserver:http://config-server:8888" }
            - { name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0" }
          startupProbe:
            httpGet: { path: /actuator/health, port: http }
            periodSeconds: 5
            failureThreshold: 30
          readinessProbe:
            httpGet: { path: /actuator/health, port: http }
            periodSeconds: 10
          resources:
            requests: { memory: 384Mi, cpu: 100m }
            limits: { memory: 640Mi }
---
apiVersion: v1
kind: Service
metadata:
  name: discovery-server
spec:
  selector: { app: discovery-server }
  ports:
    - { name: http, port: 8761, targetPort: 8761 }
```

deploy/k8s/base/course-service.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: course-service
spec:
  replicas: 1
  selector:
    matchLabels: { app: course-service }
```

```

template:
  metadata:
    labels: { app: course-service }
  spec:
    containers:
      - name: course-service
        image: localhost:5000/course-service:1.0.0-SNAPSHOT
        # SNAPSHOT tags are overwritten on every push; the default IfNotPresent
        # would keep serving whatever the node cached. Always re-check the
        # registry (cheap: only the manifest is fetched if digests match).
        imagePullPolicy: Always
        ports:
          - { name: http, containerPort: 8082 }
        env:
          # Same values as docker-compose.yml, with compose service names
          # swapped for Kubernetes Service names. They happen to be identical,
          # because both resolve a name to an address on a private network.
          - { name: SPRING_CONFIG_IMPORT, value: "configserver:http://config-server:8888" }
          - { name: EUREKA_CLIENT_SERVICEURL_DEFAULTZONE, value: "http://discovery-server:8761/"
↪ eureka/" }
          - { name: SPRING_DATASOURCE_URL, value: "jdbc:postgresql://postgres-course:5432/ts_course
↪ " }
          - { name: APP_S3_ENDPOINT, value: "http://minio:9000" }

          # Eureka on Kubernetes needs this or nothing can call anything.
          # By default a client registers its *hostname*, which inside a pod is
          # the pod name -- a string no other pod can resolve. Pod IPs are
          # routable cluster-wide, so register the IP instead.
          - { name: EUREKA_INSTANCE_PREFER_IP_ADDRESS, value: "true" }

          # Strimzi's bootstrap Service (milestone 6). This service both
          # produces (ts.assignment.created) and consumes (ts.user.activated).
          - { name: SPRING_KAFKA_BOOTSTRAP_SERVERS, value: "ts-kafka-kafka-bootstrap:9092" }

          # Zipkin is not in the cluster yet (observability is milestone 8).
          - { name: MANAGEMENT_TRACING_SAMPLING_PROBABILITY, value: "0.0" }

          - { name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0" }
        envFrom:
          - secretRef: { name: postgres-course-app }
          - secretRef: { name: minio-app }

        # ---- Health probes -----
        # No actuator in this service, so /actuator/health is a 404. The one
        # unauthenticated HTTP path that exists is springdoc's /v3/api-docs
        # (permitAll in SecurityConfig). It only answers 200 once the whole
        # Spring context is up -- which is AFTER Flyway has migrated and the
        # MinIO bucket check has run. That makes it a true "ready" signal,
        # unlike a tcpSocket check that passes as soon as Tomcat binds.

        # startupProbe: "give the JVM time to boot, then hand over."
        # While this probe is failing, readiness/liveness are NOT evaluated.
        # 12 x 10s = up to 120s of grace for a cold start on two cores.
        # If it still isn't up after that, kubelet restarts the container.
        startupProbe:
          httpGet: { path: /v3/api-docs, port: http }

```

```
    periodSeconds: 10
    failureThreshold: 12

    # readinessProbe: "should the Service send traffic here right now?"
    # Failing readiness pulls the pod out of the Service endpoints; it
    # does NOT restart anything. Cheap and frequent.
    readinessProbe:
      httpGet: { path: /v3/api-docs, port: http }
      periodSeconds: 5
      failureThreshold: 3

    # No livenessProbe on purpose. Liveness restarts the container, and
    # the only thing it would catch beyond readiness is a wedged JVM --
    # rare, and a probe firing during a long GC pause on a 2-core box
    # would kill a healthy pod. Add one later if we ever see a real hang.
    # -----

    resources:
      requests: { memory: 512Mi, cpu: 200m }
      limits: { memory: 768Mi }
      # No CPU limit on purpose. A limit throttles the JVM exactly during
      # class loading, when it wants every core it can get -- turning a
      # 60s start into a 200s one and failing the startupProbe.
  ---
apiVersion: v1
kind: Service
metadata:
  name: course-service
spec:
  selector: { app: course-service }
  ports:
    - { name: http, port: 8082, targetPort: 8082 }
  # NOTE: the direct Ingress that used to live here (ts.local/courses ->
  # course-service) is gone. It was milestone-3 scaffolding to prove the chain
  # end-to-end, but it bypassed the api-gateway's JWT filter. All external
  # traffic now enters through the gateway's Ingress (api-gateway.yaml).
  # In-cluster callers still reach this Service directly -- that is what the
  # Service object above is for.
```

deploy/k8s/base/auth-service.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: auth-service
spec:
  replicas: 1
  selector:
    matchLabels: { app: auth-service }
  template:
    metadata:
      labels: { app: auth-service }
```



```

spec:
  containers:
    - name: auth-service
      image: localhost:5000/auth-service:1.0.0-SNAPSHOT
      imagePullPolicy: Always
      ports:
        - { name: http, containerPort: 8081 }
      env:
        - { name: SPRING_CONFIG_IMPORT, value: "configserver:http://config-server:8888" }
        - { name: EUREKA_CLIENT_SERVICEURL_DEFAULTZONE, value: "http://discovery-server:8761/" }
    ↪ eureka/" }
        - { name: EUREKA_INSTANCE_PREFER_IP_ADDRESS, value: "true" }
        - { name: SPRING_DATASOURCE_URL, value: "jdbc:postgresql://postgres-auth:5432/ts_auth" }

    # Strimzi's bootstrap Service (milestone 6). Producer-only here:
    # registration/activation/admin-provisioned events.
    - { name: SPRING_KAFKA_BOOTSTRAP_SERVERS, value: "ts-kafka-kafka-bootstrap:9092" }

    # Google's OAuth callback must round-trip through the browser, so
    # it uses the PUBLIC name (the Tailscale MagicDNS host, via Traefik
    # -> gateway -> here), not a cluster-internal one. This exact URL
    # must also be registered under "Authorized redirect URIs" in the
    # Google Cloud console, or Google refuses with redirect_uri_mismatch.
    # (ts.local, the LAN-only name, was the value up to build #9.)
    - { name: SPRING_SECURITY_OAUTH2_CLIENT_REGISTRATION_GOOGLE_REDIRECT_URI,
      value: "http://ts-server.tailbfe002.ts.net/login/oauth2/code/google" }
    # After Google login, OAuth2LoginSuccessHandler redirects the
    # browser to the frontend with the tokens. config-server's default
    # says localhost:3000 (laptop Vite dev server); in the cluster the
    # frontend is the same origin as everything else (milestone 7).
    - { name: APP_OAUTH2_SUCCESS_REDIRECT_URI, value: "http://ts-server.tailbfe002.ts.net/" }
    ↪ oauth/callback" }

    # Zipkin arrives with observability (milestone 8).
    - { name: MANAGEMENT_TRACING_SAMPLING_PROBABILITY, value: "0.0" }

    - { name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0" }
  envFrom:
    - secretRef: { name: postgres-auth-app }
    # Signs JWTs with the same key the gateway validates with.
    - secretRef: { name: jwt }
    - secretRef: { name: google-oauth }

    # /actuator/health is NOT usable here: this service's SecurityConfig
    # permitAll list covers /register, /login, ... and /v3/api-docs, but
    # not /actuator/** -- so health answers 401 and the kubelet would
    # restart a perfectly healthy pod forever. Same fallback as
    # course-service: /v3/api-docs only turns 200 after the full context
    # (Flyway migration included) is up, which is an honest ready signal.
  startupProbe:
    httpGet: { path: /v3/api-docs, port: http }
    periodSeconds: 10
    failureThreshold: 12
  readinessProbe:
    httpGet: { path: /v3/api-docs, port: http }
    periodSeconds: 5

```

```
        failureThreshold: 3

    resources:
        requests: { memory: 512Mi, cpu: 200m }
        limits: { memory: 768Mi }
---
apiVersion: v1
kind: Service
metadata:
  name: auth-service
spec:
  selector: { app: auth-service }
  ports:
    - { name: http, port: 8081, targetPort: 8081 }
```

deploy/k8s/base/dictionary-service.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: dictionary-service
spec:
  replicas: 1
  selector:
    matchLabels: { app: dictionary-service }
  template:
    metadata:
      labels: { app: dictionary-service }
    spec:
      containers:
        - name: dictionary-service
          image: localhost:5000/dictionary-service:1.0.0-SNAPSHOT
          imagePullPolicy: Always
          ports:
            - { name: http, containerPort: 8083 }
          env:
            - { name: SPRING_CONFIG_IMPORT, value: "configserver:http://config-server:8888" }
            - { name: EUREKA_CLIENT_SERVICEURL_DEFAULTZONE, value: "http://discovery-server:8761/" }
            ↪ eureka/" }
            - { name: EUREKA_INSTANCE_PREFER_IP_ADDRESS, value: "true" }

            # Config-server's dictionary-service.yml says localhost:27017;
            # in the cluster mongo lives behind the `mongodb` Service.
            - { name: SPRING_DATA_MONGODB_URI, value: "mongodb://mongodb:27017/ts_dictionary" }

            # Zipkin arrives with observability (milestone 8).
            - { name: MANAGEMENT_TRACING_SAMPLING_PROBABILITY, value: "0.0" }

            - { name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0" }

            # This service's SecurityConfig permits /actuator/** -- so unlike
            # course-service (which had to use /v3/api-docs), the probe can hit
```

```

# the purpose-built health endpoint. It aggregates a Mongo ping, so
# "ready" here really means "can serve dictionary lookups".
startupProbe:
  httpGet: { path: /actuator/health, port: http }
  periodSeconds: 10
  failureThreshold: 12
readinessProbe:
  httpGet: { path: /actuator/health, port: http }
  periodSeconds: 5
  failureThreshold: 3

resources:
  requests: { memory: 512Mi, cpu: 200m }
  limits: { memory: 768Mi }
  # No CPU limit -- same class-loading argument as course-service.
---
apiVersion: v1
kind: Service
metadata:
  name: dictionary-service
spec:
  selector: { app: dictionary-service }
  ports:
    - { name: http, port: 8083, targetPort: 8083 }

```

deploy/k8s/base/notification-service.yaml

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: notification-service
spec:
  replicas: 1
  selector:
    matchLabels: { app: notification-service }
  template:
    metadata:
      labels: { app: notification-service }
    spec:
      containers:
        - name: notification-service
          image: localhost:5000/notification-service:1.0.0-SNAPSHOT
          imagePullPolicy: Always
          ports:
            - { name: http, containerPort: 8084 }
          env:
            - { name: SPRING_CONFIG_IMPORT, value: "configserver:http://config-server:8888" }
            - { name: EUREKA_CLIENT_SERVICEURL_DEFAULTZONE, value: "http://discovery-server:8761/
↳ eureka/" }
            - { name: EUREKA_INSTANCE_PREFER_IP_ADDRESS, value: "true" }

          # Strimzi's bootstrap Service: <cluster>-kafka-bootstrap.

```

```
- { name: SPRING_KAFKA_BOOTSTRAP_SERVERS, value: "ts-kafka-kafka-bootstrap:9092" }

# The config's ${MAIL_HOST:localhost} default, repointed at the
# maildev Service. Overriding the spring.mail.* keys directly
# (relaxed binding) rather than the custom placeholder names --
# same result, and consistent with how compose does it.
- { name: SPRING_MAIL_HOST, value: "maildev" }
- { name: SPRING_MAIL_PORT, value: "1025" }

# Links inside emails (activate / invite / login) must work from
# whatever device opens the mail, so they carry the PUBLIC origin
# -- the Tailscale MagicDNS name every tailnet device resolves --
# never a Service name. Overrides app.frontend.base-url from
# config-server (relaxed binding), same trick as SPRING_MAIL_HOST.
- { name: APP_FRONTEND_BASE_URL, value: "http://ts-server.tailbfe002.ts.net" }

# Zipkin arrives with observability (milestone 8).
- { name: MANAGEMENT_TRACING_SAMPLING_PROBABILITY, value: "0.0" }

- { name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0" }

# This service has no Spring Security at all -- no SecurityConfig,
# no security starter -- so /actuator/health is open and the probes
# can use the real endpoint (compare auth-service's workaround).
startupProbe:
  httpGet: { path: /actuator/health, port: http }
  periodSeconds: 10
  failureThreshold: 12
readinessProbe:
  httpGet: { path: /actuator/health, port: http }
  periodSeconds: 5
  failureThreshold: 3

resources:
  requests: { memory: 512Mi, cpu: 100m }
  limits: { memory: 768Mi }
---
# Even a pure consumer gets a Service: Eureka registration aside, the
# actuator endpoints (and later Prometheus scraping) need an address.
apiVersion: v1
kind: Service
metadata:
  name: notification-service
spec:
  selector: { app: notification-service }
  ports:
    - { name: http, port: 8084, targetPort: 8084 }
```

deploy/k8s/base/api-gateway.yaml

```
apiVersion: apps/v1
kind: Deployment
```

```

metadata:
  name: api-gateway
spec:
  replicas: 1
  selector:
    matchLabels: { app: api-gateway }
  template:
    metadata:
      labels: { app: api-gateway }
    spec:
      containers:
        - name: api-gateway
          image: localhost:5000/api-gateway:1.0.0-SNAPSHOT
          imagePullPolicy: Always
          ports:
            - { name: http, containerPort: 8080 }
          env:
            - { name: SPRING_CONFIG_IMPORT, value: "configserver:http://config-server:8888" }
            - { name: EUREKA_CLIENT_SERVICEURL_DEFAULTZONE, value: "http://discovery-server:8761/" }
            ↪ eureka/" }
            - { name: EUREKA_INSTANCE_PREFER_IP_ADDRESS, value: "true" }

            # Zipkin is not in the cluster yet (observability is milestone 8).
            - { name: MANAGEMENT_TRACING_SAMPLING_PROBABILITY, value: "0.0" }

            - { name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=75.0" }
          envFrom:
            # JWT_SECRET -- shared with auth-service, which signs the tokens the
            # gateway validates. One secret, two consumers, or logins break in a
            # way that looks like a gateway bug.
            - secretRef: { name: jwt }

            # Unlike course-service, the gateway exposes actuator, so probes can
            # use the real health endpoint. Reactive stack (Netty) boots faster
            # than the MVC services: 6 x 10s of startup grace instead of 12.
          startupProbe:
            httpGet: { path: /actuator/health, port: http }
            periodSeconds: 10
            failureThreshold: 6
          readinessProbe:
            httpGet: { path: /actuator/health, port: http }
            periodSeconds: 5
            failureThreshold: 3

          resources:
            requests: { memory: 384Mi, cpu: 100m }
            limits: { memory: 512Mi }
      ---
    apiVersion: v1
    kind: Service
    metadata:
      name: api-gateway
    spec:
      selector: { app: api-gateway }
      ports:
        - { name: http, port: 8080, targetPort: 8080 }

```

```
---
# Edge routing decision: with the gateway in the cluster there are two routers.
# Traefik (this Ingress) is the front door on ts.local; Spring Cloud Gateway
# knows Eureka and validates JWTs. Decision made here:
#
# /api          -> gateway    (all REST traffic, JWT-checked)
# /oauth2       -> gateway    (Google login redirect start)
# /login/oauth2 -> gateway    (Google login callback)
# /             -> free       (reserved for the frontend, milestone 7)
#
# The old direct `courses` Ingress on course-service is deleted with this
# change: it reached the service without passing the gateway's JWT filter,
# i.e. an unauthenticated side door. From now on the ONLY way in is through
# the gateway -- which is the entire point of having one.
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: api-gateway
spec:
  # Two hosts, identical paths. ts.local is the LAN name (hosts-file entry on
  # the laptop). The second is the machine's Tailscale MagicDNS name: every
  # device on the tailnet resolves it with no hosts-file edit, so the app is
  # reachable from anywhere Tailscale is -- the same Ingress, one more rule.
  # Traefik routes on the Host header, so a name it does not list gets 404.
  rules:
    - host: ts.local
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: api-gateway
                port: { number: 8080 }
          - path: /oauth2
            pathType: Prefix
            backend:
              service:
                name: api-gateway
                port: { number: 8080 }
          - path: /login/oauth2
            pathType: Prefix
            backend:
              service:
                name: api-gateway
                port: { number: 8080 }
    - host: ts-server.tailbfe002.ts.net
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: api-gateway
                port: { number: 8080 }
          - path: /oauth2
```

```

    pathType: Prefix
    backend:
      service:
        name: api-gateway
        port: { number: 8080 }
  - path: /login/oauth2
    pathType: Prefix
    backend:
      service:
        name: api-gateway
        port: { number: 8080 }

```

deploy/k8s/base/frontend.yaml

```

# Frontend (milestone 7): nginx serving the built React app.
#
# Image comes from frontend/Dockerfile (docker build in Jenkins), not Jib --
# it is the one non-JVM deployable. Being static files it has no config
# server, no Eureka, no secrets: the app talks to "/api" on its own origin,
# and Traefik (Ingress below) splits that off to the gateway.
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
spec:
  replicas: 1
  selector:
    matchLabels: { app: frontend }
  template:
    metadata:
      labels: { app: frontend }
    spec:
      containers:
        - name: frontend
          image: localhost:5000/frontend:latest
          imagePullPolicy: Always
          ports:
            - { name: http, containerPort: 80 }

          # nginx is up in milliseconds; no startupProbe needed. /healthz is
          # a location in nginx.conf that answers without touching disk.
          readinessProbe:
            httpGet: { path: /healthz, port: http }
            periodSeconds: 5
          livenessProbe:
            httpGet: { path: /healthz, port: http }
            periodSeconds: 20

          # Static file serving: the cheapest pod in the namespace by far.
          resources:
            requests: { memory: 32Mi, cpu: 10m }
            limits: { memory: 64Mi }

```

```
---
apiVersion: v1
kind: Service
metadata:
  name: frontend
spec:
  selector: { app: frontend }
  ports:
    - { name: http, port: 80, targetPort: 80 }
---
# Claims "/" on ts.local -- the path api-gateway.yaml's Ingress left free.
# Traefik merges all Ingress rules for a host and picks the LONGEST matching
# prefix, so /api, /oauth2 and /login/oauth2 (declared on the gateway
# Ingress) still win over this catch-all. Net effect: one origin
# (http://ts.local) for pages and API alike -- no CORS preflights, and the
# Google-login redirect lands on the same host that started it.
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: frontend
spec:
  # Two hosts, identical paths. ts.local is the LAN name (hosts-file entry on
  # the laptop). The second is the machine's Tailscale MagicDNS name: every
  # device on the tailnet resolves it with no hosts-file edit, so the app is
  # reachable from anywhere Tailscale is -- the same Ingress, one more rule.
  # Traefik routes on the Host header, so a name it does not list gets 404.
  rules:
    - host: ts.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend
                port: { number: 80 }
    - host: ts-server.tailbfe002.ts.net
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend
                port: { number: 80 }
```

frontend/Dockerfile

```
# Frontend image: the ONLY Dockerfile the pipeline builds.
#
# The JVM services use Jib (no Dockerfile, no daemon). A React app is a pile
# of static files behind a web server, so it gets the classic two-stage
```



```
# build instead: stage 1 has Node and produces dist/, stage 2 is a plain
# nginx that ships only dist/ -- Node, node_modules (~300 MB) and the
# TypeScript compiler never reach the runtime image (~15 MB).
#
# docker build -t localhost:5000/frontend:dev frontend
# docker run --rm -p 3000:80 localhost:5000/frontend:dev -> http://localhost:3000

# ---- stage 1: build -----
FROM node:24-alpine AS build
WORKDIR /app

# Copy the lockfile first and install BEFORE copying sources: this layer is
# cached until package*.json changes, so a source edit does not re-download
# every dependency. `npm ci` (not `install`) refuses to touch the lockfile,
# i.e. the build gets exactly the versions committed in git.
COPY package.json package-lock.json ./
RUN npm ci --no-audit --no-fund

COPY . .
# = `tsc -b && vite build` (package.json). tsc type-checks, Vite bundles and
# content-hashes the assets (index-<hash>.js) so they can be cached forever.
RUN npm run build

# ---- stage 2: serve -----
FROM nginx:1.27-alpine
# The stock image ships a default server block; replace it with ours.
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
```

frontend/nginx.conf

```
# nginx server block for the React frontend inside the cluster.
#
# Division of labour on ts.local (Traefik decides by longest path prefix):
# /api, /oauth2, /login/oauth2 -> api-gateway (never reach this nginx)
# everything else -> here (static files + SPA fallback)
# So this file does NOT proxy anything: in production the frontend's
# "/api" calls are same-origin and Traefik, not nginx, routes them. The
# Vite dev-server proxy in vite.config.ts plays that role only on a laptop.
server {
    listen 80;
    root /usr/share/nginx/html;
    index index.html;

    # SPA fallback. React Router (BrowserRouter) owns URLs like /courses/42
    # purely in the browser -- no such file exists on disk. On a deep link or
    # a page refresh nginx must answer index.html and let the router take
    # over, otherwise the user gets a 404 for any URL except "/".
    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

```
# Vite content-hashes everything under /assets (index-CqZING8N.js), so a
# changed file always has a new name: cache these forever. index.html is
# deliberately NOT covered -- it must be re-fetched so it can point at
# the new hashes after a deploy.
location /assets/ {
    add_header Cache-Control "public, max-age=31536000, immutable";
}

# Kubernetes probes hit this; keeps "is nginx up" separate from "/",
# which would also succeed but pollutes access logs less this way.
location = /healthz {
    access_log off;
    add_header Content-Type text/plain;
    return 200 "ok\n";
}

gzip on;
gzip_types text/css application/javascript application/json image/svg+xml;
}
```

docker-compose.yml

```
services:
  # ===== INFRASTRUCTURE =====
  postgres-auth:
    image: postgres:17
    environment:
      POSTGRES_DB: ts_auth
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: root
    ports:
      - "5433:5432"
    volumes:
      - postgres-auth-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      retries: 5

  postgres-course:
    image: postgres:17
    environment:
      POSTGRES_DB: ts_course
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: root
    ports:
      - "5434:5432"
    volumes:
      - postgres-course-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
```

```

    interval: 10s
    retries: 5

mongodb:
  image: mongo:8
  ports:
    - "27017:27017"
  volumes:
    - mongo-data:/data/db

kafka:
  image: apache/kafka:3.9.0
  hostname: kafka
  ports:
    - "9092:9092"
  environment:
    KAFKA_NODE_ID: 1
    KAFKA_PROCESS_ROLES: broker,controller
    KAFKA_LISTENERS: PLAINTEXT://:19092,CONTROLLER://:9093,EXTERNAL://:9092
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:19092,EXTERNAL://localhost:9092
    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,EXTERNAL:
    ↪ PLAINTEXT
    KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
    KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
    KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
    KAFKA_LOG_RETENTION_HOURS: 168
    CLUSTER_ID: MKU30EVBNTcwNTJENDM2Qk

kafka-ui:
  image: provectuslabs/kafka-ui:latest
  ports:
    - "9090:8080"
  environment:
    KAFKA_CLUSTERS_0_NAME: local
    KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: kafka:19092
  depends_on:
    - kafka

zipkin:
  image: openzipkin/zipkin:latest
  ports:
    - "9411:9411"

maildev:
  image: maildev/maildev:2.2.1
  ports:
    - "1080:1080"
    - "1025:1025"

minio:
  image: minio/minio:latest
  command: server /data --console-address ":9001"
  ports:
    - "9000:9000"    # S3 API
    - "9001:9001"    # Web console

```

```

environment:
  MINIO_ROOT_USER: minioadmin
  MINIO_ROOT_PASSWORD: minioadmin
volumes:
  - minio-data:/data
healthcheck:
  test: ["CMD", "mc", "ready", "local"]
  interval: 10s
  retries: 5

# ===== SPRING CLOUD =====
config-server:
  build:
    context: ./config-server
    dockerfile: Dockerfile
  ports:
    - "8888:8888"
    - "5888:5888"
  environment:
    JAVA_TOOL_OPTIONS: "-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=*:5888"
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:8888/actuator/health"]
    interval: 15s
    timeout: 5s
    retries: 10
    start_period: 30s

discovery-server:
  build:
    context: ./discovery-server
    dockerfile: Dockerfile
  ports:
    - "8761:8761"
    - "5761:5761"
  depends_on:
    config-server:
      condition: service_healthy
  environment:
    SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
    JAVA_TOOL_OPTIONS: "-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=*:5761"

api-gateway:
  build:
    context: ./api-gateway
    dockerfile: Dockerfile
  ports:
    - "8080:8080"
    - "5080:5080"
  depends_on:
    - discovery-server
  environment:
    SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
    EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://discovery-server:8761/eureka/
    MANAGEMENT_ZIPKIN_TRACING_ENDPOINT: http://zipkin:9411/api/v2/spans
    JAVA_TOOL_OPTIONS: "-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=*:5080"

```

```
# ===== APPLICATION SERVICES =====
auth-service:
  build:
    context: ./auth-service
    dockerfile: Dockerfile
  ports:
    - "8081:8081"
    - "5081:5081"
  depends_on:
    config-server:
      condition: service_healthy
    postgres-auth:
      condition: service_healthy
    kafka:
      condition: service_started
  environment:
    SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
    EURKA_CLIENT_SERVICEURL_DEFAULTZONE: http://discovery-server:8761/eureka/
    SPRING_DATASOURCE_URL: jdbc:postgresql://postgres-auth:5432/ts_auth
    SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:19092
    MANAGEMENT_ZIPKIN_TRACING_ENDPOINT: http://zipkin:9411/api/v2/spans
    GOOGLE_CLIENT_ID: ${GOOGLE_CLIENT_ID}
    GOOGLE_CLIENT_SECRET: ${GOOGLE_CLIENT_SECRET}
    JAVA_TOOL_OPTIONS: "-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=*:5081"

course-service:
  build:
    context: ./course-service
    dockerfile: Dockerfile
  ports:
    - "8082:8082"
    - "5082:5082"
  depends_on:
    config-server:
      condition: service_healthy
    postgres-course:
      condition: service_healthy
    kafka:
      condition: service_started
    minio:
      condition: service_started
  environment:
    SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
    EURKA_CLIENT_SERVICEURL_DEFAULTZONE: http://discovery-server:8761/eureka/
    SPRING_DATASOURCE_URL: jdbc:postgresql://postgres-course:5432/ts_course
    SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:19092
    MANAGEMENT_ZIPKIN_TRACING_ENDPOINT: http://zipkin:9411/api/v2/spans
    APP_S3_ENDPOINT: http://minio:9000
    APP_S3_PUBLIC_ENDPOINT: http://localhost:9000
    APP_S3_ACCESS_KEY: minioadmin
    APP_S3_SECRET_KEY: minioadmin
    JAVA_TOOL_OPTIONS: "-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=*:5082"

dictionary-service:
  build:
    context: ./dictionary-service
```

```
dockerfile: Dockerfile
ports:
  - "8083:8083"
  - "5083:5083"
depends_on:
  config-server:
    condition: service_healthy
  mongodb:
    condition: service_started
environment:
  SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
  EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://discovery-server:8761/eureka/
  SPRING_DATA_MONGODB_URI: mongodb://mongodb:27017/ts_dictionary
  MANAGEMENT_ZIPKIN_TRACING_ENDPOINT: http://zipkin:9411/api/v2/spans
  JAVA_TOOL_OPTIONS: "-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=*:5083"

notification-service:
  build:
    context: ./notification-service
    dockerfile: Dockerfile
  ports:
    - "8084:8084"
    - "5084:5084"
  depends_on:
    config-server:
      condition: service_healthy
    kafka:
      condition: service_started
  environment:
    SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
    EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://discovery-server:8761/eureka/
    SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:19092
    SPRING_MAIL_HOST: maildev
    SPRING_MAIL_PORT: 1025
    MANAGEMENT_ZIPKIN_TRACING_ENDPOINT: http://zipkin:9411/api/v2/spans
    JAVA_TOOL_OPTIONS: "-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=*:5084"

volumes:
  postgres-auth-data:
  postgres-course-data:
  mongo-data:
  minio-data:
```

config-server: configurations/api-gateway.yml

```
server:
  port: 8080
spring:
  cloud:
    gateway:
      routes:
        - id: auth-service
```

```

    uri: lb://auth-service
    predicates:
      - Path=/api/auth/**
    filters:
      - StripPrefix=2
  - id: auth-oauth2
    uri: lb://auth-service
    predicates:
      - Path=/oauth2/**
  - id: auth-oauth2-callback
    uri: lb://auth-service
    predicates:
      - Path=/login/oauth2/**
  - id: course-service-courses
    uri: lb://course-service
    predicates:
      - Path=/api/courses/**
    filters:
      - StripPrefix=2
  - id: course-service-students
    uri: lb://course-service
    predicates:
      - Path=/api/students/**
    filters:
      - StripPrefix=2
  - id: course-service-assignments
    uri: lb://course-service
    predicates:
      - Path=/api/assignments/**
    filters:
      - StripPrefix=2
  - id: course-service-enrollments
    uri: lb://course-service
    predicates:
      - Path=/api/enrollments/**
    filters:
      - StripPrefix=2
  - id: course-service-submissions
    uri: lb://course-service
    predicates:
      - Path=/api/submissions/**
    filters:
      - StripPrefix=2
  - id: dictionary-service
    uri: lb://dictionary-service
    predicates:
      - Path=/api/dictionary/**
    filters:
      - StripPrefix=2
app:
  jwt:
    secret: ${JWT_SECRET:mySecretKeyThatIsAtLeast256BitsLongForHS256Algorithm123456}
management:
  tracing:
    sampling:
      probability: 1.0

```

```
zipkin:
  tracing:
    endpoint: http://localhost:9411/api/v2/spans
endpoints:
  web:
    exposure:
      include: health,info,metrics,prometheus
logging:
  pattern:
    level: "%5p [%${spring.application.name:},%X{traceId:-},%X{spanId:-}]"
```

config-server: configurations/auth-service.yml

```
server:
  port: 8081
  error:
    include-message: always
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/ts_auth
    username: postgres
    password: root
    driver-class-name: org.postgresql.Driver
  jpa:
    hibernate:
      ddl-auto: none
      show-sql: true
    properties:
      hibernate:
        format_sql: true
        dialect: org.hibernate.dialect.PostgreSQLDialect
  flyway:
    enabled: true
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
  security:
    oauth2:
      client:
        registration:
          google:
            client-id: ${GOOGLE_CLIENT_ID:placeholder}
            client-secret: ${GOOGLE_CLIENT_SECRET:placeholder}
            scope: openid, profile, email
            redirect-uri: http://localhost:8080/login/oauth2/code/google
  app:
    jwt:
      secret: ${JWT_SECRET:mySecretKeyThatIsAtLeast256BitsLongForHS256Algorithm123456}
      access-token-expiry-ms: 900000
      refresh-token-expiry-ms: 604800000
```



```

totp:
  issuer: TeacherSupporter
activation:
  base-url: http://localhost:3000/activate
oauth2:
  success-redirect-uri: http://localhost:3000/oauth/callback
management:
  tracing:
    sampling:
      probability: 1.0
  zipkin:
    tracing:
      endpoint: http://localhost:9411/api/v2/spans
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
logging:
  pattern:
    level: "%5p [%${spring.application.name:},%X{traceId:-},%X{spanId:-}]"

```

config-server: configurations/course-service.yml

```

server:
  port: 8082
  error:
    include-message: always
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/ts_course
    username: postgres
    password: root
    driver-class-name: org.postgresql.Driver
  jpa:
    hibernate:
      ddl-auto: none
      show-sql: true
    properties:
      hibernate:
        format_sql: true
        dialect: org.hibernate.dialect.PostgreSQLDialect
  flyway:
    enabled: true
  servlet:
    multipart:
      max-file-size: 20MB
      max-request-size: 25MB
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer

```

```

consumer:
  group-id: course-group
  auto-offset-reset: earliest
  key-deserializer: org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
  value-deserializer: org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
  properties:
    spring.deserializer.key.delegate.class: org.apache.kafka.common.serialization.
    ↪ StringDeserializer
    spring.deserializer.value.delegate.class: org.springframework.kafka.support.serializer.
    ↪ JsonDeserializer
  spring.json.trusted.packages: com.ts.common.dto
app:
  s3:
    endpoint: ${APP_S3_ENDPOINT:http://localhost:9000}
    # Browser-reachable endpoint used to sign download URLs. When the service runs
    # inside Docker the internal endpoint is minio:9000, which a browser cannot resolve.
    public-endpoint: ${APP_S3_PUBLIC_ENDPOINT:http://localhost:9000}
    region: ${APP_S3_REGION:us-east-1}
    access-key: ${APP_S3_ACCESS_KEY:minioadmin}
    secret-key: ${APP_S3_SECRET_KEY:minioadmin}
    bucket: ${APP_S3_BUCKET:ts-submissions}
    presigned-url-expiry-minutes: ${APP_S3_PREIGNED_EXPIRY_MIN:15}
  resilience4j:
    circuitbreaker:
      instances:
        authService:
          sliding-window-size: 10
          failure-rate-threshold: 50
          wait-duration-in-open-state: 10s
          permitted-number-of-calls-in-half-open-state: 3
  management:
    tracing:
      sampling:
        probability: 1.0
    zipkin:
      tracing:
        endpoint: http://localhost:9411/api/v2/spans
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
  logging:
    pattern:
      level: "%5p [%${spring.application.name:},%X{traceId:-},%X{spanId:-}]"

```

config-server: configurations/dictionary-service.yml

```

server:
  port: 8083
  error:
    include-message: always
spring:

```

```

data:
  mongodb:
    uri: mongodb://localhost:27017/ts_dictionary
management:
  tracing:
    sampling:
      probability: 1.0
zipkin:
  tracing:
    endpoint: http://localhost:9411/api/v2/spans
endpoints:
  web:
    exposure:
      include: health,info,metrics,prometheus
logging:
  pattern:
    level: "%5p [%${spring.application.name:},%X{traceId:-},%X{spanId:-}]"

```

config-server: configurations/notification-service.yml

```

server:
  port: 8084
  error:
    include-message: always
spring:
  kafka:
    bootstrap-servers: localhost:9092
    consumer:
      group-id: notification-group
      auto-offset-reset: earliest
      key-deserializer: org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
      properties:
        spring.deserializer.key.delegate.class: org.apache.kafka.common.serialization.
        ↪ StringDeserializer
        spring.deserializer.value.delegate.class: org.springframework.kafka.support.serializer.
        ↪ JsonDeserializer
        spring.json.trusted.packages: com.ts.common.dto
  mail:
    host: ${MAIL_HOST:localhost}
    port: ${MAIL_PORT:1025}
    username: ${MAIL_USERNAME:}
    password: ${MAIL_PASSWORD:}
    properties:
      mail.smtp.auth: false
      mail.smtp.starttls.enable: false
# Where the links inside outgoing emails point. This is the URL a BROWSER
# uses to reach the frontend, so it differs per environment: the Vite dev
# server here and under docker-compose, the public Ingress host in the
# cluster (overridden there with the APP_FRONTEND_BASE_URL env var).
app:
  frontend:

```

```
    base-url: http://localhost:3000
management:
  tracing:
    sampling:
      probability: 1.0
  zipkin:
    tracing:
      endpoint: http://localhost:9411/api/v2/spans
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
logging:
  pattern:
    level: "%5p [%${spring.application.name:},%X{traceId:-},%X{spanId:-}]"
```